The Category of Containers

a living reference

MacBeth

Draft, June 2026, v0.1

Contents

Preface iii

1 Containers and their extension functor 1
- 1.1 Containers . . . 1
- 1.2 The representation theorem . . . 1

2 The category Cont 3
- 2.1 Container morphisms: the variance . . . 3
- 2.2 Worked example: `tail` . . . 3
- 2.3 An application vista: Weihrauch problems . . . 4

3 Which functors are containers? 5
- 3.1 A container is a family of sets . . . 5
- 3.2 The test . . . 6
- 3.3 Reading the positions back off . . . 8
- 3.4 Limits and colimits in **Cont** . . . 9
- 3.5 Two witnesses that are genuine (co)monads . . . 10
- 3.6 The boundary of the theory . . . 12

4 Algebraic structure on Cont 15
- 4.1 Products and coproducts . . . 15
- 4.2 The monoidal menagerie . . . 15
- 4.3 Closing the structures . . . 16
  - 4.3.1 The Dirichlet closure, as a hom of morphisms . . . 16
  - 4.3.2 The same hom, read as a product of composites . . . 17
  - 4.3.3 When is a convolutional tensor closed? . . . 17
  - 4.3.4 Is the condition ever really a condition? . . . 18
  - 4.3.5 The complete list . . . 20
  - 4.3.6 The other three, and a boundary . . . 22
- 4.4 Monoids and comonoids for the four structures . . . 23
  - 4.4.1 The composition product: categories and monads . . . 24
  - 4.4.2 The Dirichlet tensor: a lax/oplax duality . . . 24
- 4.5 An aside: the lens subcategory . . . 25
- 4.6 Two predicate liftings: All and Exists . . . 26
  - 4.6.1 The two liftings, defined . . . 26
  - 4.6.2 The first surprise: E is the composition product . . . 27
  - 4.6.3 The second surprise: A lives only on cartesian morphisms . . . 27
  - 4.6.4 The law: A is a module of $\triangleleft$ . . . 28
  - 4.6.5 Where this points . . . 29

5 Monoids and comonoids in Cont: directed containers, categories, and the free monoid **31**
5.1 Directed containers . . . 31
5.2 Directed containers are comonads . . . 31
5.3 Directed containers are small categories . . . 32
5.4 The morphism refinement: **DCont** ≅ **Cof** . . . 32
5.5 The dual side: monoids in **Cont** and the free monoid . . . 33

6 Monads and comonads: the free and cofree constructions **35**
6.1 The tools we borrow: initial algebras and final coalgebras . . . 36
6.2 W-types and their duals . . . 36
6.3 The free monad of a container . . . 37
6.4 The free-monad Lemma . . . 38
6.5 The cofree comonad, and cofree = cofree directed container . . . 39
6.5.1 The cofree-comonad Lemma . . . 40
6.6 Monads on the base, comonads on **Cont**: the transfer . . . 42
6.6.1 The two feeds entwine . . . 45
6.6.2 The fibrational picture: why the two feeds had to be what they are . . . 48
6.6.3 Which monads lift: the Σ-lifting is $M \triangleleft (-)$ . . . 53
6.6.4 Which liftings, all of them: monad liftings are comonads over the base . . . 55
6.6.5 The State pole: the store multiplication is invisible . . . 58
6.6.6 The general law: holonomy, and the two poles as its degeneracies . . . 60
6.7 Processes with state: the store comonad and the category of Workers . . . 63
6.7.1 The state object and the store comonad . . . 63
6.7.2 Workers, and why the state multiplies . . . 64
6.8 Syntax and behaviour . . . 66

7 Composing systems: Zappa–Szép and distributive laws **69**
7.1 The pairwise criterion . . . 69
7.2 (G) as a strict factorisation system, and its H^2 obstruction . . . 69
7.3 The classical anchor . . . 70
7.4 Three modes of composition . . . 70
7.5 When two of the modes meet: emergent holonomy . . . 73

8 Functors over the category of containers (Phase 2 — outline) **79**

A Reading & citation tracker **81**

Preface

Purpose of this document

This is a single, evolving reference centred on *the category of containers*. It is meant to grow: chapters are added and revised in place, and the whole document compiles to one PDF that can be read end to end or consulted by section. The organising thesis is the equivalence chain

Containers $\simeq$ **Directed Containers** $\simeq$ **Polynomial Comonoids** $\simeq$ **Small Categories**,

which we take as the foundation for *verified composition*: once a system (a supply chain, an agent hierarchy, a state space, a tax computation) is presented as a container, its composition is governed by the structure of **Cont** and of the comonoids inside it, and correctness of composition becomes a statement that can be proved — and, increasingly, machine-checked. Object level equivalence is settled groundwork; the live mathematics lives at the *morphism* level, and that is where this book spends most of its effort.

The flagging convention

Mathematical honesty is the first principle of this document. Every **Definition** and every result is flagged, and every **Theorem**/**Proposition**/**Lemma**/**Corollary** statement *ends with a bracketed provenance tag*, one of:

- [Cited: *author year, ref*] — a known result from the literature, not claimed as new;
- [MacBeth] — claimed and/or proved by MacBeth, informally (not machine-checked);
- [MacBeth, Lean-verified: *file*] — machine-checked in the repository's `lean/` tree, in the named source file;
- [Conjecture, MacBeth] or [Open] — for conjectures.

A tag [Cited: . . .] on a statement that MacBeth has also verified independently will say so; when in doubt the weaker (cited, not-checked) tag is used, and a footnote records the uncertainty. The bias is deliberately towards *under*-claiming. The reading and citation tracker in Appendix A records, paper by paper, how thoroughly each source has been engaged.

A note on the Lean tags. Only two Lean *source* files are committed to the repository at the time of this draft: `Basic.lean` (the container representation theorem) and `Directed.lean` (the comonad laws from the directed-container laws). Results checked in those files carry the Lean-verified tag naming the committed file. Where MacBeth has a Lean formalisation that is *not* yet committed as source to this repository (notably the **DCont** $\cong$ **Cof** and Zappa–Szép associativity developments), the statement is tagged [Cited] for the underlying theorem and a footnote flags that the Lean source is not in this tree — so the reader is never told something is machine-checked here when its source cannot be found here.

Chapter 1

Containers and their extension functor

1.1 Containers

Definition 1.1 (Container)**.** A *container* is a pair $S \triangleleft P$ where S is a set of *shapes* and $P\colon S \to \mathbf{Set}$ is a family of *positions*, one set $P(s)$ for each shape s. Its *extension* (or *interpretation*) is the functor $[\![S \triangleleft P]\!]\colon \mathbf{Set} \to \mathbf{Set}$ defined on objects by

$$[\![S \triangleleft P]\!](X) \;=\; \coprod_{s \in S} X^{P(s)} \;=\; \big\{(s, v) \mid s \in S,\; v\colon P(s) \to X\big\},$$

and on a morphism $f\colon X \to Y$ by postcomposition, $[\![S \triangleleft P]\!](f)(s, v) = (s,\; f \circ v)$.

Unpacking: a shape s is a structure with its positions $P(s)$; a labelling $v\colon P(s) \to X$ fills every position with an element of X; the coproduct over s is the disjoint union "pick exactly one shape". The functorial action relabels — it changes the data at the positions but never the shape, so the *shape projection* $\pi(s, v) = s$ is a natural transformation to the constant functor at S. A container thus separates *structure* (the shape) from *content* (the labels), and its functorial action is exactly `map`.

Example 1.2 (The container menagerie)**. Maybe**: two shapes, "nothing" ($P = \varnothing$, term $X^0 = 1$) and "just" ($P = \{*\}$, term X); extension $1 + X$. **Lists**: shape $n \in \mathbb{N}$, positions $\{0, \dots, n-1\}$; extension $\sum_n X^n$. **Binary trees**: a shape is a planar binary tree, positions are leaves; the generating function satisfies $T(X) = 1 + X \cdot T(X)^2$, so the number of shapes with n positions is the Catalan number $C_n = \frac{1}{n+1}\binom{2n}{n}$. **Matrices**: shape (m, n), positions the mn entries; extension $\sum_{m,n} X^{m \times n}$. In each case the coefficient of X^n in the generating function counts *how many* shapes have n positions, while the container remembers *which* shapes and *where* the positions are.

A standing intuition ("decategorification"): writing $\sum_s X^{P(s)}$ as if it were a polynomial in a formal indeterminate X guides the right intuitions about products, composition and derivatives. But X is a *set*, not an indeterminate: $X^{P(s)} = \mathrm{Hom}(P(s), X)$ is a set of functions, and the extension lives in $[\mathbf{Set}, \mathbf{Set}]$, not in a ring of power series.

1.2 The representation theorem

The combinatorial data of containers captures *exactly* the natural transformations between their extensions.

Theorem 1.3 (Representation / full faithfulness)**.** *The extension functor* $[\![-]\!]\colon \mathbf{Cont} \to [\mathbf{Set}, \mathbf{Set}]$ *is full and faithful: every natural transformation* $[\![S_1 \triangleleft P_1]\!] \Rightarrow [\![S_2 \triangleleft P_2]\!]$ *arises from a unique container morphism, and distinct morphisms induce distinct natural transformations.* *[Cited: Abbott–Altenkirch–Ghani 2005]*[1]

Proof sketch. By the Yoneda lemma a natural transformation α is determined by its action on the "generic element" $(s, \mathrm{id}_{P_1(s)}) \in [\![S_1 \triangleleft P_1]\!](P_1(s))$ of each shape: $\alpha_{P_1(s)}(s, \mathrm{id}) = (u(s), f_s)$ reads off both a shape map $u\colon S_1 \to S_2$ and a position map $f_s\colon P_2(u(s)) \to P_1(s)$, and naturality forces every other component. Conversely every (u, f) is natural (Chapter 2). See [1] for the full argument; the round-trip is machine-checked in `Basic.lean`. □

[1] Independently formalised by MacBeth in the committed Lean file `lean/Containers/Containers/Basic.lean` (round-trip `ContainerMorphism` ↔ `ExtNatTrans`, no `sorry`).

Chapter 2

The category Cont

2.1 Container morphisms: the variance

Definition 2.1 (Container morphism)**.** A *container morphism* $(u, f)\colon S_1 \triangleleft P_1 \to S_2 \triangleleft P_2$ consists of

- a *shape map* $u\colon S_1 \to S_2$ (*forward*), and
- for each $s \in S_1$, a *position map* $f_s\colon P_2(u(s)) \to P_1(s)$ (*backward*).

It induces the natural transformation $\alpha_X(s, v) = \big(u(s),\, v \circ f_s\big)$.

The variance is not a convention but a necessity. To build a new labelling of the *output* positions $P_2(u(s))$ out of an old labelling $v\colon P_1(s) \to X$ without inspecting labels, the only move is to precompose:

$$P_2(u(s)) \xrightarrow{f_s} P_1(s) \xrightarrow{v} X,$$

which forces f_s to run from output positions to input positions. Each output position p' asks "which input position supplies my label?" and the answer is $f_s(p')$. Consequently a container morphism can only *rearrange*, *discard* or *duplicate* labels — never *create* one. (Sorting, filtering and thresholding are *not* container morphisms: they inspect labels, and so are not natural in X.)

Definition 2.2 (Identity and composition)**.** The identity on $S \triangleleft P$ is $(\mathrm{id}_S, \mathrm{id})$. The composite of $(u, f)\colon S_1 \triangleleft P_1 \to S_2 \triangleleft P_2$ and $(u', f')\colon S_2 \triangleleft P_2 \to S_3 \triangleleft P_3$ has shape map $u' \circ u$ and position map running backward through both stages,

$$(u' \circ u)^\sharp_s \;=\; f_s \circ f'_{u(s)}\colon P_3\big(u'(u(s))\big) \xrightarrow{f'_{u(s)}} P_2(u(s)) \xrightarrow{f_s} P_1(s).$$

These satisfy the category axioms; the resulting category is **Cont**.

Proposition 2.3 (Naturality and the correspondence)**.** *For every container morphism (u, f), the family $\alpha_X(s, v) = (u(s), v \circ f_s)$ is natural in X; and (Theorem 1.3) every natural transformation between container extensions is α for a unique (u, f). Hence $[\![-]\!]\colon$ **Cont** $\to [$**Set**, **Set**$]$ is an isomorphism onto its image.* *[Cited: AAG05]*

Proof. Naturality is associativity of composition: for $g\colon X \to Y$, both ways round the naturality square send (s, v) to $(u(s),\, g \circ v \circ f_s)$, because f_s does not look at labels. Full faithfulness is Theorem 1.3. □

2.2 Worked example: `tail`

The function $\mathrm{tail}\colon \mathrm{List}(X) \to 1 + \mathrm{List}(X)$ is a container morphism (u, f): the shape map sends length n to length $n - 1$ (and 0 to "nothing"), and the (backward) position map sends position

i of the output to position $i+1$ of the input. The same (u, f) acts on lists of integers, strings, or anything — the labels are shifted one place left, never inspected. This is the content of naturality: the morphism is parametric in the label type.

2.3 An application vista: Weihrauch problems

Before leaving the bare category, it is worth flagging one place where these two ingredients — a set of shapes with a family of positions, and morphisms that run forward on shapes and backward on positions — have recently turned up far from data structures. In computable analysis a *Weihrauch problem* is a multi-valued task: a set of *instances*, and for each instance a set of admissible *solutions*. That is precisely a container — instances are shapes, solutions are positions — and a *Weihrauch reduction*, the standard notion of "problem A is no harder than problem B", is exactly a container morphism: forward on instances (translate an A-instance to a B-instance), backward on solutions (translate a B-solution back to an A-solution), the same variance for the same reason. Pradic and Price make this identification precise [41]: their category of problems is a category of containers over a base of realizability assemblies, and — pleasingly for us — their sequential composition of problems is written with the very symbol $\triangleleft$ we will give to the composition product in Chapter 4, the Weihrauch star. Their base is only *weakly* locally cartesian closed, so their composition is a mere quasi-bifunctor; the theory of this book lives over **Set**, which is genuinely locally cartesian closed, and one lesson of the comparison is to see exactly where that hypothesis earns its keep. We record the connection as a vista, not a thread we pursue here: the container calculus that follows is, unexpectedly, also a calculus of computational hardness.

Chapter 3

Which functors are containers?

Chapter 1 proved that the extension functor $[\![-]\!]\colon \mathbf{Cont} \to [\mathbf{Set}, \mathbf{Set}]$ is full and faithful (Theorem 1.3): the combinatorics of containers captures *exactly* the natural transformations between their extensions. Full and faithful means $[\![-]\!]$ is an isomorphism *onto its image*. It does not tell us what that image *is*. This chapter closes the gap. It answers, exactly, the question a reader should have been asking since the definition:

Hand me a functor $F\colon \mathbf{Set} \to \mathbf{Set}$. *Is it a container?*

Why the question has teeth. It is not idle taxonomy. Look ahead to the cofree comonad of Chapter 6: to build the behaviours of a system — its infinite unfoldings — one forms the *terminal sequence*

$$1 \longleftarrow F1 \longleftarrow FF1 \longleftarrow FFF1 \longleftarrow \cdots$$

and takes its limit, where the final coalgebra lives. That limit is a *cofiltered* one, and the whole construction works only if F *preserves* it. So before we can compose behaviours we are forced to ask which functors preserve limits of this kind. The answer, we will see, is precisely: the containers. The equivalence chain, the comonoids, the monoidal structures of the later chapters — the entire programme of this book lives *inside* the image of $[\![-]\!]$, so knowing which functors lie in that image is knowing the exact reach of the theory: what it covers, and where its boundary runs.

The promise. By the end of the chapter you will be able to look at a functor and decide whether it is a container; you will know how to read its shapes and positions back off; and you will see why the first formula anyone guesses for the positions is wrong. The tools are one structural identification and one theorem — the *wide-pullback* (connected-limit) characterisation — and both are best met after a little computation.

3.1 A container is a family of sets

Before we test functors, we should see **Cont** for what it is. Strip the extension away and look at the raw data of a container: a set S, and for each $s \in S$ a set $P(s)$. That is an S-indexed *family* of sets. Nothing more.

Families of objects of a category $\mathcal{C}$ are themselves the objects of a category, the *free coproduct completion* $\mathbf{Fam}(\mathcal{C})$. Its objects are pairs $(S,\, P\colon S \to \mathcal{C})$; a morphism $(S, P) \to (T, Q)$ is a reindexing function $u\colon S \to T$ together with, for each $s \in S$, a $\mathcal{C}$-morphism comparing $P(s)$ with $Q(u(s))$. The completion turns coproducts that $\mathcal{C}$ may lack into formal ones: the family (S, P) *is* the formal coproduct $\coprod_{s \in S} P(s)$.

Now watch the variance. A container morphism $S \triangleleft P \to T \triangleleft Q$, as Definition 2.1 had it, is a shape map $u\colon S \to T$ *forward* together with position maps $f_s\colon Q(u(s)) \to P(s)$ running *backward*. Backward is exactly a morphism in $\mathbf{Set}^{\mathrm{op}}$. So the fibrewise comparison lives in $\mathbf{Set}^{\mathrm{op}}$, not $\mathbf{Set}$, and the category is identified on the nose.

Proposition 3.1 (**Cont** is the free coproduct completion of $\mathbf{Set}^{\mathrm{op}}$)**.** *There is an isomorphism of categories* $\mathbf{Cont} \cong \mathbf{Fam}(\mathbf{Set}^{\mathrm{op}})$*, the identity on objects, sending a container morphism* (u, f) *to the reindexing* u *together with the family* $(f_s)_{s\in S}$ *read as morphisms in* $\mathbf{Set}^{\mathrm{op}}$*.* *[Folklore]*

This is not deep, but it is clarifying, and it pays for itself immediately. It is a standard fact that the free coproduct completion of any category $\mathcal{C}$ embeds into $[\mathcal{C}^{\mathrm{op}}, \mathbf{Set}]$ by the left Kan extension of the Yoneda embedding along itself — concretely, a family becomes a coproduct of representables. Feed it $\mathcal{C} = \mathbf{Set}^{\mathrm{op}}$ and the abstract statement becomes our extension functor:

$$(S, P) \;\longmapsto\; \coprod_{s\in S} y^{P(s)}, \qquad y^A = \mathrm{Hom}_{\mathbf{Set}}(A, -),$$

because a representable on $\mathbf{Set}^{\mathrm{op}}$ evaluated in $\mathbf{Set}$ is the covariant hom-functor $y^A = (-)^A$. So $[\![S \triangleleft P]\!] = \coprod_s y^{P(s)}$ is not a definition we imposed; it is what the free coproduct completion of $\mathbf{Set}^{\mathrm{op}}$ *does* when you land it in functors. Containers are exactly *coproducts of representable endofunctors of* $\mathbf{Set}$. Hold that phrase — it is the whole answer to our question, once we understand which functors it describes.

Terminology hazard: positions versus directions

The reader who also reads Spivak's *Poly* must keep two dictionaries. There a polynomial functor is written $p = \sum_{i\in p(1)} y^{p[i]}$, the elements of $p(1)$ are called *positions*, and the elements of each fibre $p[i]$ are called *directions*. In our language the elements of $p(1) = S$ are *shapes* and the elements of the fibres $P(s)$ are *positions*. The naming is exactly *opposite*: Spivak's "positions" are our shapes, his "directions" our positions. The objects are the same; only the words differ. We keep shapes-and-positions throughout, because in the applications a shape is a structure and a position is a slot inside it, and that is the intuition we want the words to carry.

3.2 The test

We now know the target shape: a container extension is a coproduct of representables, $F(X) \cong \coprod_{s\in S} X^{P(s)}$. So "is F a container?" is "can F be written that way?" We gather evidence by computing: the arithmetic of finite sets decides several cases outright and — better — teaches the method.

Here is the fact that does the work. If $F = [\![S \triangleleft P]\!]$ and X is finite with $|X| = n$, then

$$|F(n)| \;=\; \sum_{s\in S} n^{|P(s)|}.$$

The cardinality of a container extension at n is a sum of powers of n, one power per shape, the exponent being the number of positions. Small n probe the sum. At $n = 1$ every power is 1, so $|F(1)| = |S|$: the value at a one-element set counts the shapes. At $n = 0$ every positive power vanishes and $0^0 = 1$, so $|F(0)|$ counts the shapes with *no* positions — the constants. These two readings alone are often enough.

Example 3.2 (Friendly functors pass)**.** $F(X) = 1 + X$: shapes $\{0, 1\}$ with $|P| = 0, 1$; extension $\coprod\{X^0, X^1\}$. A container. $F(X) = X^2$: one shape, two positions. A container. Lists, $F(X) = \coprod_n X^n$: shape $n \in \mathbb{N}$, positions $\{0, \ldots, n-1\}$. A container. In each case the coproduct-of-powers form is visible on the page.

Example 3.3 (The covariant powerset fails — in one line)**.** Let $\mathcal{P}\colon \mathbf{Set} \to \mathbf{Set}$ be the covariant powerset functor, $\mathcal{P}(X) = \{\text{subsets of } X\}$, acting on maps by direct image. Is it a container?

Count. $|\mathcal{P}(0)| = 1$, $|\mathcal{P}(1)| = 2$, $|\mathcal{P}(2)| = 4$, in general $|\mathcal{P}(n)| = 2^n$. Suppose $\mathcal{P} = [\![S \triangleleft P]\!]$. From $n = 1$ we read $|S| = |\mathcal{P}(1)| = 2$: two shapes. From $n = 0$ we read that exactly one of them has no positions; call the other shape $s_\star$, with $|P(s_\star)| = p$. Evaluate at $n = 2$:

$$4 \;=\; |\mathcal{P}(2)| \;=\; \sum_{s \in S} 2^{|P(s)|} \;=\; 2^0 + 2^p \;=\; 1 + 2^p.$$

So $2^p = 3$, which no natural number p satisfies. The covariant powerset is *not* a container.

The powerset is worth pausing on, because the reason it fails is the reason the theory has a boundary at all. A subset of X is a collection of elements with no record of *where* each element sits. A container labelling $v\colon P(s) \to X$, by contrast, assigns each position its own element, and the position is a name that persists. Powerset forgets the names; it is a functor built by *quotienting*, and quotients are exactly what containers cannot do. We meet this boundary again in §3.4, from the colimit side, and it is the same wall.

Counting settles individual cases, but we want a criterion that works for every functor, finite or not. Here it is. The right notion is that of a *connected limit*: a limit whose indexing diagram is connected (any two objects joined by a zig-zag of arrows). A pullback is connected — its shape $\bullet \to \bullet \leftarrow \bullet$ is joined up — and so are *wide pullbacks* (many arrows into one target) and *cofiltered limits*, the terminal sequence of the chapter's opening among them. A product is *not* connected: its shape is a bare set of objects with no arrows between them, so it falls into disconnected pieces. Connected versus disconnected is exactly the line the characterisation runs along.

Theorem 3.4 (Characterisation of containers)**.** *For a functor $F\colon \mathbf{Set} \to \mathbf{Set}$ the following are equivalent.*

(i) *F is a container extension: $F \cong [\![S \triangleleft P]\!]$ for some container.*
(ii) *F preserves connected limits (equivalently: wide pullbacks and cofiltered limits).*
(iii) *F is a coproduct of representable functors.*

[Cited: Gambino–Kock 2009, §1.18 & Prop. 1.22; equivalence due to Diers 1977 and Carboni–Johnstone 1995]

Why is preserving connected limits the same as being a coproduct of representables? The intuition is short and worth carrying. A single representable $y^A = (-)^A$ preserves *all* limits: it is the right adjoint to $- \times A$, and right adjoints preserve limits. The question is what survives when we glue representables by a coproduct. In **Set**, coproducts commute with connected limits but not with arbitrary ones: a connected diagram cannot "see" the disjoint branches of a coproduct separately, so the coproduct passes through the limit, whereas a product (disconnected) can pair up elements across different branches and breaks the interchange. Preserving connected limits is therefore precisely the trace left in $[\mathbf{Set}, \mathbf{Set}]$ by "being a coproduct of things that preserve everything." Condition (ii) is the shadow of condition (iii).

Why "connected", and neither "wide pullbacks" nor "all"

Condition (ii) is easy to mis-state in either direction.
Not merely wide pullbacks. Wide pullbacks are the connected limits one meets most often, so one is tempted to shorten (ii) to "preserves wide pullbacks". That is genuinely weaker. Cofiltered limits are connected too — the terminal sequence $1 \leftarrow F1 \leftarrow FF1 \leftarrow \cdots$ that opened the chapter is one — yet they are not built from pullbacks, and must be demanded separately. This is exactly why the cofree comonad of Chapter 6 needs the full strength of (ii): its final-coalgebra limit is cofiltered, not a wide pullback. "Preserves connected limits" unpacks to wide pullbacks *and* cofiltered limits together, and dropping the latter loses functors [Cited: Gambino–Kock, §1.18].

Not all limits either. A container need *not* preserve the *empty* limit — the terminal object 1 — because $[\![S \triangleleft P]\!](1) = S$ can be any set, whereas preserving the empty limit would force $F(1) \cong 1$. The empty diagram is *disconnected* — a connected category is required to be nonempty, so the empty one fails the definition at the first clause — and it sits rightly outside (ii). Connectedness is exactly the dividing line: it keeps the cofiltered limits, which containers respect, and discards the products and the empty limit, which they do not. A one-line sanity check makes this memorable: whenever you claim "F preserves connected limits", test it against $F(1)$; if $F(1) \neq 1$ then the terminal object is *not* among the preserved limits, confirming that "connected" really must exclude it.

3.3 Reading the positions back off

The test tells us *whether* F is a container. Suppose it passes. How do we recover the data $S \triangleleft P$?

The shapes are free: $S = F(1)$, exactly as the counting used. A one-element set has one labelling per shape, so $F(1) = \coprod_s 1^{P(s)} = \coprod_s 1 = S$. That half is honest arithmetic. The positions are the subtle half, and here a natural first guess goes wrong.

Correction: the fibre of $F(2) \to F(1)$ is *not* $P(s)$

The tempting formula runs: the unique map $2 \to 1$ induces $F(2) \to F(1) = S$; take the fibre over a shape s and call it the positions. Compute what this actually returns. Since $F(2) = \coprod_{s'} 2^{P(s')}$ and the map to S forgets the labelling $P(s') \to 2$ and remembers only s', the fibre over s is

$$\{\, \varphi \colon P(s) \to 2 \,\} \;=\; 2^{P(s)},$$

the set of *subsets* of $P(s)$ — the *powerset* of the positions, not the positions. Evaluation-and-fibre overcounts by an exponential. No single evaluation of F, followed by a fibre, can extract $P(s)$: the functor at a fixed set only ever sees *labellings* of the positions, never the positions bare.

The correction is not a patch; it is a signpost pointing at the real mechanism. To see $P(s)$ itself we must ask for the labelling that names each position *by itself* — the identity labelling — and that is a universal property, not an evaluation.

Proposition 3.5 (Position recovery via the generic element)**.** *Let F be a container. For each shape $s \in F(1)$ there is a set $P(s)$ and a* generic element *$g_s \in F(P(s))$ lying over s, universal in the sense that for every set X and every $x \in F(X)$ lying over s there is a unique map $\alpha \colon P(s) \to X$ with $F(\alpha)(g_s) = x$. This $P(s)$ is the object of positions, and*

$$F(X) \;\cong\; \coprod_{s \in F(1)} X^{P(s)}$$

is the container form, recovered componentwise by the Yoneda lemma. The generic element is the initial object of the category of elements of F sitting over s; its existence is exactly the connected-limit preservation of Theorem 3.4. [Cited: Gambino–Kock 2009, Prop. 1.22]

The generic element is the identity labelling $g_s = (s, \mathrm{id}_{P(s)})$ wearing abstract clothing: it is the one element of $F(P(s))$ from which every other labelled structure of shape s is obtained by relabelling, and by exactly one relabelling. "Exactly one" is the universal property, and it is what pins $P(s)$ down to the positions rather than to any of their exponential shadows.

Remark 3.6 (The derivative sees the total position set). A second, tidier-looking phrasing is worth knowing — and worth not overtrusting. The total space of positions $\coprod_s P(s)$ is the value

at 1 of the *derivative* ∂F, the container of one-hole contexts sketched in the Phase 2 outline (Chapter 8), and $P(s)$ is its fibre over s. This is correct, but it is a restatement, not a shortcut: ∂F is itself a derived construction, not a single evaluation of F, so it does not escape the moral of the correction box. If you want the positions, you must, one way or another, invoke the universal property.

3.4 Limits and colimits in Cont

We have been studying $[\![-]\!]$ as a bridge from **Cont** into $[\mathbf{Set}, \mathbf{Set}]$. How much categorical structure does the bridge carry across? The identification of §3.1 answers the first half at a stroke: free coproduct completions are complete and cocomplete, so **Cont** *is both*. What is delicate is which limits and colimits $[\![-]\!]$ *preserves*, and the characterisation theorem has already told us where the answer turns.

Start with the structure that is easy, concrete, and machine-checked. Binary products and coproducts in **Cont** have a clean description, and the variance is the whole lesson.

Proposition 3.7 (Products and coproducts of containers)**.** *In* **Cont***:*

- *the* coproduct *of* $S \triangleleft P$ *and* $T \triangleleft Q$ *has shapes* $S + T$ *and positions inherited case-wise (*P *on the left summand,* Q *on the right);*
- *the* product *of* $S \triangleleft P$ *and* $T \triangleleft Q$ *has shapes* $S \times T$ *and, over* (s, t)*, positions* $P(s) + Q(t)$ *— a* coproduct *of position sets.*

On extensions, $[\![-]\!]$ *carries both to the pointwise coproduct and product of functors. (These reappear, framed as the monoidal* $+$ *and* $\times$*, in Chapter 4.)* *[MacBeth]*[1]

The product deserves a second look, because it inverts the naive expectation. The positions of a product are a *sum*, not a product. The moment you accept $\mathbf{Cont} \cong \mathbf{Fam}(\mathbf{Set}^{\mathrm{op}})$ (§3.1) this is not a surprise but a *compulsion*: a product in $\mathbf{Fam}(\mathcal{C})$ is computed fibrewise in $\mathcal{C}$, and a product in $\mathbf{Set}^{\mathrm{op}}$ is a coproduct in **Set**, so the fibres of a product of containers can only be coproducts. The same fact seen closer up is the variance of §3.1 playing out on universal properties: the projection $\pi_1 \colon S \triangleleft P \times T \triangleleft Q \to S \triangleleft P$ has the forward shape map $(s, t) \mapsto s$, and on positions it must run *backward*, so it is the coproduct injection $P(s) \hookrightarrow P(s) + Q(t)$. A morphism into the product is a pair of morphisms, and pairing forward shape maps forces summing backward position maps. Under $[\![-]\!]$ this reads as the familiar polynomial identity $[\![S \triangleleft P \times T \triangleleft Q]\!](X) = \coprod_{s,t} X^{P(s)+Q(t)} = \coprod_{s,t} X^{P(s)} \times X^{Q(t)}$, the product of the two extensions — so $[\![-]\!]$ preserves this product. It is worth being clear about why that is a separate fact and not a corollary of the characterisation theorem.

What $[\![-]\!]$ preserves, and where it stops

Theorem 3.4 says a container extension *preserves connected limits*. Products are the archetypal *dis*connected limit, so connected-limit preservation says nothing about them directly; nevertheless $[\![-]\!]$ does preserve products, by the explicit computation above (and coproducts, since a coproduct of shapes maps to a coproduct of extensions). The clean summary:

$$[\![-]\!] \text{ preserves } \begin{cases} \text{connected limits} & \text{(Theorem 3.4)} \\ \text{products and coproducts} & \text{(Proposition 3.7)} \end{cases}$$

but $[\![-]\!]$ is *not* cocontinuous: it does not preserve general colimits. The first thing it fails to preserve is the *coequaliser*.

[1] MacBeth has a Lean formalisation of both universal properties in the file `Cont.lean` (built, zero `sorry`); that source is *not yet committed* to this repository's `lean/` tree, so the tag is [MacBeth] here rather than Lean-verified, matching the convention of the Preface.

Why should coequalisers be where preservation breaks? Because a coequaliser is a quotient, and we have already watched a quotient carry an object out of the container world once. The covariant powerset of §3.2 is itself a quotient of a coproduct of representables: it is the list functor with its labellings identified whenever they differ by a rearrangement or a repetition. That double identification is exactly what a coproduct of representables cannot express — a representable names its positions and keeps them distinct — and it is why the counting failed. Coequalisers reproduce the phenomenon in general: coequalise two container morphisms and the resulting functor can acquire automorphisms of its positions. The mildest such quotients, by a group acting on positions, are the *analytic* functors of Joyal's species, already outside the image of $[\![-]\!]$; powerset lies further out still, its idempotent identification coarser than any group action. Either way the lesson is one line: the coproduct-of-representables form is closed under connected limits and under coproducts, and not under quotients.

The failure is not merely generic: there is a minimal two-container witness, and it is exactly the swap that turns ordered pairs into unordered ones.

Proposition 3.8 ($[\![-]\!]$ does not preserve coequalisers)**. Cont** *has coequalisers whenever the base category has equalisers and coequalisers, but* $[\![-]\!]$ *does not preserve them. For a witness, let* $\mathsf{Pair} = 1 \triangleleft \{1,2\}$ *be the container of ordered pairs, with* $[\![\mathsf{Pair}]\!]X = X^2$*, and let* $\mathrm{id},\ \sigma \colon \mathsf{Pair} \rightrightarrows \mathsf{Pair}$ *be the identity and the swap — both the single shape map, with* σ*'s backward position map transposing* $1 \leftrightarrow 2$*. Their coequaliser in* **Cont** *is* $1 \triangleleft \varnothing$*: one shape, and its positions are the* equaliser *of* $\mathrm{id}, \sigma \colon \{1,2\} \rightrightarrows \{1,2\}$ *in* **Set** *(positions run backward, so a coequaliser of shapes is an equaliser of positions), which is empty since the swap has no fixed point. Hence* $[\![1 \triangleleft \varnothing]\!] = X^{\varnothing}$ *is the constant functor* 1*. But the coequaliser of* id, σ *in* $[\mathbf{Set}, \mathbf{Set}]$ *is the* unordered-*pair functor* $X \mapsto X^2/\!\sim$*,* $(x,y) \sim (y,x)$*, which is not constant. The two disagree, so* $[\![-]\!]$ *carries neither to the other: it does not preserve this coequaliser.* *[Cited: Abbott–Altenkirch–Ghani,* Categories of containers *(FoSSaCS 2003) [2], Prop. 4.3 (***Cont** *has coequalisers when the base has equalisers and coequalisers) and Ex. 4.4 (this witness); Ex. 4.5 treats filtered colimits the same way.]*

The witness is the mechanism of the paragraph above at its smallest: quotienting by the swap introduces a position automorphism (the transposition), and the extension of the quotient — the unordered-pair functor — is exactly one of the analytic functors that lie outside the image of $[\![-]\!]$.

Remark 3.9 (**Cont** is complete and cocomplete; $[\![-]\!]$ is not cocontinuous)**.** $\mathbf{Cont} \cong \mathbf{Fam}(\mathbf{Set}^{\mathrm{op}})$ is complete and cocomplete, so **Cont** *has all limits and all colimits.* The subtlety is entirely about which the bridge $[\![-]\!]$ transports: connected limits and (co)products, yes; coequalisers, no (Proposition 3.8). This is the first place the polynomial world and the functor world part company, and it is worth holding as the boundary marker it is. [MacBeth; the coequaliser half is Proposition 3.8.]

3.5 Two witnesses that are genuine (co)monads

The coequaliser of the last section carried an object out of the container world, and the object it produced — the unordered-pair functor — was a bare functor, assembled by hand as a colimit. A sceptic could still hope the boundary is an artefact of that assembly: perhaps every functor one actually *uses* — every functor that carries algebraic structure, that is a monad or a comonad — stays inside. This section closes that hope, and closes it symmetrically. There is a bona fide *monad* on **Set** that is not a container, and, mirroring it almost line for line, a bona fide *comonad* on **Set** that is not a container. Both fail the container test in *exactly the same way* — they do not preserve the connected pullback $A \to 1 \leftarrow B$ — and both fail it for the same reason: each is a *quotient* of a polynomial structure by a symmetry, and the quotient throws away the very provenance a container must remember.

Take the monad first, because you can hold it in your hand.

The monad side: the multiset monad is not a container

The finite-multiset monad Bag sends a set X to the set $\mathsf{Bag}\,X$ of finite multisets of elements of X, with unit $\eta\,x = \{x\}$ and multiplication $\mu = \uplus$ (multiset union). It is a genuine monad — the free commutative monoid monad — and at first sight it looks exactly like a container. A multiset $m = \{x_1, \ldots, x_n\}$ has n *leaves*, labelled by the x_i, and $\mathsf{Bag}\,f$ relabels the leaves without disturbing their number; so Bag is leaf-supported, just as $[\![S \triangleleft P]\!]X = \coprod_s X^{P(s)}$ presents each element by a shape and a labelling of positions. The multiplication is even better behaved than one could ask: μ merges the inner multisets by disjoint union, so its effect on leaves is a *bijection* — no leaf of $\mu\,mm$ appears from nowhere, every one has a unique inner token of the same label. Every pointwise reason to be a container is present.

And yet Bag is not a container. The test of §3.2 is preservation of connected limits, and the cheapest connected limit to probe is the pullback of the cospan $A \to 1 \leftarrow B$, whose limit is $A \times B$. Preservation asks that the comparison map

$$\mathsf{Bag}(A \times B) \longrightarrow \mathsf{Bag}\,A \times_{\mathsf{Bag}\,1} \mathsf{Bag}\,B$$

be a bijection. Here is the point that makes this a real test rather than a triviality: $\mathsf{Bag}\,1 = \mathbb{N}$, the set of *sizes*, so the right-hand side is not a plain product but the pullback of *equal-size* multisets — a genuinely connected limit mediated by $\mathbb{N}$. Now count, with $A = B = \{0,1\}$, at size 2. On the left, the size-2 multisets over the four-element set $A \times B$ number $\binom{4+2-1}{2} = 10$. On the right, a size-2 multiset over A paired with a size-2 multiset over B: there are three of each, so $3 \times 3 = 9$. Ten does not equal nine, and the comparison map cannot be a bijection.

Proposition 3.10 (The multiset monad is not a container)**.** Bag *does not preserve the connected pullback* $A \to 1 \leftarrow B$*, so it is not the extension of any container. Concretely, at* $A = B = \{0,1\}$ *the two distinct size-2 multisets*

$$w_1 = \{(0,0),(1,1)\}, \qquad w_2 = \{(0,1),(1,0)\} \quad \textit{over } A \times B$$

have the same image in $\mathsf{Bag}\,A \times_{\mathsf{Bag}\,1} \mathsf{Bag}\,B$*, namely the pair* $(\{0,1\},\{0,1\})$*: the multiset has forgotten* which *A-element was paired with* which *B-element.* *[MacBeth, computed (`bag_not_container.py`:* $|\mathsf{Bag}(2\times 2)|_2 = 10 \neq 9$*, and* $|\cdot|_3 = 20 \neq 16$*). The underlying fact — the multiset functor is analytic, not polynomial — is classical.]*

The collision $w_1 \neq w_2$ is the whole story in miniature. A container remembers, for each position, exactly which value sits there; a multiset remembers only *how many* of each value, and w_1, w_2 differ precisely in a bookkeeping that the multiset discards — the pairing of the two coordinates. That lost bookkeeping is a symmetry: a multiset of size n is an ordered n-tuple with its order quotiented away by the symmetric group S_n. Quotient the ordered tuples — which *are* a container, the list functor $X \mapsto \coprod_n X^n$ with shape n and positions $\{1,\ldots,n\}$ — by that action and you leave the polynomial world for the *analytic* one, exactly as the swap did for ordered pairs. The difference now is only that Bag carries this all the way up to a monad: a legitimate algebraic structure on **Set** can sit outside **Cont**. Chapter 5 puts precisely this witness to work (Theorem 6.28): being leaf-supported and reverse-total is not enough to make a monad a *container* monad, and Bag is what separates the two.

The comonad side: the multigraph comonad

Now the mirror. Directed multigraphs — quivers — are the presheaves on the two-object category $\cdot \rightrightarrows \cdot$, and they are the coalgebras of a *polynomial* comonad on **Set**, the one whose extension is $F_{\mathrm{dir}}(X) = X^3 + X$ (an edge is a source, a target, and a name; a vertex is a vertex). This is a container, and under the equivalence **Cont**-comonoids $\cong$ small categories that Chapter 5 will prove, its category is the walking parallel pair $\cdot \rightrightarrows \cdot$ itself: two objects,

with three arrows out of the source (whence the X^3) and one out of the target (whence the X).

Undirected multigraphs are what you get by forgetting the orientation of each edge — by quotienting the two parallel arrows of $\cdot \rightrightarrows \cdot$ by the swap. Aaron David Fairbanks observed that this quotient can be taken *at the level of comonads*: the undirected-multigraph comonad F is the coequaliser, in the category of comonads on **Set**, of the identity and the comonad automorphism φ induced by that swap [42]. Its extension is

$$F(X) = \frac{X^3 + X^2}{2} + X,$$

the count of undirected edges (a vertex, a vertex, a name, with the two endpoints now unordered) plus the vertices. And like Bag, it fails the container test on the kernel pair of $X \to 1$. Fairbanks's computation is one line and worth reading as the exact dual of the multiset collision. Here $F(1) = 2$ — an edge slot and a vertex slot — and $F(!)$ sends the whole edge summand to the first point and the vertices to the second; so the pullback of $F(!)\colon F(X) \to F(1)$ along itself, being the pairs of elements that agree in $F(1)$, splits as the sum of the two summands squared *separately*:

$$\left(\tfrac{X^3+X^2}{2}\right)^2 + X^2, \qquad \text{whereas} \qquad F(X^2) = \frac{X^6 + X^4}{2} + X^2,$$

and these disagree — *squaring every summand is not the same as substituting a squared variable*. That slogan is the polynomial condition itself: a container's extension substitutes into positions, so $F(X^2)$ must be "F of the squared variable", not "F squared summand-wise". The halving — the orientation thrown away by the quotient — is exactly what breaks the substitution.

Proposition 3.11 (A comonad that is not a container)**.** *The undirected-multigraph comonad $F(X) = \frac{1}{2}(X^3+X^2)+X$ on* **Set** *is a genuine comonad — undirected multigraphs are comonadic over* **Set** *— yet it does not preserve pullbacks (it fails on the kernel pair of $X \to 1$), so its underlying functor is not polynomial and F is not a container.* *[Cited: A. D. Fairbanks, MathOverflow answer to "Examples of non-polynomial comonads on* **Set***" [42] (question 457580), verified verbatim by MacBeth. The pullback-non-preservation is Fairbanks's own displayed computation.]*

The boundary is two-sided

Bag and F are a matched pair. Each is a legitimate algebraic structure on **Set** — one a monad, the other a comonad — and each is a *quotient of a container by a symmetry*: ordered tuples modulo S_n for Bag, the directed-quiver comonad modulo edge-reversal for F. In both cases the quotient discards *provenance* — the pairing of coordinates, the orientation of an edge — and provenance is exactly what a polynomial functor is obliged to remember. And in both cases the failure is detected by the *same* connected limit, the kernel pair of $X \to 1$: Bag gives $10 \neq 9$, F gives squared-summands $\neq$ squared-variable. "Polynomial, not merely analytic" is therefore not a one-sided caution about which monads are container monads; it is a two-sided law that constrains, by one test, both the monads that can be container monads and the comonads that can be small categories. The same boundary returns one categorical level up: in Chapter 5 (Remark 6.32) the symmetric functors Sym^2 and Bag_2 are excluded as monad *liftings* because they admit no natural counit — "no canonical element to point at" being the lifting-level shadow of "no provenance to remember".

3.6 The boundary of the theory

Step back and the chapter's findings line up into one picture. A container is a family of sets; $\mathbf{Cont} \cong \mathbf{Fam}(\mathbf{Set}^{\mathrm{op}})$; and its extension is a coproduct of representables. Being a coproduct

of representables is a property a functor can be *tested* for — preservation of connected limits — and when it holds, the shapes are $F(1)$ and the positions are named by generic elements, not by fibres. And the same form that these coproducts take is what fails under quotients: powerset from the value side, coequalisers from the colimit side, and — §3.5 — a genuine monad (Bag) and a genuine comonad (F) from the algebraic side, each a quotient of a container by a symmetry. All of these are one failure. Containers are what you get by *summing* representables; you leave them the moment you *quotient*.

That boundary is the polynomial boundary the book keeps returning to. Inside it live the objects on which the whole equivalence chain runs — directed containers, comonoids, small categories, the monoidal structures of the later chapters — and each of those structures will silently use that its underlying functor is a coproduct of representables, that its shapes are recoverable, that connected limits pass through. Outside it lie the analytic and symmetric functors, the probability and powerset monads, the quotients: a rich world, but one where the equivalence chain fractures, and where compositional correctness has to be re-earned rather than inherited. Knowing which functors are containers is knowing where you are standing.

Chapter 5 puts the boundary to work: it asks which containers carry a *comonoid* structure for the composition product, and finds — the central surprise of the book — that the answer is small categories.

Chapter 4

Algebraic structure on Cont

This chapter is, by design, partly an *outline with cited statements*. The monoidal and closed structures on containers are most cleanly developed in the polynomial-functor setting, where **Cont** embeds as the full subcategory **Poly** of polynomial endofunctors. We record the structures, cite their sources, and flag carefully which MacBeth has personally verified. With a single exception — the closure criterion of §4.3, which is the book's own contribution — we do *not* reprove the cited structures here, and we invent no proofs of them.

Remark 4.1 (Containers and polynomial functors). A container $S \triangleleft P$ presents the polynomial functor $X \mapsto \sum_{s \in S} X^{P(s)}$; the category **Poly** of polynomial functors and natural transformations is equivalent to **Cont** (this is Theorem 1.3 read as "**Poly** $\simeq$ **Cont**"). We use the two names interchangeably and adopt Spivak's catalogue of monoidal structures on **Poly** [12].

4.1 Products and coproducts

Proposition 4.2 (Coproducts in **Cont**)**. Cont** *has all coproducts: the coproduct of* $S_i \triangleleft P_i$ *has shape set* $\coprod_i S_i$ *and positions inherited shapewise,* $P\big((i,s)\big) = P_i(s)$*. On extensions it is the pointwise coproduct of functors,* $[\![\coprod_i C_i]\!] = \coprod_i [\![C_i]\!]$. *[Cited: AAG05; Spivak 2021]*

Proposition 4.3 (Products in **Cont**)**. Cont** *has all products: the product of* $S_1 \triangleleft P_1$ *and* $S_2 \triangleleft P_2$ *has shape set* $S_1 \times S_2$ *and positions* $P\big((s_1,s_2)\big) = P_1(s_1) \sqcup P_2(s_2)$*, so that* $[\![C_1 \times C_2]\!](X) = [\![C_1]\!](X) \times [\![C_2]\!](X)$. *[Cited: Spivak 2021]*[1]

4.2 The monoidal menagerie

Spivak [12] catalogues several monoidal structures on **Poly**. We list the ones relevant to this book; the reader should treat each as *cited, not independently checked* unless tagged otherwise.

Definition 4.4 (The four products, after Spivak 2021)**.** On **Poly** $\simeq$ **Cont** there are, among others:

- the *coproduct* $+$ (Prop. 4.2), unit the initial polynomial 0;
- the *product* $\times$ (Prop. 4.3), unit the terminal polynomial 1;
- the *Dirichlet (parallel) tensor* $\otimes$: on shapes $S_1 \times S_2$, on positions $P_1(s_1) \times P_2(s_2)$, so that $[\![C_1 \otimes C_2]\!](X) = \sum_{s_1,s_2} X^{P_1(s_1)\times P_2(s_2)}$; unit the identity polynomial $X \mapsto X$ $(1 \triangleleft 1)$;
- the *composition (substitution) product* $\triangleleft$, with $[\![C_1 \triangleleft C_2]\!] = [\![C_1]\!] \circ [\![C_2]\!]$; unit the identity polynomial. This is the *sequencing* product, and comonoids for it are the subject of Chapter 5.

[1] Stated as in [12]; the coproduct (Prop. 4.2) MacBeth has checked directly on extensions, the product MacBeth has *not* independently re-derived in full and reports it as cited.

[Cited: Spivak 2021]

Remark 4.5 (Honesty about $\otimes$ vs. $\times$). The symbols are easy to confuse. The *Dirichlet* tensor $\otimes$ multiplies *position sets* ($P_1 \times P_2$); the categorical *product* $\times$ takes a *disjoint union* of position sets ($P_1 \sqcup P_2$). They have the same shape set $S_1 \times S_2$ but different position data and different extensions. Both are due to Spivak's catalogue [12]; MacBeth has verified the shape/position formulas combinatorially on small cases but defers the unit and coherence laws to [12].

The free-monad and cofree-comonad constructions associated with $\triangleleft$ are developed in Chapter 6, where they belong to the functors-over-**Cont** programme rather than to the structure of **Cont** itself.

4.3 Closing the structures

A monoidal category asks one question above all others: is its tensor *closed*? Can every map out of a tensor, $p \odot q \to r$, be traded for a map into a single object, $p \to [q, r]$, that internalises "the maps from q to r"? For the four structures on **Cont** the answers are not uniform — and the reason they are not uniform is the most instructive thing in this chapter. This section keeps one promise: *by its end you will be able to decide, for any of these tensors, whether it is closed — and you will see that closure on containers is polynomiality on sets, read through the extension.*

There is a warning to hear first. If an internal hom exists it is a right Kan extension, and a right Kan extension of container-valued data has no obligation to be a container. So the honest question is not "what is the internal hom" but "when is there one at all." The rest of the chapter cites; this section is where it earns one theorem of its own, and the theorem answers exactly that.

Throughout, write y^A for the *representable* container — a single shape whose position set is A — so that $[\![y^A]\!]X = X^A$; in particular $y := y^1$ is the identity container. Every container is a coproduct of representables, $p \cong \sum_{i \in S_p} y^{p[i]}$, and we use two facts about **Cont** constantly: co-Yoneda, $\mathbf{Cont}(y^R, q) \cong [\![q]\!]R$ (a map $y^R \to q$ is a shape of q and a map of its positions into R), and the fact that $\mathbf{Cont}(-, q)$ turns the coproduct of representables into a product, $\mathbf{Cont}\big(\sum_i y^{p[i]}, q\big) \cong \prod_i [\![q]\!](p[i])$.

4.3.1 The Dirichlet closure, as a hom of morphisms

Begin with the Dirichlet tensor $\otimes$, whose closure is the cleanest, and compute the transpose by hand. Recall that $p \otimes q$ has shape set $S_p \times S_q$ and, over (s, t), position set $p[s] \times q[t]$. A morphism $r \otimes p \to q$ is therefore a forward shape map $U \colon S_r \times S_p \to S_q$ together with, over each (a, i), a backward map $q[U(a, i)] \to r[a] \times p[i]$. Split each backward map into its two components (Hom into a product is a product of Homs), then apply the type-theoretic axiom of choice twice — once to pull the S_q-choice out over S_p, once to pull the whole package out over S_r. What falls out is: for each $a \in S_r$, a container morphism $f_a \colon p \to q$, together with a single map $\sum_{i \in S_p} q[f_a\, i] \to r[a]$. But that is precisely a morphism $r \to [p, q]$, where

$$[p, q] \;:=\; \Big(\mathbf{Cont}(p, q),\quad f \mapsto \textstyle\sum_{i \in S_p} q[f_1\, i]\Big) \;=\; \sum_{f \colon p \to q} y^{\sum_{i \in S_p} q[f_1 i]}.$$

Read this object slowly, because it is the whole point. A *shape* of $[p, q]$ is a morphism $p \to q$. A *position* over that shape f is a p-shape i paired with a q-position lying over the shape $f_1(i)$ that f sends i to. In the reading that will drive this book's applications: a *prompt* is a morphism $p \to q$; a *response* to it is a p-shape together with a q-position over the shape the prompt delegated. The shapes of the internal hom are the morphisms of the category.

Theorem 4.6 (The Dirichlet tensor is closed)**.** *For each container p the functor $(-\otimes p)$ is left adjoint to $[p,-]$: there is a bijection*

$$\mathbf{Cont}(r\otimes p,\ q)\ \cong\ \mathbf{Cont}\big(r,\ [p,q]\big),$$

natural in r and in q. Hence $(\mathbf{Cont},\otimes,y)$ is a symmetric closed monoidal category, with internal hom $[p,q]$ as displayed above. *[Cited: Niu–Spivak 2023, Ex. 4.78, eq. (4.79) [13]; Spivak 2022, eq. (44) [14]; MacBeth, Lean-verified: `DirichletClosed.lean`]*[2]

4.3.2 The same hom, read as a product of composites

Here is the first surprise. The object we just assembled by hand — shapes are morphisms, positions are delegated positions — is also, *on the nose*, a product of composites of the simplest containers:

$$[p,q]\ \cong\ \prod_{i\in S_p} q\triangleleft(p[i]\cdot y),\qquad\text{equivalently}\qquad [\![[p,q]]\!]R\ =\ \prod_{i\in S_p}[\![q]]\!](R\times p[i]).$$

Here $p[i]\cdot y=\sum_{p[i]}y$ is the monomial container of the functor $X\mapsto p[i]\cdot X=X\times p[i]$, and $q\triangleleft(-)$ is the composition product of §4.4's catalogue, whose extension is composition of functors, so that $[\![q\triangleleft(p[i]\cdot y)]\!]R=[\![q]]\!](R\times p[i])$.

That two such differently-built containers coincide is the type-theoretic axiom of choice wearing a disguise. Unfold the right-hand extension: $\prod_i[\![q]]\!](R\times p[i])=\prod_i\sum_t(R\times p[i])^{q[t]}$; distribute $\prod\sum\ \cong\ \sum\prod$ over i, and the sum-index that appears is exactly a forward map $u\colon S_p\to S_q$ together with the backward data $\prod_i p[i]^{q[u\,i]}$ — that is, a morphism $p\to q$ — while the remaining R-exponent collapses to $\sum_i q[u\,i]$. The morphism form and the product form are the same container, seen from opposite sides of one choice isomorphism.

Theorem 4.7 (The product form of the Dirichlet hom)**.** *For all containers p,q there is an isomorphism $[p,q]\cong\prod_{i\in S_p}q\triangleleft(p[i]\cdot y)$, natural in q.* *[MacBeth, Lean-verified: `DirichletHomPi.lean`]*[3]

The product form is worth the detour because it is the one that *generalises*. Nothing in it is special to $\times$; it is built from a fixed target q, the shapes of p, and one binary operation — here $\times$ — feeding each position of p into q. Change that operation and you change the tensor. That is the next subsection.

4.3.3 When is a convolutional tensor closed?

The product $\times$, the Dirichlet tensor $\otimes$, and a third family of Spivak's are not three unrelated constructions: each arises by the *same recipe* from a monoidal structure on the base category **Set**. Given a monoidal structure $(\star,I)$ on **Set**, its *Day convolution* is the tensor $\odot_\star$ on **Cont** with

$$p\odot_\star q\ =\ \Big(S_p\times S_q,\ (s,t)\mapsto p[s]\star q[t]\Big),\qquad\text{unit } y^I.$$

Taking $\star=(+,\varnothing)$ gives the categorical product $\times$ (positions $p[s]\sqcup q[t]$); taking $\star=(\times,1)$ gives the Dirichlet tensor $\otimes$ (positions $p[s]\times q[t]$); taking Garner's $\star=(\vee_S,\varnothing)$ gives Spivak's third family $\rhd_S$.

[2] The formula is not new: it is Niu–Spivak's Exercise 4.78 and the object Spivak calls "the Dirichlet closure." The derivation above reproduces it character for character. What is MacBeth's is a mechanisation: `lean/Containers/Containers/DirichletClosed.lean` constructs the currying bijection and both round-trip identities, each by `rfl` (η for Σ and $\times$ identifies the two composites on the nose), and the declaration `dirichlet_closure` depends on no axioms. To our knowledge this is the first machine-checked internal hom for the Dirichlet tensor.

[3] Machine-checked as `Container.ihomPiIso` in `lean/Containers/Containers/DirichletHomPi.lean`: an isomorphism of containers whose shape component is the choice reindexing $\prod\sum\ \cong\ \sum\prod$. Axiom-free. The anticipated transport along the choice bijection never materialised — the two containers are definitionally close enough that the shape map is a plain rearrangement.

Theorem 4.8 (Uniform closure criterion)**.** *Let $(\star, I)$ be a monoidal structure on* **Set***. The Day tensor $\odot_\star$ is* left closed *— i.e. $(-) \odot_\star p$ has a right adjoint for every container p —* ***if and only if*** *the functor $(-) \star B \colon \mathbf{Set} \to \mathbf{Set}$ is polynomial (preserves connected limits) for every set B. When it holds, the internal hom is*

$$[p, q]_\star \;=\; \prod_{i \in S_p} q \triangleleft (p[i] \star y), \qquad \textit{i.e.} \qquad [\![[p, q]_\star]\!] R \;=\; \prod_{i \in S_p} [\![q]\!](R \star p[i]).$$

[MacBeth (both directions); the three named instances are Cited: Spivak 2022, eqs. (38)–(40) [14]]

Why, in two breaths. Sufficiency. If each $(-) \star B$ is polynomial then so is $R \mapsto [\![q]\!](R \star p[i])$ (a composite of polynomial functors), and so is their S_p-indexed product (**Poly** is closed under composition and small products). Call the resulting container $[p, q]_\star$. The adjunction now reads off co-Yoneda: $\mathbf{Cont}(r \odot_\star p, q)$, expanded over the representables of r and p, is $\prod_J \prod_i [\![q]\!](r[J] \star p[i])$, which is exactly $\mathbf{Cont}\big(r, [p, q]_\star\big)$.

Necessity. This is the half that looks like it should be hard and is a single line. Suppose $\odot_\star$ is closed, and evaluate the right adjoint at the one hom $[y^B, y]_\star$. Co-Yoneda and the adjunction give

$$[\![[y^B, y]_\star]\!] R \;\cong\; \mathbf{Cont}\big(y^R, [y^B, y]_\star\big) \;\cong\; \mathbf{Cont}\big(y^R \odot_\star y^B,\; y\big) \;=\; \mathbf{Cont}\big(y^{R \star B},\; y\big) \;\cong\; R \star B,$$

using $y^R \odot_\star y^B = y^{R \star B}$. So $(-) \star B$ is the extension of a container — it is polynomial. Closedness is a statement about *all* homs; but to extract the obstruction you interrogate only one. The polynomial condition is not bolted onto closure — it *is* closure, evaluated at a point. □

Remark 4.9 (What is new here, and what is not). The three concrete closures are prior art, and this book claims none of them: the cartesian is Niu–Spivak's Theorem 5.31 [13] (and, historically, Altenkirch–Levy–Staton), the Dirichlet is Exercise 4.78 there and Spivak's eq. (44) [14], and $\triangleright_S$ is Spivak's eq. (40). Spivak writes the three internal homs *side by side*; what is MacBeth's is the observation that they are one formula, together with the biconditional that says exactly which convolutional tensors are closed and the one-line necessity argument that makes it a criterion rather than a catalogue. It is a remark on the structure, not a deep theorem — but it is the sentence that turns three coincidences into a law.

4.3.4 Is the condition ever really a condition?

Theorem 4.8 has a right-hand side — "$(-) \star B$ is polynomial for every B" — and an honest reader should ask whether that side is ever *false*. Every monoidal structure on **Set** one can name off the top of one's head ($+$, $\times$, Garner's $\vee_S$, and their finite iterates) is polynomial in each variable, so its Day tensor is closed; and it is tempting, on that evidence, to guess that the condition is automatic — that *every* monoidal structure on **Set** is polynomial in each variable, and that Theorem 4.8 carries a hypothesis it never needs. The guess is wrong. There is a symmetric monoidal structure on **Set** whose Day convolution is a bona fide tensor on **Cont** with no internal hom at all. On containers *convolutional strictly contains left-closed*, and the side-condition of Theorem 4.8 earns its keep.

Definition 4.10 (The collapse tensor)**.** Define $\star \colon \mathbf{Set} \times \mathbf{Set} \to \mathbf{Set}$, with unit $\varnothing$, by

$$A \star B \;=\; \begin{cases} B & A = \varnothing, \\ A & B = \varnothing, \\ 1 & A \neq \varnothing \text{ and } B \neq \varnothing, \end{cases}$$

where 1 is a fixed one-point set. On maps $f \colon A \to A'$, $g \colon B \to B'$, the arrow $f \star g$ is the surviving factor — g when $A' = \varnothing$, f when $B' = \varnothing$ — and the unique map to 1 when A', B' are both nonempty. [MacBeth]

Two nonempty sets, tensored, *collapse* to a single point; hence the name. The action on maps is forced and consistent, because a function out of a nonempty set has nonempty image: $A \neq \varnothing$ forces $A' \neq \varnothing$, so the "both nonempty" clause of the target is reachable only from the "both nonempty" clause of the source, where the unique map to 1 is the only thing one could write.

Proposition 4.11 (A convolutional tensor with no internal hom)**.** $(\star, \varnothing)$ *of Definition 4.10 is a symmetric monoidal structure on* **Set***, yet* $(-) \star 2$ *is not polynomial. Consequently its Day convolution* $\odot_\star$ *is a symmetric monoidal structure on* **Cont** *that is* not *left closed:* $(-) \odot_\star y^2$ *has no right adjoint.* *[MacBeth]*[4]

Why, in two breaths. Monoidal. The unitors are identities. The associator $(A \star B) \star C \to A \star (B \star C)$ and the braiding $A \star B \to B \star A$ are the evident bijections: if two or more of the factors are nonempty both sides equal 1; if exactly one is nonempty both sides are a copy of it; if all are empty both sides are $\varnothing$. Every arrow in every coherence diagram — pentagon, triangle, naturality of α and of β — is therefore one of just two things. Either it is the *unique* map into 1, which happens whenever the node it touches multiplies two or more nonempty factors; such diagrams commute because 1 is terminal. Or it is the functorial action on the *single* surviving factor, which happens when it does not; such diagrams are the coherence of the trivial unit-only structure, i.e. plain functoriality of one factor. There is no third case, so this is a complete finite proof.

Not polynomial. By the strict left unit, $R_2(\varnothing) = \varnothing \star 2 = 2$, whereas $R_2(1) = 1 \star 2 = 1$ because $1, 2 \neq \varnothing$. But every polynomial functor $F = \sum_i y^{a_i}$ satisfies $|F\varnothing| = \#\{i : a_i = \varnothing\} \leq \#\{i\} = |F1|$, and here $|R_2\varnothing| = 2 > 1 = |R_21|$. So R_2 is not polynomial — concretely, it sends the mono $\varnothing \hookrightarrow 1$ to the non-mono $2 \to 1$. By the necessity half of Theorem 4.8, $\odot_\star$ is not left closed. □

Why the collapse tensor slips through

The moral of the near-misses survives, and it is exactly what makes the collapse tensor legal. A polynomial functor is one that *remembers*, in its position sets, which inputs each output used; $\vee_S$ is coherent *because* its normal form keeps a summand for every relevant subset, and that bookkeeping *is* the exponent that makes it polynomial. The support tensor $A \sqcup B \sqcup \{\bullet\}$ tried to discard the provenance and was punished with no natural associator: its lone separator $\bullet$ could not record *which* elements it had separated. The collapse tensor discards the provenance too, and gets away with it, for one reason — it collapses not to a *marked* point but to *the* point, a terminal object, which has no provenance to be inconsistent about. There is nothing for naturality to demand back. And that is, in the same breath, why it is not polynomial: $1 \star B$ is too small to see B. Provenance, polynomiality and coherence are still three names for one thing — but a tensor is free to keep no provenance at all, provided it also builds nothing that needs any, and it forfeits polynomiality by exactly that choice.

The mechanism is worth naming, because it says where to look next. Closure fails because the unit insertion $\eta_B \colon B \cong \varnothing \star B \to 1 \star B$ is not injective — multiplying by the unit-image 1 can *shrink*. (Contrast the support tensor, whose analogous defect would *enlarge* $1 \star B$ with a phantom element. The two obstructions are independent, which is why a search calibrated to one sails past the other; the collapse tensor was missed for precisely this reason.) Injectivity of η_B — *tautness* — is thus *necessary* for closure, as is a pullback-preservation condition on the

[4]The monoidality laws and the non-polynomiality were cross-checked exhaustively on all sets of cardinality ≤ 3 (`scratch/collapse-tensor/collapse_hostile2.py`), with a two-element factor present to witness the non-injective collapse $2 \to 1$. This is a remark-level refinement of Theorem 4.8's landscape, not a Lean formalisation.

naturality squares of η. These conditions locate the failure. But for a *symmetric* tensor we can do better than locate it: we can *list* the survivors, and the list is short.

4.3.5 The complete list

The closure condition of Theorem 4.8 sorts the monoidal structures on **Set** by a single question: does the Day convolution close? The collapse tensor showed that both answers occur. We have spent the last pages at the wall between yes and no; the better question is what lives on the *yes* side. Which symmetric monoidal structures on **Set** have $(-)\star B$ polynomial for every B?

Here is the answer, and it is the cleanest kind: the two families we already met are the whole pile. Up to isomorphism, a symmetric $\star$ that survives the condition is either the product $\times$ or one of Garner's $\vee_S$ — so on containers the closed symmetric convolutional tensors are exactly $\otimes$ and the $\triangleright_S$ family. (One caveat rides along, an arity bound; it is real, and we meet it honestly at the end.) Neil's instinct that these closures are a law and not a coincidence is, in the bounded case, a theorem. Let us prove it.

Start by writing down what polynomiality gives us to work with. Each $R_B = (-)\star B$ has a normal form

$$X \star B \;\cong\; \sum_{u \in 1\star B} X^{A_{B,u}} \qquad \text{(naturally in } X\text{)},$$

so R_B is pinned by two pieces of data: its *index set* $1\star B = R_B(1)$, and over each index u its *arity*, the exponent set $A_{B,u}$. Write $d(B) := \sup_u |A_{B,u}|$ for the largest arity — the *degree* of R_B — and $\kappa := \sup_B d(B)$ for the degree of the whole family. Our two survivors are *affine* (degree ≤ 1): the product has $X\star B = B\times X$, index set B and every arity a single point; $\vee_S$ has $X\star B = B + (1 + S\times B)\times X$, a constant part B and a linear part $1 + S\times B$. We will show nothing else of bounded degree survives. Three moves do it.

Move 1: the unit is small.

Lemma 4.12 (The unit is small)**.** *Under the closure condition,* $|I| \le 1$.

Why, in two breaths. $R_1 = (-)\star 1$ is polynomial, say $R_1(X) = \sum_k X^{M_k}$. The right unitor makes $(-)\star I$ the identity, so $1\star I = 1$, and by symmetry $R_1(I) = I\star 1 = 1$. If $|I| \ge 2$ then each term I^{M_k} has at least one element, so a sum equal to 1 forces a single term with $M_k = \varnothing$: $R_1 \equiv 1$ is constant. Then $1\star\varnothing = 1$ as well, so $R_\varnothing$ has one index and $X\star\varnothing = X^A$ for a fixed set A. Evaluate at I: the left unit gives $\varnothing = I\star\varnothing = I^A$, which is impossible for nonempty I. So $|I| \ge 2$ cannot happen. □

The unit is $\varnothing$ or 1 — nothing exotic slips in underneath.[5]

Move 2: degrees multiply, and that kills high arity.

This is the engine. Associativity says $(X\star C)\star B \cong X\star(C\star B)$, that is $R_B \circ R_C \cong R_{C\star B}$: the family is closed under composition. Compose the normal forms and read the exponents off,

$$R_B\big(R_C(X)\big) \;=\; \sum_{b \in 1\star B} \;\sum_{\varphi : A_{B,b} \to 1\star C} X^{\sum_{i \in A_{B,b}} A_{C,\varphi(i)}},$$

so the arities of the composite are the sums $\sum_{i\in A_{B,b}} A_{C,\varphi(i)}$, and the largest of them has size $d(B)\cdot d(C)$. Hence

$$d(C\star B) \;=\; d(C)\cdot d(B)\,:$$

degree is a monoid homomorphism into the cardinals under multiplication. Now watch the trap close. Suppose some arity had two or more elements, so $\kappa \ge 2$, and suppose the family had

[5] The finite cardinality search agrees: over $\mathbb{N}$, the symmetric associative unital polynomial operations are exactly $a\cdot b$ (unit 1) and $a + b + s\,ab$ (unit 0, $s \ge 0$); units ≥ 2 admit none (`scratch/cardinality-classification.py`).

bounded degree, $\kappa < \infty$. Choose B realising the maximum, $d(B) = \kappa$. Then $d(B \star B) = \kappa^2 > \kappa$ — a degree strictly above the supposed maximum. That is a contradiction, and it leaves only one possibility.

Lemma 4.13 (Bounded degree forces affine)**.** *If $\kappa = \sup_B d(B)$ is finite then $\kappa \leq 1$: every R_B is affine, $X \star B \cong C_B + D_B \times X$ for sets C_B (the empty-arity indices) and D_B (the singleton-arity indices).*

The whole force of the classification is that squaring a degree-≥ 2 operation overflows any finite ceiling. It is the same phenomenon as $\vee_S$ staying affine under its own composition — 1 squared is 1 — turned into an obstruction for everything else.

Move 3: an affine, symmetric $\star$ is $\times$ or $\vee_S$.

Affineness is most of the distance; a single identity covers the rest, and it is the heart of the matter. Split on the unit (Lemma 4.12).

If $I = 1$: the right unit $X \star 1 = X$ gives $C_1 = \varnothing$, $D_1 = 1$, and in particular $R_1 = \mathrm{id}$. Then $1 \star \varnothing = \varnothing \star 1 = R_1(\varnothing) = \varnothing$, and since $R_\varnothing$ is affine and takes the value $\varnothing$ at 1, it is identically $\varnothing$: $X \star \varnothing = \varnothing$ for all X. Hence $C_B = \varnothing \star B = \varnothing$, and $C_B + D_B = 1 \star B = B$ gives $D_B = B$. So $X \star B = B \times X$: the tensor is the product.

If $I = \varnothing$: the left unit $\varnothing \star B = B$ gives $C_B = B$, so $X \star B = B + D_B \times X$. Now bring in symmetry. It says $X \star B \cong B \star X$, which spelled out is

$$B + D_B \times X \;\cong\; X + D_X \times B \qquad \text{naturally in both } X \text{ and } B.$$

Stare at this identity, because it does all the work. Read both sides as functors of B with X held fixed. The right-hand side is manifestly *affine in B* — it is $X + D_X \times B$, and D_X carries no B. So the left-hand side is affine in B too; its only not-obviously-affine part is $D_B \times X$, which forces D_B itself to be an affine functor of B:

$$D_B \;\cong\; D_\varnothing + S \times B$$

for some set S, the linear coefficient. Put $X = \varnothing$ in the identity: $B \cong D_\varnothing \times B$ naturally, so $D_\varnothing = 1$. Therefore $D_B = 1 + S \times B$, and

$$X \star B \;=\; B + (1 + S \times B) \times X \;=\; X + B + S \times X \times B \;=\; X \vee_S B.$$

Lemma 4.14 (The symmetry identity)**.** *Let $\star$ be symmetric with unit $\varnothing$ and every R_B affine, so $X \star B = B + D_B \times X$. Then the symmetry isomorphism forces $D_B \cong 1 + S \times B$ for a unique set S, and hence $\star \cong \vee_S$.* *[MacBeth]*

Uniqueness of S is free: it is the linear coefficient of the affine functor $B \mapsto D_B$, and associativity independently pins the associator by making D a strong monoidal functor $(\mathbf{Set}, \star, \varnothing) \to (\mathbf{Set}, \times, 1)$, which $1 + S \times B$ satisfies. The three moves now assemble into the payoff.

Theorem 4.15 (The complete list, bounded arity)**.** *Let $(\mathbf{Set}, \star, I)$ be a symmetric monoidal structure with $(-) \star B$ polynomial for every set B and with bounded arity ($\kappa < \infty$). Then $\star$ is isomorphic to exactly one of*

- *the product $\times$ (unit 1), whose Day convolution is the Dirichlet tensor $\otimes$; or*
- *$\vee_S$ (unit $\varnothing$) for a unique set S, whose Day convolution is Spivak's $\triangleright_S$ — with $\vee_\varnothing = +$ giving the categorical product $\times$ on* **Cont**.

Equivalently: the closed symmetric convolutional tensors on **Cont** *of bounded arity are precisely $\otimes$ and the $\triangleright_S$ family.* *[MacBeth (bounded arity)]*[6]

[6] The reconstruction was cross-checked on finite sets for $\times$ and $\vee_S$, $S \leq 3$: the affine normal form, associativity $R_B \circ R_C = R_{C \star B}$, the symmetry identity, and the strong-monoidal compatibility $D_{B \star C} = D_B \times D_C$, $D_\varnothing = 1$ all hold (`scratch/verify_reconstruction.py`).

This is the sentence the section was built to earn. The three closures we assembled by hand — cartesian, Dirichlet, and Spivak's third family — are not a sampler from a larger catalogue we have not yet drawn. In the bounded case there *is* no larger catalogue: they are all of it. The census of Theorem 4.8 becomes effective on the closed locus, and the collapse tensor is revealed for what it is — not a member we forgot, but the minimal structure ejected for keeping no provenance at all.

Remark 4.16 (The infinite-arity boundary). The hypothesis $\kappa < \infty$ is not a convenience; dropping it is a genuine open problem, and we state exactly what is missing rather than dress it as a formality. The engine of Move 2 was the inequality $\kappa^2 > \kappa$, and that inequality *fails* for infinite cardinals, where $\kappa^2 = \kappa$. The failure is structural, not a gap in the bookkeeping. Reframed, "every R_B is affine" is exactly "every R_B preserves connected *colimits*", whereas closure delivers only that each R_B preserves connected *limits* — and for polynomial functors those two preservation properties are logically independent, so no categorical shortcut converts one into the other. One can even write down a formal fixed point of the arity recursion at an infinite seed: taking $R_2 = y + y^\lambda$ with λ infinite, the associativity recursion reproduces $R_{2\star 2} = y + 2^\lambda \cdot y^\lambda$ with no cardinality obstruction and an associator natural in the visible variable. Counting is therefore provably blind to the infinite case. If an infinite-arity closed tensor is in fact impossible, the obstruction must live in the element-level pentagon, jointly in all variables — a coherence computation this book does not settle. So the classification is complete for bounded arity; the unbounded-arity case, and the non-symmetric case (a $\star$ that is left closed without being right closed, which the symmetry identity above never sees), are left open. [MacBeth; open problem]

4.3.6 The other three, and a boundary

The remaining structures round out the picture and each carries a small lesson.

The categorical product. $(\mathbf{Cont}, \times, 1)$ is cartesian closed, with exponential $[\![q^p]\!]R \cong \prod_{s \in S_p} [\![q]\!](R + p[s])$ — which is Theorem 4.8 at $\star = +$, since $\times$ is the Day convolution of $(+, \varnothing)$. This is Niu–Spivak's Theorem 5.31 [13], first observed by Altenkirch, Levy and Staton. The cautionary half: **Cont** is *not* locally cartesian closed — its slices are not all cartesian closed — so, despite the exponentials, it does not model extensional dependent type theory. Closure and pointwiseness pull in opposite directions: **Cont** is closed for the tensor whose extension is *not* pointwise $(\otimes)$ and fails to be closed in the strong local sense for the one whose extension *is* pointwise $(\times)$.

The composition product. $\triangleleft$ is not symmetric, so "closed" bifurcates. It is not closed on the left, but it does have a right coclosure — a functor C_q with $\mathbf{Cont}(p, q \triangleleft r) \cong \mathbf{Cont}(C_q\, p,\, r)$, computed by Josh Meyers [14, eqs. (68)–(69)], [13, Prop. 6.57]. A reader collating the two sources should be warned that the same isomorphism is called a *left* coclosure by Niu–Spivak and a *right* coclosure by Spivak; the variance convention differs, not the mathematics.

Outside the census. The Day recipe manufactures a tensor — closed or not — from every monoidal structure on **Set**, but not every tensor on **Cont** worth having is a Day convolution. Dorta, Jarvis and Niu's *Dialectica* tensors $\ltimes, \rtimes$ [15] are non-convolutional — their positions are not a fixed binary operation $p[s] \star q[t]$ applied fibrewise — and they behave differently under closure: $\ltimes$ is *not* closed, while $\rtimes$ is closed on the left in a directed sense. They mark the boundary of the classification of this section; their full treatment belongs with the fibrational material and is deferred.

Remark 4.17 (The sentence to carry away). Every monoidal structure on **Set** induces one on containers by Day convolution, and it is closed exactly when tensoring by a fixed set is a polynomial operation. The internal hom is then always the same shape: hold q, run the shapes of p, feed each position of p through $\star$ into q, and take the product. Cartesian closure, Dirichlet closure, and Spivak's third family are one formula read three times — and you learn whether

closure holds without ever leaving **Set**. That last clause is not a formality: the collapse tensor of Definition 4.10 is convolutional and *not* closed, so the polynomial condition is a genuine dividing line and not a fact every Day tensor enjoys for free.

4.4 Monoids and comonoids for the four structures

We now have four monoidal structures on **Cont**: the coproduct $+$, the product $\times$, the Dirichlet tensor $\otimes$, and the composition product $\triangleleft$. Each carries a notion of internal *monoid* ($\mu\colon c \odot c \to c$, $\eta\colon I \to c$) and internal *comonoid* ($\delta\colon c \to c \odot c$, $\varepsilon\colon c \to I$). That is eight questions. The purpose of this section is to answer all eight, and — more usefully — to say which of the answers are worth remembering.

Throughout, a container is $c = (S, P)$ with $P\colon S \to \mathbf{Set}$; its extension is $[\![c]\!](X) = \sum_{s \in S} X^{P(s)}$; a morphism $c \to c'$ is a forward shape map $\varphi\colon S \to S'$ together with backward position maps $\varphi^\sharp_s\colon P'(\varphi s) \to P(s)$ (the variance of Definition 2.1). The four units are the initial container $0 = (\emptyset, !)$ for $+$, the terminal container $1 = y^0 = (1, * \mapsto \emptyset)$ for $\times$, and the identity polynomial $y = y^1 = (1, * \mapsto 1)$ for both $\otimes$ and $\triangleleft$.

Here is the whole table; the rest of the section unpacks it.

$\odot$ (unit)	**comonoids**	**monoids**
$+$ (0)	only 0 itself [folklore]	every container, uniquely [folklore]
$\times$ (1)	every container, uniquely [folklore]	monoid on shapes with an *empty* unit-fibre [MacBeth, proved]
$\otimes$ (y)	families of monoids $\sum_s y^{M_s}$ [MacBeth, proved]	monoid on shapes + oplax fibre-functor [MacBeth, proved]
$\triangleleft$ (y)	small categories = directed containers [cited]	polynomial monads [cited]

The two *canonical* structures $+$ and $\times$ give nothing: their (co)monoids collapse. The two *Day-exotic* structures $\otimes$ and $\triangleleft$ carry all the content, and there it comes in dual pairs. We treat the collapse in one box and spend the rest of the section on the exotic pair.

The cartesian/cocartesian collapse

Because $\times$ is the categorical *product*, every container is a $\times$-comonoid in exactly one way: the comultiplication is the diagonal $\Delta\colon c \to c \times c$ and the counit is the unique map $c \to 1$. Dually, because $+$ is the *coproduct*, every container is a $+$-monoid in exactly one way, via the codiagonal $\nabla\colon c + c \to c$ and the initial map $0 \to c$. The two opposite corners degenerate the other way: a $+$-*comonoid* needs a counit $c \to 0$, i.e. a map $S \to \emptyset$, which forces $c \cong 0$; so the only $+$-comonoid is 0 itself.

Only the $\times$-*monoid* corner has any texture, and even it is thin. A unit $\eta\colon 1 \to c$ has a backward part $P(e) \to 1[*] = \emptyset$, which exists *only if the unit shape has no positions*. So a $\times$-monoid is an ordinary monoid $(S, \cdot, e)$ on the shape set whose identity e has an empty fibre, together with a coherent backward routing $P(s \cdot t) \to P(s) \sqcup P(t)$ of each output position into one factor. A container all of whose shapes have non-empty fibres admits *no* $\times$-monoid at all; when every fibre is empty one recovers plain monoids on S (counts $1, 4, 33$ for $|S| = 1, 2, 3$). It is a genuine, if minor, container-specific notion — worth naming once and not dwelling on. [MacBeth, proved (empty-unit-fibre refinement)]

4.4.1 The composition product: categories and monads

For $\triangleleft$ the answers are the two halves of the polynomial dictionary, and both are the subject of later chapters, so we only name them here.

Theorem 4.18 ($\triangleleft$-comonoids and $\triangleleft$-monoids)**.** *A comonoid in* $(\mathbf{Cont}, \triangleleft, y)$ *is exactly a* directed container*, equivalently a* small category *with object set* S *(Chapter 5). A monoid in* $(\mathbf{Cont}, \triangleleft, y)$ *is exactly a* polynomial monad*, equivalently a monad on* **Set** *whose functor is polynomial. [Cited: Ahman–Uustalu 2016 [7]; Dorta–Jarvis–Niu 2023 Thm 4.3 [15]; Gambino–Kock 2013 Thm 4.5 [3]]*

The comonoid side is developed in full in Chapter 5, including the morphism refinement $\mathbf{DCont} \cong \mathbf{Cof}$; MacBeth's Lean formalisation of the comonad law (Theorem 5.3) is the object-level half. The monad side is Gambino–Kock [3]; the indexed refinement — monoids in $(I\text{-}\mathbf{Cont}, \triangleleft, y)$ are monads on $\mathbf{Set}^I$ — is De Pascalis–Uustalu–Veltrì [16], and the free-monad instance is the grafting construction developed in Chapter 6. The point to carry forward is the shape of the dictionary: $\triangleleft$*-comonoid* = *category,* $\triangleleft$*-monoid* = *monad.* It is the pattern we are about to see again, in a purer form, for $\otimes$.

4.4.2 The Dirichlet tensor: a lax/oplax duality

The Dirichlet tensor multiplies position sets: $[\![c \otimes d]\!](X) = \sum_{s,t} X^{P(s)\times Q(t)}$, with unit y. Its (co)monoids are the prettiest entry in the table, because the comonoid/monoid duality here *is* a lax/oplax duality — visibly, in coordinates. We state the two classifications and then read the duality off them.

Theorem 4.19 ($\otimes$-comonoids are families of monoids)**.** *A comonoid in* $(\mathbf{Cont}, \otimes, y)$ *is exactly a container* $c = \sum_{s\in S} y^{M_s}$ *in which each position set* $M_s := P(s)$ *carries an* arbitrary *monoid structure. The comultiplication is forced to be the diagonal on shapes,* $\delta_1 = \Delta\colon S \to S \times S$*, and on positions it is the fibrewise multiplication* $M_s \times M_s \to M_s$*; coassociativity is associativity of each* M_s *and the counit is its unit. On morphisms this is a fibrewise monoid homomorphism, so*

$$\mathrm{Comon}(\mathbf{Cont}, \otimes, y) \;\cong\; \mathbf{Fam}(\mathbf{Mon}^{\mathrm{op}}),$$

the cocommutative comonoids being $\mathbf{Fam}(\mathbf{CMon}^{\mathrm{op}})$*.* *[MacBeth, proved; Lean-verified (forward)]*[7]

Theorem 4.20 ($\otimes$-monoids are monoids with an oplax fibre-functor)**.** *A monoid in* $(\mathbf{Cont}, \otimes, y)$ *is exactly a monoid* $(S, \cdot, e)$ *on the shape set together with an* oplax monoidal functor $P\colon (S, \cdot, e) \to (\mathbf{Set}, \times, 1)$ *on the fibres — that is, an oplax structure map* $\phi_{s,t}\colon P(s \cdot t) \to P(s) \times P(t)$ *and an oplax unit* $P(e) \to 1$*, subject to the oplax coherence (unit and associativity) laws. The multiplication* $\mu\colon c \otimes c \to c$ *has forward part the shape multiplication* $\cdot$ *and backward part* ϕ*; the unit* $\eta\colon y \to c$ *picks* e *with the forced terminal map* $P(e) \to 1$*.* *[MacBeth, proved; Lean-verified (forward)]*[8]

Now put the two theorems side by side. A comonoid comultiplies, so its shape part is a map $S \to S \times S$ — a comonoid in $(\mathbf{Set}, \times)$, and the only one is the *diagonal.* Forced. A monoid multiplies, so its shape part is a map $S \times S \to S$ — a monoid in $(\mathbf{Set}, \times)$, and that can be *any* monoid. Free. The single arrow reversal $\delta\colon c \to c \otimes c$ versus $\mu\colon c \otimes c \to c$ flips the

[7] Proved by unwinding the comonoid axioms in container coordinates (`proofs/2026-07-17-bare-dirichlet-comonoid.md`); the forward map $\otimes$-comonoid $\to$ $\mathbf{Fam}(\mathbf{Mon}^{\mathrm{op}})$ is machine-checked in `lean/Containers/Containers/DirichletComonoid.lean` (`toFamilyOfMonoids`, `#print axioms` = [`Quot.sound`]). This answers the **Poly**/$\otimes$ slice of Niu–Spivak's open Question 5 of Chapter 9 [13] by an elementary computation — not a deep theorem, but a clean one.

[8] Proved as the dual/sibling of Theorem 4.19 (`proofs/2026-07-19-dirichlet-monoid-classification.md`), with two independent enumerations agreeing on the counts. The forward map is machine-checked in `lean/Containers/Containers/DirichletMonoid.lean` (`toShapeMonoidOplaxFibres`, `#print axioms` = $\varnothing$). Niu–Spivak flag the $\otimes$-monoids as future work in Remark 3.78 [13]; this is an elementary answer to that item, orthogonal to the $\triangleleft$-monoid classification of De Pascalis–Uustalu–Veltrì [16], and is not claimed as a deep result.

shape map between "forced diagonal" and "arbitrary monoid," and simultaneously flips the fibre data between a *monoid on each fibre* (a lax assignment: $M_s \times M_s \to M_s$) and an *oplax functor* (structure maps $P(s \cdot t) \to P(s) \times P(t)$ pointing the other way).

Remark 4.21 (The astonishment). For the Dirichlet tensor, the comonoid/monoid duality *is* the lax/oplax duality. A $\otimes$-comonoid is a *lax* gadget — a monoid living inside each fibre — glued to the forced diagonal on shapes; a $\otimes$-monoid is the exact *oplax* mirror — structure maps out of each fibre — glued to a free choice of monoid on shapes. Reversing the one comultiplication/multiplication arrow does both flips at once. This is the cleanest instance in the book of a recurring theme: dualising a structure need not merely swap inputs and outputs; it can swap the *kind* of structure — lax for oplax, forced for free.

Remark 4.22 (One table, four statements about polynomial functors). The extension functor $[\![-]\!]\colon \mathbf{Cont} \to [\mathbf{Set}, \mathbf{Set}]$ is strong monoidal for all four structures at once: it sends $+, \times, \otimes, \triangleleft$ respectively to the pointwise coproduct, the pointwise product, the Dirichlet product, and composition of polynomial endofunctors. Consequently each row of the table is simultaneously a statement about polynomial endofunctors: the $\triangleleft$ row is the comonad/monad dictionary, the $\otimes$ row classifies Dirichlet-product (co)monoids, and the $+, \times$ rows the pointwise ones. The directed containers of the next chapter are the $\triangleleft$-comonoid cell, read as comonads on $\mathbf{Set}$. [Cited: Niu–Spivak 2023 (extension is strong monoidal)]

4.5 An aside: the lens subcategory

There is one full subcategory of **Cont** worth pointing out before we leave the algebra behind, because it is exactly where compositional systems people already live. Restrict to the containers whose position set is *constant* in the shape — the *monomials* $S \cdot y^A = (S,\ s \mapsto A)$ [13, Def. 2.15].

Monomials are bimorphic lenses

The monomials span a full subcategory of **Cont** equivalent to the category of *bimorphic lenses*. Unwinding the container morphism $S \cdot y^A \to T \cdot y^B$: the forward shape map is a *get* $f\colon S \to T$, and the backward position map, constant in the shape, is a *put* $f^\sharp\colon S \times B \to A$. These are exactly the two components of a bimorphic lens between (S, A) and (T, B), and the container composition law reproduces lens composition. [Cited: Niu–Spivak 2023, Example 3.41 (monomials $\simeq$ bimorphic lenses, after Hedges); get/put as in eqn. (3.42)][a]

[a] Convention: we present the raw morphism data $f\colon S \to T$, $f^\sharp\colon S \times B \to A$. Niu–Spivak's eqn. (3.42) relabels the get as $J \to I$ under the functional-programming reading "state = domain"; the underlying data is the same.

Which of the categorical structures of this chapter does this subcategory inherit? The answer is a clean dividing line — and it is worth stating carefully, because the intuition "lenses are a poorer setting, they lack most of the structure" overshoots.

Proposition 4.23 (What the lens subcategory keeps and loses)**.** *The full subcategory of monomials in* **Cont** *is closed under*

- binary products*:* $S \cdot y^A \times T \cdot y^B = (S \times T) \cdot y^{A+B}$*, again a monomial;*
- *the* terminal object $1 = 1 \cdot y^0$ *and the* initial object $0 = \emptyset \cdot y^A$*;*

and is not *closed under*

- binary coproducts*:* $S \cdot y^A + T \cdot y^B$ *is a monomial only when* $|A| = |B|$ *(the two summands carry different, unequated fibres);*
- initial algebras *(W-types, and the free monad of Chapter 6): the leaf-position set varies with the depth of the tree, so the fixpoint is not a monomial.*

[MacBeth, verified (hand + exhaustive check on small monomials)][9]

So the line does not fall where one might guess. Products, the terminal object and the initial *object* all stay inside the lens world; what escapes it is precisely the *colimit / inductive* side — binary coproducts and initial algebras. The lens subcategory is poorer in colimits and recursion, not in limits. That is the clean teaching point: bidirectional transformations compose and take products freely, but the moment you *branch* (coproduct) or *recurse* (initial algebra) you leave the monomials and need the full generality of **Cont**.

4.6 Two predicate liftings: All and Exists

Here is a construction Neil Ghani arrived at independently, and it turns out to name two things this book has kept apart. Give a container $X = S_X \triangleleft P_X$ and a *predicate* on its positions — for now, just a rule that attaches a set to each position, depending on the value sitting there. There are two obvious ways to combine those sets over the positions of a shape: take them *all* at once, or ask for *some* one of them. "All" is a product $\prod$; "some" is a coproduct $\coprod$. Two liftings, born of a single container, differing only in $\prod$ versus $\coprod$.

The promise of this section is that these two liftings are not new objects at all. One of them is the composition product $\triangleleft$ you already met in the menagerie (§4.3) — so the "exists" lifting *is* sequencing. The other is a construction you cannot always perform, and *why* you cannot is the boundary the whole rest of the book circles: it exists on all of **Cont** only when the map it rides is position-preserving. The punchline is a clean law relating the two — a Fubini identity that makes "for all, and then for all" into "for all pairs."

4.6.1 The two liftings, defined

Fix a container $X = S_X \triangleleft P_X$. A *predicate* we take to be a second container $Y = S_Y \triangleleft P_Y$: its shapes S_Y are the *values* the predicate can report, and its positions $P_Y(t)$ are the *witnesses* carried by the value t. (This is exactly the container fibration $\mathbf{Cont} \to \mathbf{Set}$, $S \triangleleft P \mapsto S$: an object over a shape set is a position family, i.e. a container. We return to it below.) Evaluate the predicate at a filled shape of X: an element of $[\![X]\!]S_Y$ is a pair (s, g) with $s \in S_X$ and $g\colon P_X(s) \to S_Y$ — a shape of X with a value $g(p)$ hung at each position p.

Definition 4.24 (The two liftings of a container)**.** For X and Y as above and $(s, g) \in [\![X]\!]S_Y$,

$$(\mathsf{All}\ X\ Y)(s,g) \;=\; \prod_{p \in P_X(s)} P_Y\big(g(p)\big), \qquad (\mathsf{Exists}\ X\ Y)(s,g) \;=\; \coprod_{p \in P_X(s)} P_Y\big(g(p)\big).$$

Each assembles into a bifunctor on **Cont** with the *same* shapes $[\![X]\!]S_Y = \coprod_s S_Y^{P_X(s)}$ and differing only on positions:

$$\mathsf{A}\,X\,Y = \big([\![X]\!]S_Y,\ \mathsf{All}\ X\ Y\big), \qquad \mathsf{E}\,X\,Y = \big([\![X]\!]S_Y,\ \mathsf{Exists}\ X\ Y\big).$$

[The liftings and the two operations A, E are Neil Ghani's (personal communication, 2026); the identifications and laws below are MacBeth's.]

Example 4.25 (A node with two children)**.** Let X have a single shape with two positions, $P_X = \{1, 2\}$ (a binary node), and let $Y = \mathsf{Maybe} = \{\mathrm{n}, \mathrm{j}\} \triangleleft P_Y$ with $P_Y(\mathrm{n}) = \varnothing$, $P_Y(\mathrm{j}) = \{*\}$

[9] The product formula is also Niu–Spivak's (p. 134); the coproduct and initial-algebra failures are checked by hand and confirmed by an exhaustive 81/81 enumeration in `scratch/2026-07-19-lens-monomial-closure.md`.

(Example 1.2). A filled shape is a choice $g\colon \{1,2\} \to \{\mathrm{n},\mathrm{j}\}$ — one of the four words nn, nj, jn, jj. Then All takes the *product* of the two witness sets and Exists their *sum*:

g	$\mathsf{All} = P_Y(g1) \times P_Y(g2)$	$\mathsf{Exists} = P_Y(g1) + P_Y(g2)$
nn	$\varnothing \times \varnothing = \varnothing$	$\varnothing + \varnothing = \varnothing$
nj	$\varnothing \times 1 = \varnothing$	$\varnothing + 1 = 1$
jn	$1 \times \varnothing = \varnothing$	$1 + \varnothing = 1$
jj	$1 \times 1 = 1$	$1 + 1 = 2$

All is nonempty only when *every* position reports a witness (jj); Exists is nonempty as soon as *some* position does, and at jj, where both positions report, it records *which* one — the two ways to pick that All collapses to a single point. Hold on to that last cell: it is the whole difference between the two liftings.

4.6.2 The first surprise: E is the composition product

Unfold $\mathsf{E}\,X\,Y$ against the definition of $\triangleleft$. A shape of $X \triangleleft Y$ is an X-shape with a Y-shape planted at each X-position, i.e. a pair (s, g) with $g\colon P_X(s) \to S_Y$ — the shapes of $\mathsf{E}\,X\,Y$ on the nose. A position of $X \triangleleft Y$ over (s, g) is an X-position p *together with* a Y-position beneath the value there, i.e. an element of $\coprod_{p \in P_X(s)} P_Y(g(p))$ — the positions of Exists on the nose.

Proposition 4.26 ($\mathsf{E} = \triangleleft$)**.** $\mathsf{E}\,X\,Y = X \triangleleft Y$*, and thus* $(\mathbf{Cont}, \mathsf{E}, 1) = (\mathbf{Cont}, \triangleleft, y)$ — *the monoidal spine of this book (Definition 4.4).* E *is a bifunctor on* all *of* **Cont**. *[MacBeth (identification); the $\triangleleft$-monoidal structure is Lean-verified, `lean/Containers/Containers/Monoidal.lean` (pentagon and triangle by `rfl`).]*

The reader hook Neil offers is worth repeating, because it makes $\coprod$ inevitable. Read X as a conversation with several prompts (its positions) and Y as a follow-up conversation. A *reply* in the composite $X \triangleleft Y$ is a reply to one prompt of X *plus* a reply to the follow-up it opened — you choose *which* prompt you are answering, then answer downstream of it. Choosing which, then answering, is exactly $\coprod_p P_Y(g(p))$. Sequencing is existential.

4.6.3 The second surprise: A lives only on cartesian morphisms

Now try the same for A, and watch it fail in an instructive way. Both liftings share their shape map: a container morphism $\alpha = (u, f)\colon X \to X'$ sends $(s, g) \mapsto (u\,s,\ g \circ f_s)$, using the backward position map $f_s\colon P_{X'}(u\,s) \to P_X(s)$ to re-address the values. That is defined for *every* α. The liftings part on *positions*, and this is where the variance bites.

- For E the induced position map is on *sums*, $\coprod_{p'} P_Y(g\,f p') \to \coprod_p P_Y(g\,p)$, sending $(p', b) \mapsto (f p', b)$. A coproduct pushes forward *along* f — always available.
- For A it would be on *products*, $\prod_{p'} P_Y(g\,f p') \to \prod_p P_Y(g\,p)$. A product is *contravariant* in its index set: to produce a coordinate at each $p \in P_X(s)$ from coordinates indexed by $p' \in P_{X'}(u\,s)$ you must re-address p *backward* through f — you need a section of f.

How many such sections are there? Exactly $\prod_{p \in P_X(s)} f_s^{-1}(p)$, the choices of a preimage for each position. This one count settles everything.

Theorem 4.27 (A is a cartesian-only bifunctor)**.** *The natural pushforwards* $\mathsf{A}\,X\,Y \to \mathsf{A}\,X'\,Y$ *over* $\alpha = (u, f)$ *are indexed by* $\prod_s \prod_{p \in P_X(s)} f_s^{-1}(p)$*. Hence:*

- *if some* f_s *is* not surjective*, that product is* empty — no *natural map exists;*
- *if every* f_s *is surjective but some is not injective, maps exist but* none is canonical*;*
- *if every* f_s *is a* bijection *(*α cartesian*), the product is a* singleton*: the pushforward is forced,* $\theta_s = f_s^{-1}$.

So A *extends canonically to a bifunctor* $\mathbf{Cont}_{\mathrm{cart}} \times \mathbf{Cont} \to \mathbf{Cont}$*, and to no non-cartesian morphism. In its* second *argument* A *is functorial for all morphisms, which act inside each factor pointwise.* *[MacBeth, proved. The section-set count is a one-line Yoneda calculation (a natural transformation between the representable functors* $\mathbf{Set}^{P}(D,-)$*, with* $D(p) = f^{-1}(p)$*); details and a* 3000/3000 *random check in* `proofs/2026-08-08-A-E-predicate-liftings.md`*, §2.]*

This is the object-level shape of a flag Neil raised: *"I cannot see how to define* A *on polynomial functors, and it is functorial in the first argument only on cartesian morphisms."* The theorem says exactly why, and that it is not a defect of imagination: along a morphism that drops a position, there is no natural pushforward to be had, because $\prod$ has no coordinate to carry at the dropped position. Contrast $\coprod = \mathsf{E}$, which simply forgets that position's contribution and moves the rest forward.

Remark 4.28 (This is T_M, one level down). $\mathsf{All} = \prod$ is the proof-relevant $\prod$-lifting T_M of Ahman–Bauer [10] ("proof-relevant" is the type theorist's name for *records witnesses, not truth*; we use the fibrational wording and log the synonym here, once). Take the first argument to be a monad's multiplication $\alpha = \mu^M \colon M \triangleleft M \to M$. Theorem 4.27 then reads: μ^M lifts to a *canonical* T_M-multiplication *iff* μ^M is cartesian. So the tempting "core theorem" — *every* **Set** *monad lifts to a container monad via* T_M — is **false**. It fails outright for the reader and state monads, whose μ^M *drops* leaves: this is exactly Theorem 4.27's non-surjective case, where no natural pushforward exists at all. It *survives* for a wide class — the "polynomial" monads — that includes List (whose μ is cartesian) and, more delicately, finite powerset Pf, whose $\mu = \bigcup$ *merges* leaves rather than dropping them and so still supports a multiplication, though not the cartesian-canonical one. So cartesianness is the border for the *canonical* lift, while merging buys a non-canonical one and dropping buys nothing: the precise ladder — with two fibrations, a $\mathbb{Z}/2$ grading, and the $\coprod$-lifting that Reader *does* carry — is §6.6.2 in Chapter 6. Neil's functoriality flag and our "multiplication drops leaves" are one statement. [MacBeth, proved (`proofs/2026-08-07-proof-relevance-boundary.md`); $\prod$-lift is [10]. Two-fibration framing: Hermida–Jacobs [38].]

4.6.4 The law: A is a module of $\triangleleft$

Cartesian-only or not, A obeys a clean composition law at the level of objects, and it is the reason Neil wanted these two operations named side by side. The law is a Fubini identity for the dependent product.

Theorem 4.29 (The action law)**.** *For all containers* X, Y, C*,*

$$\mathsf{A}\,X\,(\mathsf{A}\,Y\,C) \;\cong\; \mathsf{A}\,(X \triangleleft Y)\,C,$$

naturally in C*, with unit* $\mathsf{A}\,y\,C = C$*. Thus* A *is a* left module *of the* $\triangleleft$*-monoidal category* $(\mathbf{Cont}, \triangleleft, y) = (\mathbf{Cont}, \mathsf{E}, 1)$ *acting on* $\mathbf{Cont}$*: the object-level associativity and unit equations hold unconditionally; as a* functorial *action it lives on* $\mathbf{Cont}_{\mathrm{cart}}$ *(Theorem 4.27).* *[MacBeth, proved (`proofs/2026-08-08-A-E-predicate-liftings.md`, §3; shape/position multiset check* 2000/2000*).]*

Why it holds. On positions the identity is pure Fubini for $\prod$. A position of the left-hand side over a filled shape is an element of $\prod_{p \in P_X(s)} \big(\prod_{q \in P_Y(t_p)} P_C(\cdots)\big)$ — for every X-position, a choice over every Y-position beneath it — and $\prod_p \prod_q = \prod_{(p,q)}$ collapses this to a single choice over the composite positions $\coprod_p P_Y(t_p)$ of $X \triangleleft Y$, which is precisely a position of the right-hand side. On shapes the matching iso is the distributivity of $\prod$ over $\coprod$ (type-theoretic choice) followed by currying. Nothing about a base monad enters: the law is a fact about $\prod$ and $\triangleleft$ alone. □

The intuition Neil gives is debate preparation. To hold $\mathsf{A}\,X\,(\mathsf{A}\,Y\,C)$ is to prepare, for *every* line of questioning p your opponent might open, and then under it for *every* follow-up q, a full

answer. Fubini says that is the same as being ready for every *pair* (p, q) at once — a single "for all" over the composite conversation $X \triangleleft Y$. Universal preparation composes.

Remark 4.30 (The contrast that names the two structures). The parallel law for E, $\mathsf{E}\,X\,(\mathsf{E}\,Y\,C) = \mathsf{E}\,(X \triangleleft Y)\,C$, is nothing but associativity of $\triangleleft$ (Proposition 4.26): E acts by *being* the monoidal product. A acts *alongside* it, as a module. The mixed law does *not* hold: $\mathsf{A}\,X\,(\mathsf{E}\,Y\,C)$ and $\mathsf{E}\,(\mathsf{A}\,X\,Y)\,C$ carry $\prod_p$ against $\coprod_p$ in the same slot, and these agree iff every shape of X carries *exactly one* position (X linear) — strictly finer than non-branching, since an empty-position shape already breaks it ($\coprod_\varnothing = 0 \neq 1 = \prod_\varnothing$). $\prod$ and $\coprod$ genuinely refuse to commute, and that refusal is the same branching gap that obstructs the arrow-category composition of Chapter 6. [MacBeth, proved (`proofs/2026-08-08-A-E-predicate-liftings.md`, §4).]

4.6.5 Where this points

Two liftings, one container. $\mathsf{E} = \coprod$ is sequencing, defined everywhere, and its self-composition is just associativity of $\triangleleft$. $\mathsf{A} = \prod$ is universal quantification, defined only over cartesian morphisms, and its composition with itself is the module law above. The asymmetry — $\coprod$ pushes forward, $\prod$ needs a section — is the same variance that decides, in Chapter 6, which **Set**-monads lift to **Cont** and which only lift their existential shadow. We have met the boundary here in miniature, on a single morphism; there we meet it as a fibration and read the whole ladder off it.

Remark 4.31 (Honesty). The action law is stated as a canonical isomorphism of containers. Whether it is a strict definitional equality in a given encoding — as it is for the $\triangleleft$-associator, whose coherence is `rfl` in Lean — is a formalisation question flagged but not settled here. We have checked the two module *equations* (associativity and unit) at the object level; the module *coherence* 2-cells (the pentagon and triangle relating the associator to $\triangleleft$'s own) are automatic for a strict encoding and remain to be verified for a bicategorical one.

Chapter 5

Monoids and comonoids in Cont: directed containers, categories, and the free monoid

This is the heart of the book. We study the two internal algebraic structures for the composition product $\triangleleft$. On the *comonoid* side — the bulk of the chapter — we show that comonoids for $\triangleleft$ are *directed containers*, that directed containers are *small categories*, and — the refinement that the rest of the grant turns on — that the right *morphisms* are *cofunctors*, not functors. On the *monoid* side, we recall that a $\triangleleft$-monoid is a polynomial monad (Theorem 4.18) and, at the close of the chapter, *state* the one construction we will need in Chapter 6: the *free* $\triangleleft$-monoid on a container. Its universal property is deferred — and proved — in the next chapter; here we only pin down what it is.

5.1 Directed containers

Definition 5.1 (Directed container)**.** A *directed container* is a container $S \triangleleft P$ together with

- a *root* $\mathsf{o}(s) \in P(s)$ for each s;
- a *sub-shape* operation $s \downarrow p \in S$ for $p \in P(s)$;
- a *shift* $p \oplus q \in P(s)$ for $p \in P(s)$, $q \in P(s \downarrow p)$;

subject to the five laws (for all s, $p \in P(s)$, $q \in P(s \downarrow p)$, $q' \in P((s \downarrow p) \downarrow q)$):

$$\text{(D1)}\quad s \downarrow \mathsf{o}(s) = s; \qquad \text{(D2)}\quad \mathsf{o}(s) \oplus q = q; \qquad \text{(D3)}\quad p \oplus \mathsf{o}(s \downarrow p) = p;$$
$$\text{(D4)}\quad s \downarrow (p \oplus q) = (s \downarrow p) \downarrow q; \qquad \text{(D5)}\quad (p \oplus q) \oplus q' = p \oplus (q \oplus q').$$

D2, D3, D5 make each $P(s)$ a monoid-like structure with unit $\mathsf{o}(s)$; D1, D4 track how sub-shapes nest. (The well-typedness of D5 needs D4 — exactly the dependency that recurs as a dependent-cast obstacle in the Lean formalisation.)

5.2 Directed containers are comonads

Definition 5.2 (The comonad of a directed container)**.** On $D := [\![S \triangleleft P]\!]$ define

$$\varepsilon(s, v) = v(\mathsf{o}(s)), \qquad \delta(s, v) = \big(s,\ \lambda p.\,(s \downarrow p,\ \lambda q.\, v(p \oplus q))\big).$$

Theorem 5.3 (Comonad laws $\iff$ D1–D5)**.** *(D, ε, δ) is a comonad, and each comonad law is exactly one directed-container law: left counit is D1 (shape) and D2 (value); right counit is D3; coassociativity is D4 (shapes) and D5 (deepest values). Conversely a comonad of this form forces D1–D5.* *[Cited: Ahman–Chapman–Uustalu 2014; MacBeth, Lean-verified: Directed.lean]*

Proof sketch. Direct calculation. For left counit, $\varepsilon(\delta(s,v))$ reduces to $(s \downarrow \mathsf{o}(s),\ \lambda q.\, v(\mathsf{o}(s) \oplus q))$, which is (s,v) by D1 then D2. Right counit is D3; coassociativity unfolds both composites to a common triple nest, equal by D4 (inner shapes) and D5 (deepest values). The full computation, and the converse, are checked with zero `sorry` in the committed file `lean/Containers/Containers/Directed.` (`left_counit`, `right_counit`, `coassoc`); the "container is a comonad" direction is Ahman–Chapman–Uustalu [6]. □

5.3 Directed containers are small categories

Theorem 5.4 (Object dictionary)**.** *Directed-container structures on $S \triangleleft P$ are in bijection with small categories $\mathcal{C}$ with object set S and, for each s, $P(s) = \coprod_{s'} \mathcal{C}(s,s')$ the morphisms* out of *s, under*

$$\mathrm{ob}\,\mathcal{C} = S, \quad \mathcal{C}(s,s') = \{p \in P(s) : s \downarrow p = s'\}, \quad \mathrm{id}_s = \mathsf{o}(s), \quad q \circ p = p \oplus q.$$

In particular $s \downarrow p = \mathrm{cod}(p)$, and every small category arises from a unique directed container. *[Cited: Ahman–Uustalu 2016]*

Combining Theorems 5.3 and 5.4 with the polynomial incarnation gives the object-level chain promised in the Preface: a directed container is a comonoid in $(\mathbf{Poly}, \triangleleft)$, hence

Containers $\supseteq$ **Directed Containers** $\cong$ **Polynomial Comonoids** $\cong$ **Small Categories**.

5.4 The morphism refinement: DCont $\cong$ Cof

A dictionary on objects invites a dictionary on morphisms. The naive guess — a morphism of directed containers is a *functor* — is *false on variance*. A container morphism has a backward position map $f^\sharp_s \colon P'(fs) \to P(s)$, whereas a functor pushes morphisms forward. The maps native to directed containers are *cofunctors*.

Definition 5.5 (Cofunctor / retrofunctor)**.** A *cofunctor* $\varphi \colon \mathcal{C} \leftharpoonup \mathcal{D}$ is an object map $\varphi_0 \colon \mathrm{ob}\,\mathcal{C} \to \mathrm{ob}\,\mathcal{D}$ together with a *lift*: for each object c and each $h \colon \varphi_0 c \to d$ in $\mathcal{D}$, a morphism $\varphi^\uparrow(c,h) \colon c \to c'$ in $\mathcal{C}$, subject to

$$\text{(C0)}\ \varphi_0(\mathrm{cod}\varphi^\uparrow(c,h)) = \mathrm{cod}(h); \quad \text{(C1)}\ \varphi^\uparrow(c, \mathrm{id}_{\varphi_0 c}) = \mathrm{id}_c;$$
$$\text{(C2)}\ \varphi^\uparrow(c, h' \circ h) = \varphi^\uparrow(c', h') \circ \varphi^\uparrow(c,h), \quad c' = \mathrm{cod}\varphi^\uparrow(c,h).$$

Small categories and cofunctors form a category **Cof**. [Cited: Aguiar 1997; Clarke 2020; Spivak 2021 (Def. 3.21)]

Remark 5.6 (**Cof** $= \mathbb{C}\mathrm{at}^\sharp$). **Cof** is exactly $\mathbb{C}\mathrm{at}^\sharp = \mathrm{Comod}(\mathbf{Poly})$, the category of comonoids in $(\mathbf{Poly}, \triangleleft)$ and their morphisms: a cofunctor *is* a comonoid homomorphism [12, 18]. This is why the object dictionary (Theorem 5.4) cannot extend to **Cat**: functors are carried not by comonoid morphisms but by bicomodules [17].

Definition 5.7 (Directed-container morphism)**.** A *morphism of directed containers* $D \to D'$ is a container morphism $(f, f^\sharp)$ with $f \colon S \to S'$ and $f^\sharp_s \colon P'(fs) \to P(s)$, satisfying, for all s, $h \in P'(fs)$, $k \in P'((fs) \downarrow' h)$,

$$\text{(M0)}\ f(s \downarrow f^\sharp_s(h)) = (fs) \downarrow' h; \quad \text{(M1)}\ f^\sharp_s(\mathsf{o}'(fs)) = \mathsf{o}(s);$$
$$\text{(M2)}\ f^\sharp_s(h \oplus' k) = f^\sharp_s(h) \ \oplus\ f^\sharp_{s \downarrow f^\sharp_s(h)}(k).$$

Theorem 5.8 (The morphism dictionary)**.** *The assignment* $(f, f^\sharp) \mapsto (\varphi_0 := f,\ \varphi^\uparrow(s,h) := f^\sharp_s(h))$ *is a bijection on every hom-set carrying (M0)–(M2) to (C0)–(C2) term for term, and it respects identities and composition. It is moreover a bijection on objects (Theorem 5.4). Hence there is an isomorphism of categories*

$$\mathbf{DCont} \ \cong\ \mathbf{Cof}.$$

[Cited: Ahman–Uustalu 2016 §3.2–3; Shapiro–Spivak 2023, Cor. 5.12][1]

Proof sketch. Under the object dictionary a position $h \in P'(fs)$ is a morphism out of fs and $f^\sharp_s(h)$ a morphism out of s, so $(f, f^\sharp)$ and $(\varphi_0, \varphi^\uparrow)$ are literally the same data. Reading $\oplus$ as reverse composition, (M0)⇔(C0) (codomain), (M1)⇔(C1) (unit), (M2)⇔(C2) (composition). Identities and composites match, and objects are in bijection by Theorem 5.4; a bijective-on-objects, fully faithful functor is an isomorphism of categories. The full term-by-term match is in `papers/dcont-cof.tex` (Lemma 2.5–Theorem 2.7 there). □

Proposition 5.9 (Functors are the opposite variance)**.** *The* covariant *position maps* $\phi_s \colon P(s) \to P'(fs)$ *subject to the mirror conditions (N0)–(N2) of (M0)–(M2) are in bijection with* functors $\mathcal{C}(D) \to \mathcal{C}(D')$*. Thus a directed structure carries two notions of map of opposite variance: covariant gives functors, contravariant gives cofunctors, and only the latter is a morphism of the underlying container.* *[MacBeth]*[2]

Corollary 5.10 (Cofunctors are the Put of a delta lens)**.** *A* delta lens $A \rightleftarrows B$ *is a* Get *functor together with a* Put *cofunctor sharing one object map, related by the section condition (PutGet)* $f \cdot \varphi(a,u) = u$*. Under* **DCont** $\cong$ **Cof** *the Put cofunctor is exactly a morphism of directed containers; so a directed-container morphism is the update-propagating half of a bidirectional transformation.* *[Cited: Clarke 2022 (arXiv:2108.00390), Def. 4 / Thm. 9]*[3]

5.5 The dual side: monoids in Cont and the free monoid

The whole chapter so far has been about *comonoids* for $\triangleleft$. Their mirror-image, the $\triangleleft$*-monoids*, we have already named: by Theorem 4.18 a monoid in $(\mathbf{Cont}, \triangleleft, y)$ is exactly a *polynomial monad* — a monad on **Set** whose functor is a container extension. Where comonoids gave us categories, monoids give us monads; where the comonoid comultiplication δ *split* a state into a state-of-states, the monoid multiplication $\mu \colon c \triangleleft c \to c$ *composes* two layers of operations into one.

We do not develop the monoid side here — that is Chapter 6. But one monoid will be the engine of that chapter, and it is cleanest to *state* it now, while the $\triangleleft$-monoid laws are fresh, and *use* it there. It is the *free* $\triangleleft$-monoid: the smallest monad built from a bare container.

Proposition 5.11 (The free $\triangleleft$-monoid on a container — statement)**.** *Every container* $C = S \triangleleft P$ *generates a* $\triangleleft$*-monoid* $C^* = S^* \triangleleft P^*$*, the* free $\triangleleft$-monoid on C*, whose shapes are the well-founded* C*-trees with a variable-leaf constructor and whose positions are the variable leaves:*

$$S^* \ =\ \mu Y.\,\big(1 + \textstyle\sum_{s:S}(P(s) \to Y)\big), \qquad P^*(\bullet) = 1, \quad P^*(\mathsf{node}(s,c)) = \textstyle\sum_{p:P(s)} P^*\big(c(p)\big).$$

[1] MacBeth has formalised this isomorphism in Lean (file `Cofunctor.lean`, built and reported zero-`sorry` in `papers/dcont-cof.tex`), but that source is *not yet committed* to this repository's `lean/` tree; so the statement is tagged [Cited] here, not Lean-verified, pending the source landing in the repo. The *identification* of Ahman–Uustalu's "relative split pre-opcleavages" with Spivak's cofunctors, and the bridge to delta lenses below, are MacBeth's connective contribution; see `papers/dcont-cof.tex`.

[2] Verified combinatorially over 36 ordered pairs of small categories drawn from $\{\mathbf{1}, \mathbf{2}, \mathbf{3}, \mathbb{Z}/2, \mathbb{Z}/3, M\}$; #DCont-morphisms = #cofunctors in all 36 cases, #op-variance-morphisms = #functors in all 36, and #functors ≠ #cofunctors in 17 of the 36. See `papers/dcont-cof.tex` §5. The underlying fact that functors are carried by bicomodules, not comonoid morphisms, is [17].

[3] The section condition is (PutGet) $f \cdot \varphi(a,u) = u$, *not* "$f \circ \varphi = \mathrm{id}$"; MacBeth flags this because the loose form recurs in the folklore.

Its unit $\eta\colon y \to C^*$ picks the leaf $\bullet$, and its multiplication $\mu\colon C^* \triangleleft C^* \to C^*$ is tree grafting *(plant a tree at each variable leaf). The three $\triangleleft$-monoid laws — $\bullet$ a two-sided unit for grafting, and grafting associative — hold.* *[Cited: Gambino–Kock 2009, Thm 4.5 (free polynomial monad) [3]; Abbott–Altenkirch–Ghani 2005 [1]. The three laws in container coordinates are MacBeth's, Lean-verified (`Free.lean`).]*[4]

What makes C^* the *free* monoid — that every $\triangleleft$-monoid receiving a map from C receives a unique monoid map from C^* — is its universal property, the *free-monad Lemma* (Theorem 6.10). That is the first result of the next chapter, where C^* reappears as the *syntax* generated by C read as a signature. We have stated the object; there we prove it is universal.

[4]The construction S^* is a W-type (an initial algebra in **Set**), so it exists by the initial-algebra theorem, and the position family P^* is well-defined by recursion *over* that W-type — an indexed W-type. The full construction, the grafting figure, the three-law verification, and the `Free.lean` status are given at Definitions 6.6–6.7 and Theorem 6.8; the dependency of P^* on S^* is spelt out at Remark 6.5. We state the object here and build it there.

Chapter 6

Monads and comonads: the free and cofree constructions

Read a container as a *signature*. The shapes S are operation symbols; the fibre $P(s)$ is the *arity* of s, the set of argument slots the operation s has. The container $1 + X$ is the signature with one nullary symbol (nil, arity $\varnothing$) and one unary symbol (cons, arity 1); the container $1 + X^2$ is one constant and one binary operation. Now two questions, of the kind a logician and a control theorist would each ask about a signature:

- *What is the language it generates?* What are the well-formed *terms* you can build from these operations, with variables at the leaves?
- *What behaviour can it exhibit?* If a machine can, at each step, perform one of these operations and branch according to its arity, what are the possibly-infinite *execution trees* it can unfold?

The first answer is the *free monad* on the signature; the second is the *cofree comonad*. They are the syntax and the semantics of the same data — and the fact that organises this whole chapter is that **both answers are again containers**, with shape and position data you can write down, and that the second one is not merely a comonad but a *directed container*: a small category. The behaviour of a signature lands back on the equivalence chain of Chapter 5.

The promise. By the end of this chapter you will be able to compute, for any container C, the free monad C^* and the cofree comonad C^∞ as containers; you will know why C^* is the *universal* monad built from C (the free-monad Lemma); and you will see that C^∞ is the cofree *directed* container on C, whose category is the category of subtrees.

The landscape. Chapters 4 and 5 studied the monoids and comonoids that already *sit inside* **Cont**: a monoid for $\triangleleft$ is a monad, a comonoid is a directed container. This chapter runs the other way. It does not ask which containers happen to carry a (co)monad structure; it asks how to *manufacture* one — the smallest monad receiving a map from a given container C, and the largest comonad mapping to it. In categorical language these are two adjunctions, natural in the monoid M and the comonoid D,

$$\mathrm{Mon}(\mathbf{Cont})\big(C^*,\ M\big)\ \cong\ \mathbf{Cont}\big(C,\ UM\big), \qquad \mathrm{Comon}(\mathbf{Cont})\big(D,\ C^\infty\big)\ \cong\ \mathbf{Cont}\big(UD,\ C\big),$$

the free monad $C \mapsto C^*$ left adjoint to the forgetful $U\colon \mathrm{Mon}(\mathbf{Cont}) \to \mathbf{Cont}$, the cofree comonad $C \mapsto C^\infty$ right adjoint to $U\colon \mathrm{Comon}(\mathbf{Cont}) \to \mathbf{Cont}$. The free monad is built by an *initial algebra* and the cofree comonad by a *final coalgebra*, so we begin by borrowing exactly that much machinery — and no more.

6.1 The tools we borrow: initial algebras and final coalgebras

Everything below is built from two dual constructions. This section states them, names the existence theorems, and stops; the mathematics of the chapter is in the container instances, not here.[1]

Let $\mathcal{E}$ be a category and $F\colon \mathcal{E} \to \mathcal{E}$ an endofunctor. An *F-algebra* is an object A with a map $FA \to A$; a morphism of algebras is a map commuting with the structure. The *initial* F-algebra, when it exists, is written μF with structure map an isomorphism $F(\mu F) \xrightarrow{\sim} \mu F$ (Lambek's lemma); dually the *final* F-coalgebra νF carries an iso $\nu F \xrightarrow{\sim} F(\nu F)$. We write $\mu X.\, FX$ and $\nu X.\, FX$ when we want to name the bound variable.

Theorem 6.1 (Existence by the initial-algebra sequence)**.** *If $\mathcal{E}$ has an initial object 0 and colimits of ω-chains, and F preserves those colimits, then $\mu F = \operatorname{colim}_n F^n 0$ exists. Dually, if $\mathcal{E}$ has a terminal object 1 and limits of ω^{op}-chains and F preserves them, then $\nu F = \lim_n F^n 1$ exists. More generally it is enough that F be* accessible *— that it preserve κ-filtered colimits for some regular cardinal κ — in which case the sequence is run out to length κ. [Cited: Adámek 1974 (initial-algebra sequence); Adámek–Rosický (accessible functors). Classical.]*

The two shapes we need are the ones that turn a plain endofunctor into a monad and a comonad.

Definition 6.2 (Free monad and cofree comonad of an endofunctor)**.** For $F\colon \mathcal{E} \to \mathcal{E}$ the *free monad* on F is the endofunctor

$$T_F X \;=\; \mu Y.\, (X + FY),$$

the initial algebra of $Y \mapsto X + FY$ with X held fixed; its unit is the injection $X \hookrightarrow T_F X$ and its multiplication is substitution at the variable leaves. Dually the *cofree comonad* on F is

$$D_F X \;=\; \nu Y.\, (X \times FY),$$

the final coalgebra of $Y \mapsto X \times FY$; its counit reads off the X-label at the root and its comultiplication relabels each node by the subtree hanging from it. [Cited: Barr 1970 (free monad on an endofunctor); the cofree comonad is dual. Standard.]

That is the whole toolkit. The rest of the chapter is the single observation that when F is a container extension, both fixed points are computed *inside* **Cont** — and computed concretely enough to program.

6.2 W-types and their duals

The initial and final algebras we need are of the special functors that a container itself determines, and these have a name.

Definition 6.3 (W-type and M-type)**.** For a container $S \triangleleft P$ the *W-type* is the initial algebra $W\,S\,P = \mu Y.\, [\![S \triangleleft P]\!]Y = \mu Y.\, \sum_{s:S} (P(s) \to Y)$: the set of *well-founded trees* whose nodes are labelled by shapes s and which branch over $P(s)$. The *M-type* is the final coalgebra $M\,S\,P = \nu Y.\, \sum_{s:S} (P(s) \to Y)$: the same trees, now allowed to be infinitely deep.

Proposition 6.4 (W- and M-types exist in **Set**)**.** *For every container $S \triangleleft P$ the types $W\,S\,P$ and $M\,S\,P$ exist in* **Set**. *[Cited: Abbott–Altenkirch–Ghani 2005 (containers and W-types); Adámek (Theorem 6.1).]*

[1] Neil's steer for this book is that the general fixed-point machinery belongs in a Preliminaries chapter, with this chapter kept to the container instances. The book has no Preliminaries chapter *yet*; until the global structure is fixed, this section carries the borrowed tools in tight, cited form, and can be lifted out wholesale when the Preliminaries chapter exists.

The two halves of this proposition are true for different reasons, and the asymmetry is worth a moment because it is a payoff of Chapter 1.

For the W-type, the point is that $[\![S \triangleleft P]\!] = \sum_s (P(s) \to -)$ is *accessible*: each representable $(P(s) \to -)$ preserves κ-filtered colimits once κ exceeds the cardinality of $P(s)$, and a coproduct of accessible functors is accessible. So the initial-algebra sequence converges (Theorem 6.1) and $W\,S\,P$ is its colimit — the finite-depth trees, assembled by depth.

For the M-type we could invoke the dual of accessibility, but **Cont** hands us something sharper. The final-coalgebra sequence $1 \leftarrow F1 \leftarrow F^2 1 \leftarrow \cdots$ is a diagram of *connected* shape, and Chapter 1 proved that a container extension *preserves connected limits* (Theorem 3.4). So $[\![S \triangleleft P]\!]$ preserves the limit that defines ν, the sequence converges on the nose, and $M\,S\,P = \lim_n F^n 1$ is the set of possibly-infinite trees.

Why the cofree side is the *easy* side here

It is a small surprise worth flagging. Coinductive constructions are usually the delicate ones — infinite objects, no induction principle, bisimulation instead of equality. But for containers the *final* coalgebra falls out of a theorem we already own: connected-limit preservation (Theorem 3.4). The final-coalgebra sequence is connected; container extensions preserve connected limits; done. The polynomial boundary of Chapter 1 is not a technical sidebar — it is exactly what makes the behaviour construction go through.

One more remark is needed before the constructions, because the free monad's positions are themselves defined by a fixed point that lives *over* the tree of shapes.

Remark 6.5 (Positions are an indexed W-type). In the free-monad container below, the shapes S^* form a W-type in **Set**, and the positions P^* are then defined by recursion *on that W-type* — an initial algebra in the slice $\mathbf{Set}^{S^*}$ (equivalently, an indexed, or dependent, W-type). Such indexed W-types exist whenever the base W-type does; this is what makes P^* well-defined as a function of the tree, and not merely a heuristic "count the leaves". [Cited: Gambino–Hyland 2004 (dependent polynomial functors, indexed W-types).]

6.3 The free monad of a container

We can now write the free monad down. It is the extension of the free $\triangleleft$-monoid C^* stated at Proposition 5.11; here we build that object explicitly — shapes, positions, and grafting — and in the next section prove it universal. Fix a container $C = S \triangleleft P$.

Definition 6.6 (Free monad container)**.** $C^* = S^* \triangleleft P^*$ where the shapes are the well-founded C-trees *with a variable-leaf constructor*,

$$S^* \;=\; \mu Y.\,\big(1 + \textstyle\sum_{s:S}(P(s) \to Y)\big) \;=\; W\,\big(1 + S\big)\,\widehat{P}, \qquad \widehat{P}(\mathsf{inl}\,*) = \varnothing, \quad \widehat{P}(\mathsf{inr}\,s) = P(s),$$

generated by a nullary *leaf* $\bullet$ and a node $\mathsf{node}(s, c)$ for each shape s and each family $c\colon P(s) \to S^*$ of subtrees; and the positions are the *variable leaves*,

$$P^*(\bullet) = 1, \qquad P^*(\mathsf{node}(s,c)) = \textstyle\sum_{p:P(s)} P^*\big(c(p)\big).$$

Equivalently $P^*(w)$ is the set of paths from the root of w that end at a $\bullet$.

The monoid structure is tree *grafting*. Writing trees as $t ::= \mathsf{lf} \mid \mathsf{nd}\,s\,\kappa$ (leaf, or node with $\kappa\colon P(s) \to S^*$) and leaves for P^* — so that, in the signature-functor presentation $1 + \sum_s (P(s) \to -)$ of Definition 6.6, $\mathsf{lf} = \mathsf{inl}\,*$ and $\mathsf{nd}\,s\,\kappa = \mathsf{inr}(s,\kappa)$, and P^* is the indexed W-type of leaves (Remark 6.5):

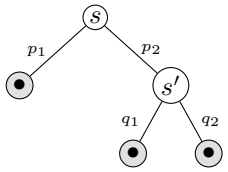

Figure 6.1: A shape of C^* for C with a binary shape s (arity $\{p_1, p_2\}$) and a binary s': a node s whose first child is a variable leaf and whose second is a node s' with two variable leaves. Its P^* has three elements — the three shaded leaves, addressed by the paths $\langle p_1\rangle$, $\langle p_2, q_1\rangle$, $\langle p_2, q_2\rangle$.

Definition 6.7 (Grafting monoid)**.** The unit $\eta\colon I \to C^*$ sends the unique shape of $I = 1 \triangleleft 1$ to lf. The multiplication $\mu\colon C^* \triangleleft C^* \to C^*$ grafts: given a tree t and a family u assigning a tree u_ℓ to each leaf ℓ of t,

$$\mathrm{graft}(\mathsf{lf}, u) = u_{()}, \qquad \mathrm{graft}(\mathsf{nd}\, s\, \kappa,\, u) = \mathsf{nd}\, s\, \big(\lambda p.\ \mathrm{graft}(\kappa_p,\, u_p)\big),$$

plants u_ℓ in place of each variable leaf ℓ. On positions $\mu^\sharp$ is the canonical bijection $\mathrm{leaves}(\mathrm{graft}(t, u)) \cong \sum_{\ell:\mathrm{leaves}\, t} \mathrm{leaves}(u_\ell)$ that cuts a leaf-path of the grafted tree at its unique t-prefix.

Theorem 6.8 (C^* is a monoid for $\triangleleft$)**.** *(C^*, η, μ) satisfies the three $\triangleleft$-monoid laws: lf is a two-sided unit for grafting and grafting is associative (with the position bijections matching on the nose). Hence $[\![C^*]\!]$ is a monad, and $[\![C^*]\!]A \cong \mu X.\,(A + [\![C]\!]X)$ is the free monad on $[\![C]\!]$: the trees with A-labelled leaves.* *[Construction cited: Gambino–Kock 2009 (free polynomial monad); Abbott–Altenkirch–Ghani 2005. The three $\triangleleft$-monoid laws in container coordinates are MacBeth's; see the footnote for their Lean status.]*[2]

Example 6.9 (Maybe and binary trees)**.** For $C = 1 + X$ (nil with $P = \varnothing$, cons with $P = 1$), a C-tree is a finite list of conses ending in either nil or a variable leaf, so $[\![C^*]\!]A$ is the free monad on Maybe: lists over A with an optional nil terminator. For $C = 1 + X^2$, C^* is the container of finite binary trees with variable leaves, and $[\![C^*]\!]A$ is binary trees with A-labelled leaves — the term algebra of one constant and one binary operation. In each case grafting is exactly substitution of terms for variables.

6.4 The free-monad Lemma

Definition 6.6 builds *a* monad from C. The content of this section is that it builds the *universal* one: every monad receiving a map from C receives a unique map from C^*. This is the free-monad Lemma, and in container coordinates its proof is a single structural induction whose base case is a monoid unit law and whose step is a monoid associativity law — the target monoid's own laws, applied, never proved.

Write $U\colon \mathrm{Mon}(\mathbf{Cont}) \to \mathbf{Cont}$ for the functor forgetting a $\triangleleft$-monoid down to its underlying container. The *insertion of generators* is the container morphism

$$\alpha = \alpha_C\colon C \longrightarrow U(C^*), \qquad \alpha_1(s) = \mathsf{nd}\, s\, (\lambda p.\, \mathsf{lf}), \quad \alpha_s^\sharp\langle p, ()\rangle = p,$$

sending a shape s to the one-node tree of shape s with all children variable leaves, and identifying that node's leaves with $P(s)$.

[2]The laws are checked with zero `sorry` in `Free.lean` (`Container.freeMonoid`), using only `Quot.sound`; the associativity law reduces to a `split/graft` compatibility (`graft_assoc`, `split_assoc`). That source is not yet committed to this repository's `lean/` tree, so the tag reads [MacBeth] rather than Lean-verified, per the Preface convention.

Theorem 6.10 (The free-monad Lemma)**.** *For every container C, $\alpha_C \colon C \to U(C^*)$ is universal from C to U: for every $\triangleleft$-monoid M and every container morphism $g \colon C \to U(M)$ there is a* unique *monoid morphism $\widehat{g} \colon C^* \to M$ with $U(\widehat{g}) \circ \alpha_C = g$. Equivalently $C \mapsto C^*$ is left adjoint to U, with unit α. Consequently, since $[\![-]\!]$ is strong monoidal and fully faithful, $[\![C^*]\!]$ is the free monad on the polynomial functor $[\![C]\!]$.* *[Cited: Gambino–Kock 2009, Thm 4.5 (free monad on a polynomial functor; single-variable case Gambino–Hyland 2004) and Spivak 2022 (arXiv:2202.00534) — two independent prior-art anchors for the adjunction. The container-coordinate proof of the universal property is MacBeth's and is graded* proved.*][3]*

The proof is short to describe even though it is long to check, and the description is the part worth carrying away. Define $\widehat{g}$ by recursion on the tree:

$$\widehat{g}_1(\mathsf{lf}) = e_M, \qquad \widehat{g}_1(\mathsf{nd}\, s\, \kappa) = \mu_M\big(g_1 s,\ \lambda q.\, \widehat{g}_1(\kappa(g_s^\sharp q))\big),$$

so a tree is evaluated by reading each node as an M-operation and *multiplying up* from the leaves. That $\widehat{g}$ is a monoid morphism is one induction: the base case $\mathrm{graft}(\mathsf{lf}, u) = u$ is M's *left-unit* law, and the inductive step, comparing $\widehat{g}$ of a graft with the product of the $\widehat{g}$'s, is M's *associativity* law. The triangle $\alpha; \widehat{g} = g$ uses M's *right-unit* law — fittingly, since α places a generator at the root with leaf children, exactly the right-unit configuration. Uniqueness is forced because the position bijection of grafting (Definition 6.7) is invertible: any morphism agreeing with g at the generators is pinned, node by node, by the multiplication law.

Where the induction really lives

Notice what is and is not proved. The laws of the *free* monoid C^* — that grafting is unital and associative — are Theorem 6.8, checked once in tree combinatorics. The free-monad *Lemma* proves something different: that the induced map into a *given* monoid M is a morphism. That proof never touches a law of C^*; it discharges each obligation using a law of M. The same tree-induction serves twice — once to establish C^*'s own laws, once to establish morphisms out of C^* — with the roles of "prove" and "apply" exchanged. This is the shape of every freeness proof, shown here in coordinates concrete enough to have been run on a machine and on paper both.

6.5 The cofree comonad, and cofree = cofree directed container

Now the dual, and the chapter's payoff. Dualising Definition 6.6 — swap the initial algebra for the final coalgebra, products for sums, leaves for nodes — gives the cofree comonad, and Proposition 6.4 already guarantees it exists.

Definition 6.11 (Cofree comonad container)**.** $C^\infty = S^\infty \triangleleft P^\infty$ where $S^\infty = \nu Y.\, \sum_{s:S}(P(s) \to Y) = M\, S\, P$ is the set of possibly-infinite C-trees, and $P^\infty(w)$ is the set of *nodes* of w, presented as finite paths from the root:

$$P^\infty(\langle s, c\rangle) \;=\; 1 + \textstyle\sum_{p:P(s)} P^\infty\big(c(p)\big)$$

(the empty path $\bullet$, plus the paths continuing into each child $c(p)$). Every path is finite even when w is infinite.

[3] MacBeth's proof is in `proofs/2026-07-24-free-monad-universal-property.md`: $\widehat{g}$ is defined by W-type recursion, and that it is a monoid morphism, that the triangle holds, and that it is unique are each proved by induction on the tree. Partial Lean: `FreeUniversal.lean` machine-checks the full triangle $\alpha; \widehat{g} = g$, the unit law, and object-level uniqueness (`Quot.sound` only); the multiplication-homomorphism law and backward uniqueness are not yet formalised. Not committed to `lean/`.

The contrast with the free monad is exact and worth stating, because it is the source of a persistent confusion. *The free monad's positions are its leaves; the cofree comonad's positions are its nodes — all* of them, the root included, not the leaves. A tree may have no leaves at all (if it is infinite) yet always has nodes, and it is the nodes that index P^∞.

Proposition 6.12 (Extension and comonad structure)**.** $[\![C^\infty]\!]A \cong \nu X.\,(A \times [\![C]\!]X)$ *is the carrier of the cofree comonad on* $[\![C]\!]$*: the possibly-infinite* C*-trees with* every node *labelled in* A*. The counit* $\varepsilon\colon C^\infty \to I$ *reads the root label; the comultiplication* $\delta\colon C^\infty \to C^\infty \triangleleft C^\infty$ *relabels each node* n *by the subtree* w/n *hanging from it, and on positions concatenates paths,* $\delta^\sharp(n, m) = n \cdot m$*.* *[Cited: Niu–Spivak 2023,* Polynomial Functors*, Prop. 8.18 & 8.33, Thm. 8.45 (cofree comonoid on a polynomial functor); carrier via Abbott–Altenkirch–Ghani 2005 (*M*-types). Container-coordinate structure maps re-derived here.]*

Here is the punchline the chapter has been driving at. A cofree comonad is, a priori, just a comonad — a comonoid for $\triangleleft$. But Chapter 5 taught us that a comonoid for $\triangleleft$ is never *just* a comonoid: it is a *directed container*, hence a small *category*. So the cofree comonad must secretly be a category. Which one?

Theorem 6.13 (The cofree comonad is a directed container)**.** C^∞ *is a directed container (Definition 5.1) with*

root $\mathsf{o}(w) = \bullet$ *(empty path),* *sub-shape* $w \downarrow n = w/n$ *(subtree at* n*),* *shift* $n \oplus m = n \cdot m$ *(concatenation*

Laws D1–D5 *hold; the only non-formal one is* $w/(n \cdot m) = (w/n)/m$ *behind* D4*. By Theorem 5.3 this directed container* is *the comonad of Proposition 6.12: the cofree comonad of a container is the cofree* directed *container on it. Its category (Theorem 5.4) is the* subtree category*: objects are trees, a morphism* $w \to w'$ *is a node* n *of* w *with* $w/n = w'$*, identities are roots (empty paths), and composition is path concatenation.* *[Cited: Niu–Spivak 2023, Thm. 8.45 (cofree comonoid = tree comonoid); Ahman–Chapman–Uustalu 2014 (polynomial comonad = category). The explicit* $\mathsf{o}/\downarrow/\oplus$ *data in D1–D5 form, and their verification, are MacBeth's.]*[4]

The identification is concrete enough to read as an algorithm. To compose two subtree-morphisms — a node n of w landing at w/n, then a node m of w/n landing at $(w/n)/m$ — you concatenate the addresses, $n \cdot m$, and land at $w/(n \cdot m)$; law D4 is the assertion that this equals $(w/n)/m$, i.e. that addressing a subtree of a subtree is addressing along the joined path. That is the whole content, and it is why the category is exactly the poset-like category of "w' is a subtree of w, witnessed by a specific occurrence".

Example 6.14 (Colists and infinite binary trees)**.** For $C = 1 + X$ the shapes $S^\infty = \nu Y.\,1 + Y = \bar{\mathbb{N}}$ are the conaturals, and $[\![C^\infty]\!]A$ is the non-empty colists over A — the classical head/tails comonad, whose subtree category is $(\bar{\mathbb{N}}, \leq)$ read as "drop a finite prefix". For $C = 1 + X^2$, C^∞ is the possibly-infinite A-labelled binary trees, δ relabelling each node by its subtree, and the subtree category has a morphism $w \to w'$ for every occurrence of w' as a subtree of w.

6.5.1 The cofree-comonad Lemma

Definition 6.11 builds *a* comonad from C; as with the free monad, it builds the *universal* one — now on the co-side. Every comonad mapping *to* C factors uniquely through C^∞. This is the exact dual of the free-monad Lemma, and its proof is the exact dual induction: where the free proof discharged its obligations using the laws of the target *monoid*, this one discharges them using the laws D1–D5 of the source *directed container*.

[4] A Lean formalisation of C^∞ as a directed container is *in progress* but blocked: the cofree carrier is an M-type (coinductive), and the repository's Lean core (v4.30.0, no Mathlib) has no coinduction. Adding Mathlib's `PFunctor.M` is the intended route; until then no Lean-verified tag is claimed for the cofree side.

Write $U\colon \mathrm{Comon}(\mathbf{Cont}) \to \mathbf{Cont}$ for the functor forgetting a comonoid — equivalently a directed container, equivalently a small category — to its underlying container. The *read-the-root* morphism is

$$\pi = \pi_C\colon U(C^\infty) \longrightarrow C, \qquad \pi_1(w) = \mathrm{root}(w), \qquad \pi_w^\sharp(p) = \langle p, \bullet\rangle,$$

sending a tree to its root shape and, on positions, sending $p \in P(\mathrm{root}\, w)$ to the depth-one path "step to child p, stop at its root". This π is the counit of the intended adjunction, and it is *not* the comonoid counit $\varepsilon\colon C^\infty \to I$ of Proposition 6.12 (which reads the root *label* into I); π reads the root *shape* out into C.

Theorem 6.15 (The cofree-comonad Lemma)**.** *For every container C, $\pi_C\colon U(C^\infty) \to C$ is couniversal from U to C: for every comonoid D (a directed container) and every container morphism $g\colon U(D) \to C$ there is a* unique *comonoid morphism $\widehat{g}\colon D \to C^\infty$ with $\pi_C \circ U(\widehat{g}) = g$. Equivalently $C \mapsto C^\infty$ is right adjoint to U, with counit π. Consequently, since $[\![-]\!]$ is strong monoidal and fully faithful, $[\![C^\infty]\!]$ is the cofree comonad on $[\![C]\!]$.* *[Cited: Niu–Spivak 2023, Prop. 8.18 & 8.33, Thm. 8.45, and Spivak 2022 (arXiv:2202.00534), Eq. (244)–(249) (the adjunction $U \dashv \mathfrak{c}$ and its limit tower) — two independent prior-art anchors. The container-coordinate proof of the couniversal property is MacBeth's and is graded* proved.*]*[5]

The proof is again short to describe. Fix a directed container $D = T \triangleleft Q$ with root o, sub-shape $\downarrow$ and shift $\oplus$, and a morphism $g\colon U(D) \to C$ (so $g_1\colon T \to S$ and, backward, $g_\tau^\sharp\colon P(g_1\tau) \to Q(\tau)$). Write $\tau_i := \tau \downarrow g_\tau^\sharp(i)$ for the sub-shape of τ named by the i-th position of its root image. Define $\widehat{g}$ in two layers:

$$\underbrace{\mathrm{root}(\widehat{g}_1\tau) = g_1\tau, \quad \mathrm{child}(\widehat{g}_1\tau, i) = \widehat{g}_1(\tau_i)}_{\text{forward: corecursion on the tree}} \qquad \underbrace{\widehat{g}_\tau^\sharp(\bullet) = \mathsf{o}_\tau, \quad \widehat{g}_\tau^\sharp\langle i, w\rangle = g_\tau^\sharp(i) \oplus \widehat{g}_{\tau_i}^\sharp(w)}_{\text{backward: recursion on the finite path}}.$$

The forward layer *unfolds* D's state τ into the tree that reads g_1 at every reachable sub-shape; the backward layer *folds* a path back down into a single D-position by iterated shift. That $\widehat{g}$ is a comonoid morphism is one induction on the path: the comultiplication law's base case is D's unit laws (D1 forward, D2 backward) and its inductive step is D's nesting and associativity laws (D4 forward, D5 backward); the counit law is the base clause $\widehat{g}_\tau^\sharp(\bullet) = \mathsf{o}_\tau$. The triangle $\pi \circ U(\widehat{g}) = g$ turns on D3 — fittingly the dual of the free triangle's use of the target monoid's *right*-unit law, since $\pi^\sharp$ picks the depth-one path $\langle i, \bullet\rangle$ and $g_\tau^\sharp(i) \oplus \mathsf{o}_{\tau_i} = g_\tau^\sharp(i)$ is exactly D3. Uniqueness has two halves: the forward map is forced by *finality* of the M-type S^∞ (any comonoid morphism over g is a coalgebra morphism into the final coalgebra, hence $= \widehat{g}_1$), and the backward map is then forced, path by path, by the comultiplication law.

The coinduction is confined to the shapes

The free monad's two layers were both finite *induction*: shapes are W-type trees (finite), positions are leaves (finite). The cofree comonad breaks the symmetry in exactly one place. Its shapes are M-type trees, possibly infinite, so the shape layer is genuine *coinduction* — and it is used precisely twice, both times as finality of S^∞: once to *define* $\widehat{g}_1$ (an anamorphism) and once to prove it *unique*. But its positions are *nodes presented as finite paths*, so the entire backward/position layer — the transport-heavy half where the directed-container laws D2, D5 do their work — stays ordinary finite *induction on the path*. This is

[5] MacBeth's proof is in `proofs/2026-07-25-cofree-comonad-universal-property.md`, the coordinate dual of the free-monad proof. *Lean status: blocked.* The forward map $\widehat{g}_1$ is defined by corecursion into the M-type S^∞, and the repository's Lean core (v4.30.0, no Mathlib) has no coinduction; adding Mathlib's `PFunctor.M` is the intended route (an infrastructure decision pending). The *backward/position* layer — finite path recursion — is portable now and is the natural partial target dual to `FreeUniversal.lean`.

the payoff of getting the position count right (Definition 6.11): had positions been leaves, they would not even be closed under the path-prefix operation the backward induction runs on. Positions-are-nodes is what lets the dual proof reduce cleanly to D's five laws.

Every step is the coordinate dual of the free-monad Lemma; the dictionary is worth tabulating, because it makes the free/cofree duality of the whole chapter completely explicit.

	free monad C^* (target monoid M)	**cofree comonad C^∞ (source d.c. D)**
carrier shapes	$S^* = W$-type (finite trees)	$S^\infty = M$-type (infinite trees)
carrier positions	leaves (finite)	*nodes* = finite rooted paths
induced map forward	$\widehat{g_1}$ by W-recursion	$\widehat{g_1}$ by M-corecursion
induced map backward	$\widehat{g}^\sharp$ via $\mu^\sharp$ split	$\widehat{g}^\sharp$ via $\oplus$ (path rec.)
morphism law fwd	M-left-unit (base), M-assoc (step)	**D1** (base), **D4** (step)
morphism law bwd	M-left-unit / assoc backward	**D2** (base), **D5** (step)
triangle	M-right-unit	**D3**
uniqueness forward	tree *induction*	tree *coinduction* (finality)
uniqueness backward	path ind. + split invertible	path ind. + comult law

The single asymmetry — induction becomes coinduction only in the *shape* row — is precisely the free / cofree = W-type / M-type, leaves / nodes duality, and nothing else in the two proofs differs.

6.6 Monads on the base, comonads on Cont: the transfer

The free monad and the cofree comonad manufacture a $\triangleleft$-(co)monad from a container. This section runs a different machine, one that takes a monad M living *one level down*, on the base **Set**, and transfers it to a comonad living *up* on **Cont**. It is short, it is exact, and it turns on the one structural fact this book keeps returning to: *positions are contravariant.*

Let $M = (M, \eta, \mu)$ be any monad on **Set** — Maybe, a writer monad, a free monad, anything. Postcompose it into the position fibres of a container.

Definition 6.16 (The transferred functor)**.** For a monad M on **Set** define $G\colon \mathbf{Cont} \to \mathbf{Cont}$ by

$$GS \triangleleft P = S \triangleleft M \circ P, \qquad G(u, f^\sharp) = (u, \{M(f^\sharp_s)\}_s),$$

i.e. replace each position set $P(s)$ by $M(P(s))$ and each backward map $f^\sharp_s$ by $M(f^\sharp_s)$. Equip G with a *counit* $\varepsilon\colon G \Rightarrow \mathrm{Id}$ and a *comultiplication* $\delta\colon G \Rightarrow G^2$, both the identity on shapes and, backward at each shape s,

$$\varepsilon^\sharp_s = \eta_{P(s)}\colon P(s) \to MP(s), \qquad \delta^\sharp_s = \mu_{P(s)}\colon MMP(s) \to MP(s).$$

The variances line up exactly. A container morphism runs its position map *backward*, so a *unit* $\eta_A\colon A \to MA$ of M points the right way to be the *backward* part of a counit ε, and $\mu_A\colon MMA \to MA$ the right way to be the backward part of a comultiplication. That is the whole idea, and it forces the theorem.

Proposition 6.17 (The monad→comonad transfer)**.** *(G, ε, δ) is a comonad on* **Cont**. *Moreover each comonad law is* exactly *one monad law of M, read backward through the position-contravariance:*

$$\textit{counit-left} \Leftrightarrow M \textit{ right-unit}, \quad \textit{counit-right} \Leftrightarrow M \textit{ left-unit}, \quad \textit{coassociativity} \Leftrightarrow M \textit{ associativity},$$

and the correspondence is a biconditional: G is a comonad iff M is a monad. Dually, a comonad W *on* **Set** *transfers to a* monad $HS \triangleleft P = S \triangleleft W \circ P$ *on* **Cont**, *with η^H backward $= \varepsilon^W$ and μ^H backward $= \delta^W$.* *[MacBeth, proved (`proofs/2026-07-25-monad-comonad-transfer.md`); Lean-verified, `MonadComonadTransfer.lean` (`Quot.sound` only). Construction = fibred-category folklore, specialised to* **Cont** $\to$ **Set**; *distinguished from Ahman–Uustalu below.]*

Proof sketch. Everything is identity on shapes, so every equation localises to a family of backward maps in **Set**, one per shape s; write $A = P(s)$. Functoriality of G is functoriality of M ($M(f \circ g) = Mf \circ Mg$); naturality of ε, δ is naturality of η, μ. For the three laws, recall that container morphisms compose backward maps in the opposite order, so a composite's backward map at s is the composite of the pieces read right-to-left. Then: counit-left is $\mu_A \circ \eta_{MA} = \mathrm{id}$, which is M's right-unit law; counit-right is $\mu_A \circ M\eta_A = \mathrm{id}$, M's left-unit law; coassociativity is $\mu_A \circ M\mu_A = \mu_A \circ \mu_{MA}$, M's associativity. The converse uses the single-shape container $(1, A)$, on which the three comonad laws *are* M's three laws at the set A; letting A range recovers all of M's laws. The dual H is the same computation with $(\eta, \mu) \rightsquigarrow (\varepsilon^W, \delta^W)$. The Lean file discharges each of the three comonad laws by `ext'` to the corresponding monad-law field, with no transport. □

Why it is a duality, in one line

Send **Cont** to its base by $(S, P) \mapsto S$. This is a (bi)fibration whose fibre over S is the category of position families with *vertical* (backward) maps — that is, $(\mathbf{Set}^{\mathrm{op}})^S$. "Positions are contravariant" *is* the $(-)^{\mathrm{op}}$. A monad M on **Set** is a *comonad* M^{op} on $\mathbf{Set}^{\mathrm{op}}$; postcomposing it fibrewise gives a fibred comonad, hence a comonad on the total category **Cont**. The single construction $M \mapsto (M^{\mathrm{op}})_*$ produces G from a monad and H from a comonad. The coordinate proof above is this mechanism written out; the fibrational viewpoint on **Cont** is von Glehn's.

There is also a clean *universal* description of G, which is where Neil first located it: G is a left co-closure of the composition product.

Remark 6.18 (G is the $\triangleleft$-left-coclosure with M in the numerator)*.* The composition product is left co-closed (Niu–Spivak, *Polynomial Functors*, Prop. 6.57, after Meyers): there is $\{q/p\} = \sum_{i \in p(1)} y^{q(p[i])}$ with $\mathbf{Poly}(p,\ r \triangleleft q) \cong \mathbf{Poly}(\{q/p\},\ r)$. Taking numerator the monad M and denominator $p = (S, P)$ gives $\{M/(S, P)\} = \sum_s y^{M(Ps)} = G(S, P)$. For *any* endofunctor M one has, by Yoneda,

$$\mathbf{Poly}\big(Gp,\ r\big) \ \cong\ [\mathbf{Set}, \mathbf{Set}]\big([\![p]\!],\ r \circ M\big) \qquad \text{naturally in } r,$$

so $G(S, P) = \{M/(S, P)\} = \mathrm{Lan}_{(S,P)} M$ is a left Kan extension (Trimble's observation, Niu–Spivak Ex. 6.63). The comonad data is the coclosure applied to M's structure maps: $\varepsilon = \{\eta/p\}$ and $\delta = \{\mu/p\}$. On **Poly** the transferred comonad $\hat{G}$ *applies M to the fibres of the direction bundle*: $[\![G(S, P)]\!](A) = \sum_s (MPs \to A)$, with counit re-exponentiating each fibre along η and comultiplication along μ. [Cited: Niu–Spivak 2023, Prop. 6.57, Ex. 6.63 [13]. Identification and universal property MacBeth's, proved.]

Remark 6.19 (Where the transfer sits in the literature)*.* The nearest neighbour is Ahman–Bauer's *Comodule representations of second-order functionals* [10]. They work in this very category **Cont**, with the same covariant extension $[\![A \triangleleft P]\!]X = \sum_a (Pa \to X)$ and — crucially

— the same *contravariant* cointerpretation $\prod_a(Pa \times X)$, which they attribute to Ahman–Uustalu's update monads. That cointerpretation *is* the fibrewise $(-)^{\mathrm{op}}$ of the teachbox above: the framing and the machinery are prior art, and we claim neither. Two of their results sit close enough to be mistaken for the transfer, and are not it.

- Their monad $\leftrightarrow$ comonad correspondence (Prop. 4.1–4.2) is the *trivial* opposite-category duality — a monad on $\mathcal{C}$ is a comonad on $\mathcal{C}^{\mathrm{op}}$, "just a matter of taking opposites." It relabels one gadget; it does not *apply* a base monad to a container at all.
- Their construction of monads *on* **Cont** (Thm. 6.3) sends $A \triangleleft P$ to $MA \triangleleft P^\star$: the base monad M acts on the *shapes*, the positions are merely extended, and — because, in their words, "the shape part is independent of the position part" — the result is a *monad*.

The transfer is the mirror image of the second. It applies M to the *positions*, leaves the shapes fixed, and — because positions are contravariant — is *forced* to be a comonad, namely $GS \triangleleft P = S \triangleleft M \circ P = \mathrm{Lan}_{S \triangleleft P} M$ (Remark 6.18). Neither the object $S \triangleleft M \circ P$ nor the direction "a base monad on the positions gives a comonad" appears in Ahman–Bauer. That shapes-versus-positions, monad-versus-comonad dichotomy — sharpened in the next teachbox — together with the coclosure identity, is what is new here; the coordinate proof turning each comonad law into the correspondingly-named monad law is its substance.

Two further pointers, neither a scoop. The Topos-Institute account of free PLTL algebras [11] builds a linking map $\lambda\colon M\mathsf{P} \Rightarrow \mathsf{P}I^{\mathrm{op}}$ from a modal monad M acting on *predicates* (through a hyperdoctrine P) to an *already-given* comonad I on the base; its slogan — that the linking obstructs once the interface branches, degree ≥ 2 — rhymes with the transfer's variance story, but the underlying mathematics is disjoint, since there M never touches positions. Older still, and running the opposite way, Ahman–Uustalu's *update monads* [7] *extract* a base monad *from* a directed container, while Purdy–Damato's distributive laws compose two *given* monadic containers horizontally; Hinze's transport of a (co)monad across an adjunction is a related mechanism one level of abstraction up, with no containers in sight. [Lead cite: Ahman–Bauer 2024 [10] (deep-read; Prop. 4.1–4.2 and Thm. 6.3 distinguished). Also: Topos-Institute PLTL blog [11] (deep-read; parallel slogan, disjoint mathematics); Ahman–Uustalu 2016 [7] (update monads, opposite direction); Purdy–Damato 2025 and Hinze WG2.8 distinguished in prose (not repository sources). Contribution — positions→comonad, the coclosure identity, and the coordinate proof — MacBeth's, proved and Lean-verified.]

Two ways to feed a Set-monad into a container

A monad M on **Set** can enter a container $S \triangleleft P$ along *either* of its two axes, and the axis you choose fixes, with no further freedom, whether you get a monad or a comonad.

- **Along the shapes:** $S \triangleleft P \mapsto MS \triangleleft P^\star$ applies M to the shape set and extends the positions to match (Ahman–Bauer, Thm. 6.3). Shapes are *covariant*, so this is a *monad* on **Cont**.
- **Along the positions:** $S \triangleleft P \mapsto S \triangleleft M \circ P$ applies M to each position fibre and leaves the shapes alone (the transfer). Positions are *contravariant*, so this is a *comonad* on **Cont**.

There is no third axis, and no choice in the outcome: the shape/position axis and the monad/comonad axis are welded together by the fibrewise $(-)^{\mathrm{op}}$ that makes a container a container. Apply M where the variance runs forward and monads stay monads; apply it where the variance runs backward and monads become comonads.

Example 6.20 ($M = \mathsf{Maybe}$)**.** For $M = \mathsf{Maybe} = (1 + -)$ the comonad G adjoins a fresh *basepoint* to every position fibre: $GS \triangleleft P = S \triangleleft 1 + P$. The counit forgets the basepoint (backward η includes $P(s) \hookrightarrow 1 + P(s)$) and the comultiplication merges the two basepoints of $1 + 1 + P(s)$ (backward μ). On extensions, $[\![G(S,P)]\!](A) = \sum_s A^{1+Ps}$: an operation of arity $P(s)$ becomes one of arity $P(s)$ *plus one distinguished slot* — a default, or exception, argument. This is the smallest non-trivial instance, and it is exactly the kind of "add an ambient slot to

every operation" move an orchestration layer performs on a signature.

6.6.1 The two feeds entwine

The teachbox left two liftings of one monad M standing side by side: the shape-monad $TS \triangleleft P = MS \triangleleft P^\star$ of Ahman–Bauer, and the position-comonad $GS \triangleleft P = S \triangleleft M \circ P$ of the transfer. They are built from the same M; they act on the same category **Cont**; one is a monad and one is a comonad. A reader who has met a distributive law will want to know whether they *interact* — whether the monad passes through the comonad coherently. They do, and the way they do is the cleanest possible: it is M's own failure to be a strong monoidal functor for products, read on positions. This subsection states the interaction, identifies the law, and — the part worth slowing down for — shows that the interaction runs in *exactly one* direction, obstructed by branching.

Fix, for this subsection, the $\prod$-cointerpretation of Ahman–Bauer (their §6.1, §6.5): writing an element $m \in MS$ through its support of leaves $b \in \mathrm{lv}(m)$ with labels $x_b \in S$,

$$P^\star(m) \;=\; \prod_{b \in \mathrm{lv}(m)} P(x_b).$$

In Neil's language $P^\star$ is a *predicate lifting*: it lifts the position family $P\colon S \to \mathbf{Set}$ along M to a family $P^\star\colon MS \to \mathbf{Set}$, and the product formula is the *universal* (that is, $\prod$-based) such lifting. The monad unit η^T is backward the Mendler projection i out of the singleton product $P^\star(\eta_S s) = \prod_{\{*\}} Ps = Ps$; the multiplication μ^T is backward the Mendler restriction j that reindexes a product along the leaf-map of μ^M. The flagship instances of this lifting are $M = \mathsf{Pf}$ (finite powerset), $M = \mathsf{Maybe}$, $M = \mathsf{Writer}$, and $M = \mathrm{Id}$; the reader monad A^K is *not* one, which is why the lifting must be weak.

The two composites. Both TG and GT are the identity on shapes over MS, so it suffices to read their positions at a fixed shape $m \in MS$. Localise at m, write $Z_b := P(x_b)$ for the position set at each leaf, and compute:

functor	positions at m
GT	$M\big(P^\star m\big) = M\Big(\prod_b Z_b\Big)$
TG	$(M \circ P)^\star(m) = \prod_b M(Z_b)$

(GT: run T, then fatten its positions with M. TG: fatten first, then take the product-lifting of the M-fattened positions.) Between $M\big(\prod_b Z_b\big)$ and $\prod_b M(Z_b)$ there is exactly one canonical map, the *oplax product-comparison*

$$\mathrm{str}_{(Z_b)} \;:\; M\Big(\prod_b Z_b\Big) \longrightarrow \prod_b M(Z_b), \qquad w \longmapsto \big(M(\pi_b)\, w\big)_b,$$

built from M and the product projections alone. It exists for *every* functor M by the universal property of the target product; it is the structure map exhibiting M as an oplax monoidal functor for $(\mathbf{Set}, \times)$. No algebra structure on M is used to write it down.

The direction is forced. str runs GT-positions to TG-positions. Container morphisms run their position map *backward*; so str is the backward part of a morphism the other way,

$$\lambda_X \;:\; T\,G\,X \longrightarrow G\,T\,X, \qquad \text{forward } \mathrm{id}_{MS}, \quad \text{backward str},$$

that is, a 2-cell $\lambda\colon TG \Rightarrow GT$. This is the *standard* orientation of a mixed distributive law of the comonad G over the monad T (Beck; Power–Watanabe; the entwining-structure language

is Brzeziński–Majid). The naive opposite orientation $GT \Rightarrow TG$ would need the reverse map $\prod_b M(Z_b) \to M(\prod_b Z_b)$, which is not canonical — and, we will see, genuinely does not assemble into a law.

Theorem 6.21 (The two feeds entwine)**.** *Let M be a* **Set***-monad with the $\prod$-cointerpretation predicate lifting $(-)^\star$ (e.g.* Pf*,* Maybe*,* Writer*,* Id*), let T, G be its two container liftings, and let $\lambda\colon TG \Rightarrow GT$ be the* 2*-cell backward* str*. Then (T, G, λ) is an entwining structure on* **Cont***: λ is natural and satisfies the four entwining axioms. Consequently G lifts to a comonad on the category of T-algebras and T lifts to a monad on the category of G-coalgebras. [MacBeth, proved (`proofs/2026-07-27-monad-comonad-entwining.md`); axioms E1, E3, E4 by naturality, E2 machine-verified for the $\prod$-lifting examples incl. branching* Pf*. T = Ahman–Bauer Thm 6.3 [10]; G = the transfer, Prop. 6.17. Framework: Beck [29], Power–Watanabe [30], Brzeziński–Majid [31], standard.]*

Proof sketch. Everything is identity on shapes, so each axiom localises to a position-level equation in **Set**, with backward maps composing contravariantly; localise at Z_b and write $\mathrm{str} = (M\pi_b)_b$. The two unit/counit axioms and the comultiplication axiom are each one naturality square:

- **Counit-G** (ε_G against λ) is $M(\pi_b) \circ \eta_{\prod Z} = \eta_{Z_b} \circ \pi_b$, the *naturality of η at π_b*.
- **Unit-T** collapses to a single-leaf product, where $\mathrm{str} = M(\mathrm{id}) = \mathrm{id}$ and the Mendler i is the singleton projection $= \mathrm{id}$; both sides are id_{MZ}.
- **Comult-G** (δ_G against λ) is $M(\pi_b) \circ \mu_{\prod Z} = \mu_{Z_b} \circ MM(\pi_b)$, the *naturality of μ at π_b*.
- **Mult-T** is the only axiom touching the Mendler j. Both paths are built from str and the reindexing of a product along the leaf-map of μ^M; since str is componentwise M applied to a projection, it commutes with that reindexing, and the two backward maps coincide. This is the *naturality of* str *with respect to product-reindexing*, i.e. the j-naturality diagram of Ahman–Bauer Def. 6.2.

Naturality of λ in X is naturality of str and of the projections. The general symbolic index-chase for Mult-T over an arbitrary weak Mendler j is mechanical and not spelled out; it is machine-verified for the $\prod$-lifting examples, including the branching commutative case $M =$ Pf. □

What λ is, in one line

The entwining says one thing: *M oplax-preserves products, coherently with its unit and multiplication, transported onto positions.* Fibrationally (the teachbox after Proposition 6.17, made precise in §6.6.2), $G = (M^{\mathrm{op}})_*$ is vertical and T covers the base monad M; λ is the *(oplax, generally non-invertible) Beck–Chevalley mate* comparing *apply M on the base, then fatten positions* against *fatten positions, then apply M on the base.* The comparison exists one way for free — that is str — and the four axioms are exactly its coherence with the two monad structures M carries on the two feeds. It is invertible — strict Beck–Chevalley — precisely when M is a *pure writer* $A \times (-)$, which is strictly stronger than non-branching (§6.6.2); the interesting content is its *failure*.

One direction only: the branching obstruction. Here is the part a reader should carry away. The opposite orientation $GT \Rightarrow TG$ needs the *lax* comparison $\prod_b M(Z_b) \to M(\prod_b Z_b)$, which not every monad has; a commutative monad supplies the cartesian one (for Pf, $(S_b)_b \mapsto \prod_b S_b$, the set of choice-tuples). Even when it exists, it *fails to be a distributive law* the moment M branches.

Example 6.22 (Pf breaks the reverse orientation)**.** Take $M =$ Pf and the container $X = \big(\{a, b\},\ a \mapsto \{0, 1\},\ b \mapsto \{0\}\big)$. The lax $GT \Rightarrow TG$ map satisfies unit-T, counit-G, and comult-G, but fails mult-T. At the double shape $\{\{a, b\}, \{a\}\} \in \mathsf{Pf}\,\mathsf{Pf}\,S$ the two paths return, on one side, the *correlated* two-element set $\{((1, 0), (1)),\ ((0, 0), (0))\}$ — a union of products — and on the other the full four-element product of unions. The multiplication $\mu = \bigcup$ merges the leaf

a shared by $\{a, b\}$ and $\{a\}$, and the cartesian lax map cannot see that the two choices at that shared leaf must agree: *union of products* $\neq$ *product of unions*.

The obstruction is **branching, not non-commutativity**. Pf is commutative and still fails; what breaks the law is μ identifying leaves, which needs a shape with at least two positions so that $\bigcup$ can overlap. When M has arity at most one (Maybe, Writer, Id) no two leaves are ever shared, so μ can identify none, mult-T holds trivially, and *both* orientations are entwinings at once. The dichotomy is clean.

M	$TG \Rightarrow GT$ (str, oplax)	$GT \Rightarrow TG$ (lax)
arity ≤ 1 (Maybe, Writer, Id)	entwining ✓	entwining ✓ (no overlap)
genuine branching (Pf, List, ...)	entwining ✓ (**universal**)	fails mult-T ✗

So the interaction of the two feeds is real, and it lives in the standard orientation, where it holds for *all* M; the orientation one might guess first is the one that branching forbids. This is the pedagogically interesting fact: the variance that forced G to be a comonad (positions are contravariant) also forces the *single* direction in which the monad feed can pass through it.

From a failed law to an arrow calculus. A distributive law that fails looks, at first, like a dead end. It is not. The reverse comparison $\kappa\colon GT \Rightarrow TG$ has a job to do that does not require it to be a global law, and reading it through that job is where the two feeds pay off. Name it: write

$$\kappa : GT \Longrightarrow TG, \qquad \text{forward } \mathrm{id}_{MS}, \quad \text{backward the lax comparison } \textstyle\prod_b M(Z_b) \to M(\prod_b Z_b),$$

the 2-cell of the right-hand column of the dichotomy table. The morphisms it composes are the *effect–coeffect arrows*. An arrow $p \rightsquigarrow q$ is a container map that reads its input through the coeffect comonad G and writes its output through the effect monad T — exactly the shape of a computation that consults a context and returns with a side effect.

Definition 6.23 (The arrow profunctor Arr_M)**.** For a monad M with the two feeds $T = T_M$, $G = G_M$, define

$$\mathsf{Arr}_M(p, q) \;:=\; \mathbf{Cont}\big(G_M\, p,\; T_M\, q\big).$$

Because G_M and T_M are *functors* on **Cont** (the transfer, Proposition 6.17; Ahman–Bauer, Theorem 6.3), this is a *profunctor* $\mathsf{Arr}_M\colon \mathbf{Cont}^{\mathrm{op}} \times \mathbf{Cont} \to \mathbf{Set}$ — contravariant in p by precomposition with G_M, covariant in q by postcomposition with T_M. An element of $\mathsf{Arr}_M(p, q)$ is an effect–coeffect arrow $p \rightsquigarrow q$. The *intended identity* $p \rightsquigarrow p$ is $\eta^T_p \circ \varepsilon_p\colon G_M p \to p \to T_M p$, and the *intended composite* of $f\colon G_M p \to T_M q$ with $g\colon G_M q \to T_M r$ runs the coeffect out, threads the first effect past the second coeffect through κ, and multiplies the effects:

$$G_M p \xrightarrow{\delta_p} G_M G_M p \xrightarrow{G_M f} G_M T_M q \xrightarrow{\kappa_q} T_M G_M q \xrightarrow{T_M g} T_M T_M r \xrightarrow{\mu^T_r} T_M r.$$

[MacBeth. Arr_M as a profunctor is immediate from functoriality of the two feeds; the biKleisli identity and composite are the standard ones for a comonad–monad pair welded by a compatible κ.]

The profunctor is the free part. It exists, with these exact hom-sets, for *every* monad M: the underlying *data* of "an effect–coeffect arrow from p to q" is always available, because it is nothing but a container map out of $G_M p$ into $T_M q$, and G_M, T_M are always there. What is *not* free is that the composite above be associative and unital — that Arr_M be the hom-profunctor of an actual *category*, with **Cont** mapping into it identity-on-objects as its pure part (a Freyd category over $(\mathbf{Cont}, \times)$). That last step needs κ to be coherent, and κ is the very reverse comparison the dichotomy table said branching forbids. So the boundary is already drawn: the category structure on Arr_M exists precisely when M does not branch. We state the classification where it belongs, alongside the other two modes of composition, in Theorem 7.6 of §7.4; here we record only the shape of the answer.

The profunctor is free; the category is what branching costs

Every monad M gives an effect–coeffect *profunctor* $\mathsf{Arr}_M = \mathbf{Cont}(G_M -, T_M =)$ on $\mathbf{Cont}$ — no conditions, because both feeds are functors. Turning that profunctor into a *category* — giving arrows an associative composition — requires the compositor κ, and κ is a coherent law *iff* M *is non-branching* (every shape has arity ≤ 1; Theorem 7.6). Branching does not destroy the arrows; it destroys their *composition*. And the non-branching monads of this ($\prod$-liftable) class are no curiosity: they are exactly the *writer-with-exceptions* monads $MX \cong E + A \times X$ (a monoid A of logs acting on a set E of exceptions), the effect monads of logging-that-may-fail. The arrow calculus of a computation that writes a log and may abort composes; the arrow calculus of a computation that branches does not. That is the price of branching, stated as a boundary rather than a defect.

Remark 6.24 (Scope, and what is open). The theorem is stated, and proved, for the $\prod$-cointerpretation predicate lifting, because str uses the product structure of $P^\star$. Whether a canonical λ exists for a general (non-$\prod$) weak Mendler algebra is open: there is no evident map $M(P^\star) \to (MP)^\star$ without the product. Within the $\prod$-class the content is complete; the general Mendler case, and a from-scratch uniqueness proof for λ, are not claimed. What is new here is not the entwining formalism — that is Beck, Power–Watanabe, and Brzeziński–Majid — but the specific law $\lambda = \mathrm{str}$ welding one monad's two container feeds, its forced orientation, and the branching obstruction that Example 6.22 exhibits.

A nearby line of work should be distinguished from ours, lest a reader mistake it for prior art. Spivak's *Categories by Kan extension* [28] also builds structure from distributive laws of monads over comonads — but there the carrier is $(\mathbf{Set}, \triangleleft)$, the density comonad manufactures a *new base category* as the law's output (Lawvere theories, Δ^{op}), and the orientation is monad-over-comonad producing an object of $\mathbb{C}\mathrm{at}^\sharp$. Our λ instead entwines two liftings of a *fixed* base monad M on the *fixed* category $\mathbf{Cont}$, with output a comonad on T-algebras rather than a fresh category. Different carrier, orientation, and output; the overlap is vocabulary.[6] [MacBeth, proved for the $\prod$-class; general Mendler case open (registry: dead-end). Contribution over prior art: the law, its orientation, and the obstruction.]

6.6.2 The fibrational picture: why the two feeds had to be what they are

Everything in this section has turned on one phrase — *positions are contravariant* — invoked first to force G to be a comonad, then to fix the single direction of λ. It is time to name the structure that phrase describes. There is exactly one, a fibration, and once it is in view the two feeds, the entwining, and the branching obstruction stop being three separate computations and become three features of one picture. None of the fibrational machinery here is ours; it is standard, and we cite it. What is ours is only the observation of *which* standard feature each phenomenon of this section is.

The container fibration. Forget the positions of a container and keep its shapes:

$$p\colon \mathbf{Cont} \longrightarrow \mathbf{Set}, \qquad S \triangleleft P \mapsto S, \qquad (u, f^\sharp) \mapsto u.$$

The fibre over a set S — the containers with shape set exactly S and the morphisms lying over id_S — is the category of position families $P\colon S \to \mathbf{Set}$ with *vertical* (backward) maps $\{Q(s) \to P(s)\}_s$; that is, it is $(\mathbf{Set}^{\mathrm{op}})^S$. This is where "positions are contravariant" becomes literal: the fibre is a power of $\mathbf{Set}^{\mathrm{op}}$, and the fibrewise $(-)^{\mathrm{op}}$ is the entire source of every variance flip in this chapter. A morphism $(u, f^\sharp)\colon S \triangleleft P \to T \triangleleft Q$ is *p-cartesian* exactly when

[6] The distinction rests on [28]'s statement of results; a full deep-read is a standing to-do, so the comparison is drawn at the level of carrier and orientation rather than proof.

each backward map $f^\sharp_s \colon Q(u\,s) \to P(s)$ is a bijection — when it merely *re-indexes* positions without discarding or duplicating them, so that $P \cong Q \circ u$. The cartesian lift of $T \triangleleft Q$ along a shape map $u \colon S \to T$ is the reindexing

$$u^*T \triangleleft Q = S \triangleleft Q \circ u \qquad \text{(precompose the position family)},$$

and because **Set** is locally cartesian closed, u^* has adjoints on both sides, $\Sigma_u \dashv u^* \dashv \Pi_u$: p is a split bifibration. Every construction below is read against this one gadget. [Cited: von Glehn [33] (**Cont** $\cong \int_S (\mathbf{Set}/S)^{\mathrm{op}}$, so $\mathbf{Cont}(q) = \Sigma_q(q^{\mathrm{op}})$); Streicher [34] for the fibred-category language and the fibrewise-opposite. We claim none of the fibration itself.]

G is a vertical fibred comonad. Read the transfer (Proposition 6.17) through p. The comonad G is *vertical* — $p \circ G = p$, it never moves a shape — and fibrewise it is postcomposition by M^{op}, the pushforward $(M^{\mathrm{op}})_*$ along $M^{\mathrm{op}} \colon \mathbf{Set}^{\mathrm{op}} \to \mathbf{Set}^{\mathrm{op}}$. Since a monad M on **Set** *is* a comonad M^{op} on $\mathbf{Set}^{\mathrm{op}}$, pushing it into every fibre gives a *vertical fibred comonad* in Jacobs's sense (Ex. 1.7.9 [35], the vertical case: a comonad in the 2-category $\mathbf{Fib}(\mathbf{Set})$ of fibrations over a *fixed* base, so with base component id), and a fibred comonad is automatically a comonad on the total category **Cont**. That is the one-line proof of Proposition 6.17 — and it explains the odd biconditional there. Each comonad law of G is fibrewise, hence is the same-named law of the fibre comonad M^{op}; localised at the representing fibre $(1, A)$ it is the same-named *monad* law of M read through $(-)^{\mathrm{op}}$. Counit is unit, comultiplication is multiplication, coassociativity is associativity: the correspondence is the fibrewise op and nothing more. Moreover G preserves cartesian morphisms for *every* M, with no hypothesis at all — $u^*G = G\,u^*$, both sending Q to $M \circ Q \circ u$ — so G is a *cartesian* vertical fibred comonad, and its counit and comultiplication are stable under reindexing. Three of the four entwining axioms of Theorem 6.21 (the two involving ε_G, δ_G, and the unit of T) are *exactly* this reindexing-stability; they cost nothing once G is seen to be fibred.

T lies over the base monad M. The shape feed sits differently, and the difference is the whole story. T does move shapes: $p \circ T = M \circ p$ strictly, and its unit and multiplication cover η^M, μ^M. So T does not live inside one fibre — it *lies over* the base monad M, as a *monad morphism*, or a *lifting of M along p*: a monad on the total category **Cont** whose image under p is the honest monad M (Street's formal theory of monads [36]; the general apparatus for such liftings along a fibration — logical-predicate liftings, $\top\top$-liftings — is Hermida's [37] and Katsumata's [40]). This lifting is *unconditional*; it exists for every M. What is *not* unconditional is the stronger word "fibred", and it is worth saying exactly why, because the word carries the crown boundary of this chapter inside it. There are two grades of "fibred" monad, and T's home is not G's:

- A *vertical* fibred monad — a monad in $\mathbf{Fib}(\mathbf{Set})$, with base component id (Jacobs, Ex. 1.7.9 [35]). This is G's home, dualised; it is never T's, since T moves shapes.
- A fibred monad over a *nontrivial* base monad — a monad in the larger 2-category **Fib** whose base component is $M \neq \mathrm{id}$ (Hermida, Def. 5.4.1 [37]). This is the right shape for T. But Hermida's definition demands that the *total* functor be a *fibred functor*, i.e. preserve cartesian morphisms — and T_M does that exactly when M is a cartesian monad (the crown boundary, made precise in the stratification box below).

So the honest statement is: *T_M is a lifting of M for every M, and a fibred monad over M in Hermida's sense (Def. 5.4.1) precisely when M is cartesian.* The word "fibred" is not decoration; it bakes in cartesian-preservation, which is the very boundary this chapter keeps circling. What Neil calls the *predicate lifting* — the product formula $P^\star(m) = \prod_b P(x_b)$ — is this lifting in coordinates; we claim none of the apparatus, only the identification of which grade T_M occupies. And here is the asymmetry to carry away: G is cartesian for *every* M,

while T is cartesian *only* when M is. All the branching fragility of this chapter lives on the shape feed T; the position feed G is serene.

λ, honestly: a mixed distributive law, not a Beck–Chevalley square. Now the entwining law. It is tempting to christen λ a *Beck–Chevalley mate* and be done — the picture rhymes with one — but the word would be a genuine category error, and it is worth spelling out why. Beck–Chevalley, in the fibrational sense (Jacobs, Def. 1.9.4 [35]), is a statement about the *adjoints to reindexing* $\Sigma_u \dashv u^* \dashv \Pi_u$: for every pullback square in the base, the canonical mate comparing quantification with *substitution* is an isomorphism. That is not what λ is. The 2-cell $\lambda\colon TG \Rightarrow GT$ is a *mixed distributive law* — an *entwining* of the comonad G with the lifting T (Beck [29]; Power–Watanabe [30]) — which is exactly the content of Theorem 6.21. It compares M with a fibrewise product, not quantification with reindexing; conflating the two constructions is the error the first draft made.

What *is* true, and is the honest residue of the Beck–Chevalley intuition, is a statement about when λ is *invertible*. Fibrewise the predicate lifting $P^\star(m) = \prod_b P(x_b)$ is a Π-pushforward — a product Π along the map $\mathrm{lv}(m) \to 1$ collapsing the leaves of m — and λ's fibrewise component is the canonical oplax comparison asking whether applying M downstairs commutes with taking that product upstairs. This comparison always exists — it is built from str — but it is *oplax*, and *in general not invertible*. Now one must be careful, because it is tempting — and wrong — to read off a single condition that makes it iso. The comparison is invertible, a *Beck–Chevalley-style* coherence (though not Def. 1.9.4 itself), exactly when M preserves *all* those products, *including the empty one*. The empty product is the nullary shape: str there is $M(1) \to 1$, and for this to be a bijection M must fix the singleton, $M(1) \cong 1$. So strict BC forces M to be a *pure writer* $A \times (-)$ (arity constantly 1, no nullary shape) — and this is *strictly stronger* than non-branching. The exception monad $\mathsf{Maybe} = (1 + -)$ is non-branching, yet its λ is *not* invertible: at the nullary shape str is $(1 + 1) \to 1$, which is not injective. So the honest statement is not "λ is Beck–Chevalley" and not the collapse "strict BC = non-branching"; it is that strict BC sits one level *inside* non-branching, and the interesting content is the *failure* at each level — Maybe breaks strict BC without branching; Pf (Example 6.22) breaks even the reverse orientation, by branching.

The fibration stratifies the monad zoo; it does not collapse it

The first draft of this book's payoff was a slogan — "containers preserve cartesian morphisms = M non-branching = strict Beck–Chevalley" — and it is *false*: those are three *different* levels, with named monads separating each pair. What the fibration actually buys is sharper and more honest: a strict, four-step *stratification* of the ($\prod$-Mendler) monad zoo,

$$\underbrace{A \times (-)}_{\text{pure writer}} \subsetneq \underbrace{E + A \times (-)}_{\text{writer+exception}} \subsetneq \{\,\text{cartesian monads}\,\} \subsetneq \{\,\text{polynomial}/\textstyle\prod\text{-Mendler monads}\,\},$$

each inclusion *strict*, with a named splitter:

- **Pure writer** $A \times (-)$: λ invertible = *strict* Beck–Chevalley. Splitter into the next level: $\mathsf{Maybe} = 1 + (-)$ is non-branching but has a nullary shape, so its λ is not strict.
- **Writer-with-exception** $E + A \times (-)$ = the *non-branching* monads (arity ≤ 1): here, and here exactly, the reverse compositor κ exists and the effect–coeffect arrows form a *category* (Theorem 7.6). *This is the one true biconditional* — non-branching $\Leftrightarrow$ reverse κ — the arrow core of §7.4. Splitter into the next level: List (the free monoid) is a *cartesian* monad, but it *branches* (arity $\mathbb{N}$), so it is non-branching's outside.
- **Cartesian monads**: exactly the M for which T_M *preserves cartesian morphisms* (no leaf-merging in μ^M). List lives here but not below; Pf (whose $\mu = \bigcup$ merges

leaves) lives *outside* even this.

- **Polynomial / $\prod$-Mendler monads**: the ambient class on which the whole apparatus is defined.

So G_M preserves cartesian morphisms for *every* M (the vertical-fibred-comonad fact, Lean-certified); Neil's fibred intuition "containers preserve cartesian morphisms" is exactly right *on the* G_M *side*. The error was to carry it over to the T_M side and equate it with non-branching and with strict BC. The fibration's real gift is that it makes these four levels — and the three monads that separate them — *visible as one ladder*, rather than collapsing them into a single false line.

One rung further out — two fibrations, subobject and codomain. The outermost class is still not the edge, and to see past it we must name a choice that has been implicit all along: *along which fibration over* **Set** *are we lifting* M*?* There are two, and they are genuinely different fibrations.

- The **subobject fibration** $\mathrm{Sub}(\mathbf{Set}) \to \mathbf{Set}$: over a set X sit the *subobjects* $P \hookrightarrow X$, and a leaf carries a *truth value* — whether or not it lies in P.
- The **codomain / families fibration Cont** $\to$ **Set** — the container fibration p of this very section: over a shape set sit the position families, and a leaf carries an *actual set of positions* — a datum, not merely a truth value.

This is the distinction Hermida and Jacobs draw in their fibrational account of induction and coinduction [38], and — Neil's own words — it is the axis this rung is really about. It is tempting to summarise the boundary as "the subobject lift survives, the codomain lift dies." That is the right first cut but too coarse, and the honest statement, below, is a 2×2.

First, the headline is a failure. The plausible "core theorem" one would want — ***every* Set *monad lifts to a container monad via*** T_M — is **false**. It holds for cartesian M and fails for the reader monad A^K and for state, because their μ^M *drops* leaves: Reader keeps only the diagonal $mm[e][e]$, discarding every off-diagonal entry, so the Mendler restriction j, which would have to source a position at *every* inner leaf, has nothing to reindex a dropped leaf from, and no T_M-monad forms.

Now the one sentence that made this land, and it lives on a single row — fix the "for all leaves" quantifier and *flip the fibration*:

> *A "for all leaves"* proposition *only gets* easier *to satisfy when leaves are dropped, so the subobject lift* $\Box P(m) = \forall b.\, P(x_b)$ *survives; but a* data*-valued assignment to* every *leaf cannot be reindexed through a drop — there is no datum to carry at the vanished leaf — so the codomain lift* $\prod_b P(x_b) = T_M$ *dies.*

Make it fully concrete on Reader with $E = \{0, 1\}$ and the diagonal multiplication. A double element $mm \in (X^E)^E$ has four inner entries; μ^M keeps the diagonal $(0,0), (1,1)$ and *drops* the off-diagonal $(0,1), (1,0)$. The proposition "all four entries lie in P" certainly implies "the two surviving entries lie in P": $\Box$ lifts. But a rule that assigns a *position* to each of the four entries has, after the drop, no coordinate left to hand to the vanished $(0,1)$: $\prod$ has no multiplication. Same drop, opposite fates — because a truth value is only *weakened* by forgetting a conjunct, while a datum, once its leaf is gone, has nowhere to live.

But the codomain fibration does not die uniformly, and this is the correction the $\forall$-row alone hides. Line up all four canonical ways to lift a leaf-family along μ^M: the two *codomain* (data-valued) lifts $\prod_b P(x_b)$ and $\coprod_b P(x_b)$, and the two *subobject* (truth-valued) lifts $\Box P = \forall b.\, P(x_b)$ and $\Diamond P = \exists b.\, P(x_b)$. Write R for the leaf-matching relation — inner tokens against surviving leaves, matched on label. Each lift's multiplication exists exactly when

R is total in a direction fixed by a clean $\mathbb{Z}/2$ parity:

$$\text{direction} \;=\; \text{(is-a-limit)} \;\oplus\; \text{(records data, not truth)},$$

	records data (codomain)	records truth (subobject)
$\forall$ (limit)	$\prod$ $(= T_M$, Ahman–Bauer$)$: forward	$\Box$: reverse
$\exists$ (colimit)	$\coprod$ $(= T^\Sigma)$: reverse	$\Diamond$: forward

where forward-total means *dies on a drop* and reverse-total means *survives a drop*. (A brute-force check over all predicates confirms the grading with zero mismatches.) So **neither fibration survives uniformly**: the survivor in the subobject world is the $\forall$-lift $\Box$, and the survivor in the codomain world is the $\exists$-lift $\coprod$ — data at *one* chosen leaf, not at all of them. Reader and State are reverse-total (each surviving leaf *is* an inner token — the diagonal (e, e) for Reader, the threaded $(s, h\, s)$ for State) and forward-partial (they drop), so both reverse-column lifts $\Box, \coprod$ survive and both forward-column lifts $\prod, \Diamond$ die. *Merging* (as in Pf, whose $\mu = \bigcup$ invents no new labels) is total both ways, so all four survive; *dropping* kills exactly the two forward ones.

In particular, losing the $\prod$-monad does *not* cost Reader every data-recording lift. The Σ-*container* lift

$$T_M^\Sigma S \triangleleft P = MS \triangleleft P^\Sigma, \qquad P^\Sigma(m) = \coprod_{b \in \mathrm{lv}(m)} P(x_b),$$

with unit the codiagonal fold and multiplication a reindexing along the diagonal/threading section, is a genuine monad on **Cont** for Reader (every E) and State (every state set). It is the $\coprod$ partner of Ahman–Bauer's $\prod$, and it is the concrete answer to Neil's question "does Reader have *any* lifting that records positions, not just truth values?": yes — the Σ one, not the $\prod$ one. ("Records positions rather than truth" is exactly what the type theorist calls *proof-relevant*; we use Neil's fibrational wording throughout and log the synonym here, once.)

This boundary carries an independent machine-checked witness. In Orestis's Agda development of liftings to **Cont** [39], the $\forall$-over-leaves codomain lift (his `□Lift`, a type-valued $\prod$) is *proved* to yield a container monad only when the leaf-reindexing map pr is split surjective — i.e. only when μ drops nothing — with `Reader` named, on the nose, as the counterexample (`CoLift.agda`, ll. 163–184, comment l. 175); his state and nondeterminism examples route instead through the surviving $\exists/\coprod$ lift. One honest caveat: his framework is *entirely* type-valued, so his $\Box$ and $\Diamond$ (his notation — *not* the truth-valued $\Box, \Diamond$ of the table above) are our $\prod$ and $\coprod$, both sitting in the *codomain* column. He therefore witnesses the $\prod$-dies/$\coprod$-survives split *within* the codomain fibration; he has no truth-valued box, so he does *not* touch the subobject leg. Even so, it is the same boundary, reached independently and checked by a proof assistant. [MacBeth, proved. The $\prod/\Box$ boundary is `proofs/2026-08-07-proof-relevance-boundary.md`, with the companion census `proofs/2026-08-06-state-reader-ladder-census.md` (Lean-verified); the $\mathbb{Z}/2$ grading is the brute-force `fourfold.py` (all predicates, zero mismatches). The Σ-monad of Reader (every E) and State (every state set) is *proved* (`proofs/2026-08-07-sigma-monad-proved.md`) and *Lean-verified for Reader*: a reduction lemma turns each monad law into one label-preserving index identity (the backward maps are all coproduct-pushforwards Σ_α, and $\coprod$ is faithful, tested at $P \equiv 1$), discharged by the diagonal section (Reader) or the threaded section (State). The subobject-vs-codomain fibration distinction is Hermida–Jacobs [38]; the $\prod$-lift is Ahman–Bauer [10]; the $\Box$/predicate-lifting apparatus is Hermida [37] and Katsumata [40]; the independent Agda witness is [39]. *Resolved* in §6.6.3: the general implication "reverse-total $\Rightarrow$ Σ-monad" is *false* — the true criterion is that M be a $\triangleleft$-monoid (Corollary 6.27), with Bag separating the two (Theorem 6.28); reverse-totality is only the pointwise

shadow of that polynomial fact. *Still open:* the exhaustiveness of the parity dichotomy, i.e. whether every data-valued lifting of Reader/State is $\prod$, $\coprod$, or a mixture.]

Be honest about the reach. The fibration *organizes* and *characterizes*; it does not *eliminate* the one hard axiom. As noted, three of the four entwining axioms fall out for free from G being a fibred comonad. But the fourth, the multiplication axiom E2 of T, touches T's fibre multiplication j — the Mendler restriction of Ahman–Bauer's $\star$ — and that datum is *extra* structure living in the $\star$-algebra, not forced by p. Cartesianness of T does not by itself hand over E2; the general symbolic chase stays where Remark 6.24 left it. The fibration is a lens that makes the *shape* of the obstruction legible; it is not a machine that discharges it. [The four-level stratification, the refutation of the collapse slogan, and the splitters Maybe (strict BC $\subsetneq$ non-branching) and List (non-branching $\subsetneq$ cartesian) are MacBeth, proved (`proofs/2026-08-05-cartesian-preservation-nonbranching.md`). The non-branching $\Leftrightarrow$ reverse-κ biconditional is Theorem 7.6; G_M cartesian for all M is Lean-certified (`FibredTransfer.lean`). Fibrational vocabulary cited: von Glehn [33], Jacobs [35], Street [36].]

Remark 6.25 (A word of caution: comonadic because contravariant). It is tempting to read the transfer as saying something deep about *comonadic* computation — that M's effects were secretly *coeffects* all along. The fibrational picture counsels restraint, and Plotkin and Turi's standing suspicion of too-easy monad/comonad dualities is well taken here. The comonad structure of G is nothing more than the *contravariant image* of M's monad structure: unit becomes counit, multiplication becomes comultiplication, associativity becomes coassociativity, each by the single fibrewise $(-)^{\mathrm{op}}$ and by nothing else. G "looks comonadic" exactly *because* positions are contravariant — a structural fact about *where* in the fibration one applies M, not a claim that M was a comonad in disguise or that the transfer captures an operational duality of effects and contexts. The real content is elsewhere: that *both* liftings exist, from the one M, and that they *entwine*. To present the comonad side as an operational-semantics discovery would be to oversell contravariance. We take the transfer for the clean structural fact it is.

6.6.3 Which monads lift: the Σ-lifting is $M \triangleleft (-)$

The grading table of the last box did two things and left two undone. It *located* the surviving codomain lift — the $\coprod$-lift T^{Σ}_{M}, data at one chosen leaf — and it *exhibited* that lift as a genuine monad on **Cont** for Reader and State. What it did not say is *which* base monads M carry the Σ-lift as a monad, nor *why* the "canonical section" that threads the multiplication should cohere. Both were flagged open. Both are answered by a single observation, and it is not a coincidence of sections but an identity of endofunctors: *the Σ-lifting is left composition by M.* Once you see it, the entire 08-07 verification for Reader and State collapses into an instance of a theorem you already proved in Chapter 5.

Proposition 6.26 ($T^{\Sigma}_{M} = M \triangleleft (-)$)**.** *Let $M = S_M \triangleleft P_M$ be a container, carrying candidate morphisms $\eta^M : y \to M$ and $\mu^M : M \triangleleft M \to M$ (not yet assumed to satisfy any law). For every container $C = S \triangleleft P$ there is an equality of containers*

$$T^{\Sigma}_{M}\, C \;=\; M \triangleleft C,$$

natural in C; and under it the unit and multiplication of the Σ-lifting are the whiskerings $\eta^{\Sigma} = \eta^M \triangleleft (-)$ and $\mu^{\Sigma} = \mu^M \triangleleft (-)$ of M's own container-unit and container-multiplication. [MacBeth, proved (`proofs/2026-08-08-sigma-monad-is-triangle-monoid.md`, Prop. 2.1) and Lean-verified: `lean/Containers/Containers/S` where `sigmaLift_eq_seq` closes M.sigmaLift $C = M \triangleleft C$ by `rfl` (a* definitional *equality; sorry-free, axiom-free).]*

The proof is a match of shapes and positions, and it is worth doing in the open because it explains the definition of T^{Σ} rather than merely confirming it. Write an element $m \in MX$ as a

shape $s \in S_M$ together with a labelling $x\colon P_M(s) \to X$, so its leaves are exactly $\mathrm{lv}(m) = P_M(s)$ and the label at b is $x_b = x(b)$. Now unfold the composite:

$$[\![M \triangleleft C]\!]X = [\![M]\!]([\![C]\!]X) = \coprod_{s \in S_M} \Big(\coprod_{s' \in S} X^{P(s')} \Big)^{P_M(s)}.$$

Its *shapes* are $\coprod_s S^{P_M(s)} = M(S)$ — an M-structure whose leaves are labelled by shapes of C, which is exactly the shape set of $T_M^\Sigma S \triangleleft P$. Its *positions* over such a shape m are $\coprod_{b \in P_M(s)} P(x_b) = \coprod_{b \in \mathrm{lv}(m)} P(x_b)$ — pick an M-leaf, then a C-position beneath the shape sitting there — which is precisely $P^\Sigma(m)$. The unit's backward map (the codiagonal fold) is η^M read on positions; the multiplication's backward map — the "canonical section" σ of the 08-07 proof — is nothing but the backward position map of M's own multiplication μ^M. There was never a section to construct: it was μ^M all along.

This turns the mysterious coherence into a triviality and, better, into a *characterisation*.

Corollary 6.27 (Σ-monad $\Leftrightarrow$ container monad)**.** *For a container M with candidate η^M, μ^M as in Proposition 6.26, the following are equivalent.*

(1) $\big(T_M^\Sigma, \eta^\Sigma, \mu^\Sigma\big)$ *is a monad on* **Cont**.
(2) $\big(M, \eta^M, \mu^M\big)$ *is a $\triangleleft$-monoid in* $(\mathbf{Cont}, \triangleleft, y)$ — *a* container monad *(Theorem 4.18).*
(3) $\big([\![M]\!], [\![\eta^M]\!], [\![\mu^M]\!]\big)$ *is a monad on* **Set** *whose functor* and *whose unit and multiplication are polynomial.*

Under any of them the three coherence conditions of the 08-07 proof hold automatically*: they are the backward halves of the $\triangleleft$-monoid's unit and associativity laws, read on positions. [MacBeth, proved (2026-08-08-sigma-monad-is-triangle-monoid.md, Thm. 3.1); the three monad laws are Lean-verified in SigmaLift.lean as the whiskered $\triangleleft$-monoid laws (each closes by the monoid law plus definitional coherence).]*

Why it holds. Every monoid A in a monoidal category $(\mathcal{V}, \otimes, I)$ makes $A \otimes (-)$ a monad, with structure maps $m \otimes (-)$ and $e \otimes (-)$; conversely, evaluating the monad laws of $A \otimes (-)$ at the unit object I returns the monoid laws of A. Take $(\mathcal{V}, \otimes, I) = (\mathbf{Cont}, \triangleleft, y)$ and $A = M$: by Proposition 6.26 that monad *is* T_M^Σ, so (1) $\Leftrightarrow$ (2). And (2) $\Leftrightarrow$ (3) because the extension $\mathbf{Cont} \simeq \mathbf{Poly} \hookrightarrow [\mathbf{Set}, \mathbf{Set}]$ is fully faithful and strong monoidal ($\triangleleft \mapsto \circ$, $y \mapsto \mathrm{Id}$; Niu–Spivak [13]), and a fully faithful strong monoidal functor reflects and creates monoids. □

So the surviving half of the proof-relevance boundary does not live in some exotic corner: it lands *exactly* on the composition-monoid spine of Chapter 5. Directed containers are the $\triangleleft$-*comonoids* (Theorem 5.4); container monads are the $\triangleleft$-*monoids*, the dual partner; and the Σ-lift is a monad on **Cont** precisely for the latter. The running witnesses are two of the smallest such monoids. Reader $y^E = 1 \triangleleft E$ is the container monad whose $\triangleleft$-monoid structure is the *unique comonoid on the set E* — the diagonal $E \to E \times E$ — so its section σ is $e \mapsto (e, e)$: the diagonal of the 08-07 proof was μ^M's backward map. State is the store container monad, whose σ is the threading map. The entire 08-07 computation was, in retrospect, a hand-verification that Reader and State are $\triangleleft$-monoids — which Chapter 5 had already established.

The counterexample that names the real content. The open flag asked whether *reverse-total* — the pointwise existence of a label-preserving section of every μ^M — is enough on its own to make T_M^Σ a monad. Corollary 6.27 says the honest requirement is stronger: M must be a container monad, a *polynomial* fact, not a pointwise one. A single monad realises the gap.

Theorem 6.28 (Bag refutes "reverse-total $\Rightarrow$ Σ-monad")**.** *The finite-multiset monad* Bag *(with $\eta\, x = \{x\}$ and $\mu = \uplus$) is reverse-total in the strongest possible way — μ is a leaf*-bijection*, so a section exists with $\sigma = \mathrm{id}$ — yet T_{Bag}^Σ is* not a monad on **Cont***; it is not even a functor on* **Cont***, because* Bag *is not a container. [MacBeth, proved (2026-08-08-sigma-monad-is-triangle-monoid.md, §4; cardinality witness in scratch/sigma-monad-coherence/bag_not_container.py).]*

Why it holds. Bag fails to preserve the connected pullback $A \to 1 \leftarrow B$, which every container (equivalently, every polynomial functor) preserves. Concretely, at $A = B = \{0,1\}$ the two size-two multisets $\{(0,0),(1,1)\}$ and $\{(0,1),(1,0)\}$ over $A \times B$ have the *same* image under (π_A, π_B), namely $(\{0,1\},\{0,1\})$: the multiset has forgotten the pairing. (Counting by total size confirms it: $|\mathsf{Bag}(A \times B)|_2 = 10 \neq 9 = |\mathsf{Bag}\, A \times_{\mathsf{Bag}\,1} \mathsf{Bag}\, B|_2$.) So $\mathsf{Bag} \notin \mathbf{Cont}$. But $T^{\Sigma}_{\mathsf{Bag}}$ being a functor on **Cont** would supply, for each relabelling f, a natural backward leaf-map — exactly a container structure on Bag. None exists, because a relabelling that identifies two labels forces a choice of *which* symmetric token survives, and no such choice is natural. □

The discriminator is the oldest one in the book: *polynomial*, not merely *analytic*. Bag is analytic — built from symmetric powers, carrying nontrivial S_n symmetries — and it is those symmetries that block a natural leaf-assignment. Reverse-total is the pointwise, analytic shadow ("some backward map exists at each μ^M"); the container-monad condition is the coherent, polynomial fact ("the backward map is natural in the labels and unital-associative"). Lining the two up:

reverse-total : $\triangleleft$-monoid :: analytic : polynomial :: forgets provenance : tracks provenance.

Bag forgets which input each output came from; that is the precise sense in which it loses provenance, and here that loss costs it the functoriality of T^{Σ} itself.

Both legs of the codomain fibration, with clean criteria

The codomain fibration p: **Cont** $\to$ **Set** carries *two* data-valued leaf-liftings of a base monad M, and each now has a crisp lift-criterion — one on each side of the $\mathbb{Z}/2$ grading:

leaf-lifting	is a monad on **Cont** $\iff$	totality direction
$\prod$ $(= T_M$, Ahman–Bauer [10])	M **cartesian**	forward
$\coprod$ $(= T^{\Sigma}_M = M \triangleleft -)$	M a **$\triangleleft$-monoid**	reverse

The two criteria are independent axes, not nested: cartesianness is about μ^M *merging* leaves, the $\triangleleft$-monoid condition about the polynomial coherence of μ^M's backward map. List satisfies both; Reader and State are $\triangleleft$-monoids but non-cartesian, so they carry the $\coprod$-lift and not the $\prod$-lift — exactly the split the grading table predicted. This is the payoff Neil's fibrational framing was pointing at: *both* legs of the non-thin (codomain) side of the boundary, which the classical predicate-lifting literature — confined to thin, faithful fibrations (Hermida–Jacobs [38]) — never reaches, now have monad-lift criteria as clean as the cartesian one.

One caveat, at the boundary with a base monad's own join. The identity $T^{\Sigma}_M = M \triangleleft -$ and Corollary 6.27 are strict, object-level, and base-monad-free: pure $\triangleleft$-algebra. This is a *different* layer from Orestis's oplax comparison $\Lambda P \circ \mathrm{join} \subseteq \Lambda(\Lambda P)$ [39], which asks how the $\forall$-lift interacts with a *base* monad's join and is an $\subseteq$, not an equality. The pure composition law is strict here; it is once the base multiplication re-enters — the T_M-boundary of the stratification box — that it degrades to Orestis's oplax $\subseteq$.

6.6.4 Which liftings, all of them: monad liftings are comonads over the base

Corollary 6.27 said *which* base monads carry the $\coprod$-lift, and the grading table said the $\prod$-lift of Reader dies. But both are statements about *two named liftings*. A more basic question has gone unasked. Fix Reader as the base monad and let the aggregator range freely: what is the *complete* list of its data-valued monad liftings? Is every lifting a $\coprod$, a $\prod$, or — the natural guess — some leafwise blend of the two? The answer is none of those tidy shapes, and it is the

most satisfying one this book could offer. The liftings are *small categories* — the very objects Chapter 5 taught us directed containers were (Theorem 5.4), reappearing one categorical level up, and for exactly the same reason.

Recall the Reader container $R = y^E = 1 \triangleleft E$: one shape, E positions, $R(S) = S^E$. Along the shape fibration $p\colon \mathbf{Cont} \to \mathbf{Set}$ a fibred endofunctor over R cannot see the shapes at all — they are fixed to be S^E — so all its content is a single recipe that turns the positions-over-the-leaves into new positions. That recipe is an *aggregator*.

Proposition 6.29 (Reduction to an aggregator)**.** *Fibred endofunctors of* **Cont** *lying over* $R = y^E$ *correspond bijectively (up to natural iso) to functors* $L\colon \mathbf{Set}^E \to \mathbf{Set}$*, via*

$$TS \triangleleft P = \big(S^E,\ m \mapsto L(P \circ m)\big), \qquad m\colon E \to S \text{ a Reader-shape.}$$

A monad structure on such a T *lying over* (R, η, μ) *is exactly a pair of natural transformations*

$$\varepsilon_A\colon L(\langle A\rangle) \to A, \qquad \delta_D\colon L\big(b \mapsto D(b,b)\big) \longrightarrow L\big(b \mapsto L(c \mapsto D(b,c))\big),$$

($\langle A\rangle$ *the constant family at* A*;* $D\colon E \times E \to \mathbf{Set}$*) subject to three laws.* *[MacBeth, proved (`proofs/2026-08-09-reader-liftings-are-categories.md`, Prop. 2.1). The functor part uses that the reindexing arrow* (m, id) *is cartesian, so a fibred* T *preserves it; the monad part reads off* ε *from* η^T *over* η *and* δ *from* μ^T *over* μ*.]*

Now look hard at the shape of that data. The unit $\varepsilon\colon L \circ \Delta \Rightarrow \mathrm{Id}$ is a *counit*; the multiplication δ is a *comultiplication*; and the three monad laws become a counit law and coassociativity. The lifting is a *monad* on **Cont**, yet the aggregator that defines it carries a *comonad*. This is the one astonishment of the section, and it is not an accident. The multiplication $\mu^T\colon TT \to T$ is a *container morphism*, so its only interesting half is a *backward* position map — and a backward multiplication is a comultiplication. It is precisely the one-operation-two-faces phenomenon of the transfer, §6.6: the fibrewise op turns a monad multiplication upstairs into a comonad comultiplication downstairs. Monad liftings are comonads — and comonads, we already know, are categories.

Before the classification, one clean lemma explains *why* $\coprod$ was the survivor and $\prod$ was not, in a single line that recovers the cartesian boundary of the whole chapter.

Lemma 6.30 ($\prod$ needs a monoid, $\coprod$ needs nothing)**.** *Let* L *be an aggregator built from the leaves of* E*.*

(1) $\coprod$ *— coproduct over any index — carries a canonical comonad structure, hence is a monad lifting of* R*, for* every *index. It is the free comonoid on the leaves.*

(2) $\prod$ *over an index* I *carries a comonad structure, hence lifts,* iff I *carries a* monoid*.*

[MacBeth, proved (`proofs/2026-08-09-lifting-dichotomy-exhaustiveness.md`, Lemma U, both sides; the $\prod$ *side is the standard fact that* $Q \mapsto Q^I$ *is a comonad on* **Set** *iff* I *is a monoid, pulled back along the monad morphism* $\mathrm{ev}\colon R \Rightarrow \mathrm{Id}$*).]*

Reader's leaf-set E carries the *diagonal* comonoid $E \to E \times E$ — the unique comonoid on any set — but generally no monoid. So $\coprod$ lifts and full $\prod$ over E does not: a cross-leaf product would demand a multiplication $E \times E \to E$, and Reader supplies none. This is T_M-lifts-iff-M-cartesian (§6.6.2) read one categorical level down: a cartesian μ^M is a bijective, monoid-like law on positions; a non-cartesian one drops leaves, leaves no monoid, and $\prod$ has nowhere to multiply. An *auxiliary* monoid M supplies its own multiplication, so the single-leaf power $\prod_M = (-)^M$ does lift — and that single-leaf power, we are about to see, is a one-object category.

Theorem 6.31 (Reader's liftings are small categories)**.** *Assume the aggregator* $L\colon \mathbf{Set}^E \to \mathbf{Set}$ *is polynomial. Then the data-valued monad liftings of Reader* $R = y^E$ *are in bijection with* E*-indexed families of small categories* $(\mathcal{C}_v)_{v\in E}$*, under*

$$L(B) = \coprod_{v\in E} \coprod_{i\in \mathrm{ob}\,\mathcal{C}_v} B_v^{\coprod_j \mathcal{C}_v(i,j)},$$

with counit ε *the identity morphisms and comultiplication* δ *the composition.* *[MacBeth, proved (2026-08-09-reader-liftings-are-categories.md, §3, Steps A–E), computationally corroborated by two independent engines (enum.py vs catcount.py, agreeing on* #liftings = #categories *on every tested out-degree profile). The final step is the equivalence* polynomial comonads ≅ small categories *— Theorem 5.4, from Ahman–Uustalu [7] — read here one categorical level up.]*

Why it holds. By Proposition 6.29 a lifting is an aggregator L with a counit ε and coassociative comultiplication δ. Writing L in polynomial normal form $L(B) = \coprod_s \prod_v B_v^{P_L(s,v)}$, the counit forces every shape to have a *distinguished* position (a marked identity), and the left- and right-unit laws force each shape to read *one* leaf only — an off-diagonal, cross-leaf arity is annihilated by the empty diagonal (this is where $\prod$, whose only shape reads two leaves, is excluded outright: it admits no δ). What remains, per leaf v, is a polynomial comonad on **Set**: objects the shapes at v, morphisms out of a shape its positions, identities the marked positions, composition the backward map of δ. By the equivalence of Theorem 5.4 that is a small category. The correspondence is mutually inverse. □

The clean liftings decode as the degenerate categories, and the exotic one becomes visible.

aggregator L	category data	lifts?
$\prod_v B_v$ (Ahman–Bauer T_R)	one object reading *all* leaves	**No**
$\coprod_{v\in U} B_v$ (Σ_U)	*discrete* category on U	Yes
$\coprod_v W_v \cdot B_v$ (weighted)	discrete category, objects $\coprod W_v$	Yes
$B_0 \times B_0$, swap δ	one-object group $\mathbb{Z}/2$ (non-Σ, non-$\prod$)	Yes
$B_0 \times B_1$ (cross-leaf shape)	a cross-leaf object	**No**

So the survivors are categories; Σ_U are exactly the *discrete* ones; and the imagined "leafwise mix of $\prod$ and $\coprod$" was a mirage. The honest parameter is a small category per leaf: a shape may read one leaf with multiplicity ≥ 2 — these are the hom-sets of a *non*-discrete category, the products that survive because they live *within* a single leaf — but never across leaves, which is exactly the $\prod$-obstruction of Lemma 6.30.

Remark 6.32 (Polynomial is the boundary — again). The theorem assumes L polynomial. The counit alone rules out the natural non-polynomial rivals: a natural $\varepsilon\colon L(\langle - \rangle) \Rightarrow \mathrm{Id}$ must pick a *canonical* element, and a symmetric summand — an unordered pair, the symmetric square, the size-two multiset Bag_2 — has no swap-invariant canonical element, so it admits no ε and is not a lifting. It is the same discriminator as Theorem 6.28, on the same functor: *polynomial*, not merely *analytic*. (A full removal of the polynomiality hypothesis would need the lemma "every accessible **Set**-comonad with a counit is polynomial", which is not proved here; the natural rivals are excluded, the general statement is flagged open.) [MacBeth, proved (2026-08-09-reader-liftings-are-categories.md, Prop. 5.1), verified in analytic.py: Sym^2 and Bag_2 admit no natural ε, the ordered square does.]

The whole boundary lands on Cat

Stand back. Chapter 5 proved that the $\triangleleft$-comonoids — the directed containers — are the small categories (Theorem 5.4), the promised **Containers** $\supseteq$ **Directed Containers** $\cong$ **Small Categories**. This section proves that the proof-relevant monad *liftings* of Reader are *also* small categories, and by the identical mechanism: a monad lifting is secretly a comonad (its multiplication is a backward map), and comonads are categories. The two facts are the same equivalence, Theorem 5.4, read at two different heights. The directed-container spine of the book is not one theorem but a gravitational well: the objects, their morphisms (the cofunctors of **DCont** $\cong$ **Cof**), and now their liftings all fall into **Cat**. Neil's two feeds and the $\mathbb{Z}/2$ grading were the map; the destination is the category of small categories.

Reader was the base case $S_M = 1$: one shape, one aggregator, one small category per leaf. What happens at the opposite extreme — the *richest* finite store there is?

6.6.5 The State pole: the store multiplication is invisible

Take $M = \mathrm{State}$, $MX = (S \times X)^S$. Its shape set is S^S, the full transformation monoid of the state set — every possible way to overwrite S, composed under function composition. If any base monad were going to force its liftings into some elaborate graded shape, it is this one: the reduction of Proposition 6.29 now hands back not a single aggregator but a whole *family* $(A_\sigma \colon \mathbf{Set}^S \to \mathbf{Set})_{\sigma \in S^S}$, one per store-shape, and the multiplication of State *threads* them together along the monoid S^S, coupling the source states through the rule $\sigma_\mu(s) = t_s(T(s))$. Every instinct says the answer is an "S^S-graded category": a category that transforms as you push it around the store.

Here is the punchline, and it is the one this section was built to earn.

Theorem 6.33 (State's liftings are small categories)**.** *The polynomial data-valued monad liftings of* $\mathrm{State} = S^S \triangleleft S$ *are in bijection with small categories* $\mathcal{C}$*, under* $\mathcal{C} \mapsto \mathbb{S} \times \mathcal{C}$*, where* $\mathbb{S}$ *is the action category of* $S^S \curvearrowright S$*. Explicitly the aggregator is*

$$A_t(Q) \;\cong\; \coprod_{s \in S} \; \coprod_{c \in \mathrm{ob}\,\mathcal{C}} Q_s^{\coprod_{c' \in \mathrm{ob}\,\mathcal{C}} \mathcal{C}(c,c')} \qquad \textit{(independent of the grade } t \in S^S\textit{)},$$

with ε *the* $\mathcal{C}$*-identities and* δ *the source-routed* $\mathcal{C}$*-composition carrying* trivial *transport. The whole* S^S*-grading is invisible: State's liftings are the* same **Cat** *as a single-object Reader's.* *[MacBeth, proved at the object level; the morphism level by the mirror argument and exhaustive machine check at* $|S| = 2$ *(`2026-08-11-state-liftings-holonomy-triviality.md`). Soundness* **Cat** $\hookrightarrow$ *liftings,* $\mathcal{C} \mapsto \mathbb{S} \times \mathcal{C}$*, is Lean-verified (`StateProductLifting.lean`).]*

Why it holds — the store multiplication is a mirage. Two things could make State richer than Reader, and both collapse.

First, the grade could matter. A priori the aggregator A_t depends on the store-shape $t \in S^S$. It does not. The left-unit law hands each grade a comparison map $\mathrm{sh}_t \colon A_t \to A_{\mathrm{id}}$, and a factorisation whose threading returns to id hands back a map pr_t the other way; the *outermost-object component of associativity* forces these two to be mutually inverse. So $A_t \cong A_{\mathrm{id}}$ for every grade — this is *grade-independence* — and A_{id}, by the Reader theorem with $E = S$ (at grade id the threading is trivial, $s \mapsto \mathrm{id}(s) = s$, so the store does not couple the states), is an S-indexed family of small categories $(\widetilde{\mathcal{C}}_s)_{s \in S}$.

Second, the coupling could carry holonomy. The only freedom left is how δ transports the fibre $\widetilde{\mathcal{C}}_s$ along a store-arrow $s \to g(s)$. Read the *deepest* object of the associativity law. Writing $\tau^g(s, c)$ for the transport of an object $c \in \widetilde{\mathcal{C}}_s$ along g, associativity says

$$\tau^{g'}(s, c) \;=\; \tau^{t_s}\big(T(s),\, \tau^T(s, c)\big), \qquad g'(s') = t_{s'}(T(s')).$$

Look at the asymmetry. The right-hand side sees only the middle map *at the source state* s; the left-hand side depends on the whole function g', stitched from *all* the middles. The only way the two can agree for every choice of the off-s middles is if the transport cannot see them either — that is, if $\tau^g(s,c)$ depends on g *only through the endpoint* $g(s)$. This is *endpoint-locality*, and it is strictly stronger than functoriality.

Endpoint-locality is exactly the statement that the transport is a functor out of the *codiscrete* category on S — the category with a unique arrow $s \to m$ for every ordered pair. A functor out of a codiscrete category is a trivial coherent system of isomorphisms: choose a basepoint $0 \in S$, identify every fibre with $\widetilde{\mathcal{C}_0} =: \mathcal{C}$ along the unique arrows, and every transport becomes the identity. The S source-fibres fuse into one category $\mathcal{C}$, and the store transport is trivial. What is left is $A_t(Q) = \coprod_s \coprod_c Q_s^{\coprod_{c'} \mathcal{C}(c,c')}$ with threaded $\mathcal{C}$-composition — precisely $\mathbb{S} \times \mathcal{C}$. □

Remark 6.34 (Why the graded category was a mirage). The tempting rivals all satisfy functoriality but fail endpoint-locality, and endpoint-locality is the sharper blade. A copresheaf transport $F\colon \mathbb{S} \to \mathbf{Set}$ — say the representable $\mathbb{S}(0,-)$ — is a perfectly good functor, obeying $\tau^{b\circ a} = \tau^b \circ \tau^a$; yet $F(g)$ reads the whole function g, not just $g(s)$, so it fails the deepest-object identity and breaks associativity. Per-state-different fibres $\widetilde{\mathcal{C}_0} \not\cong \widetilde{\mathcal{C}_1}$ are impossible too: S^S acts transitively on S, so the codiscrete category is connected and all fibres are forced isomorphic. The finer graded object one hoped for simply does not exist; the honest answer is the *coarser* **Cat**. [MacBeth, proved/computed (`2026-08-10-state-liftings-holonomy-free.md` §5; `2026-08-11-state-liftings-holonomy-triviality.md` §5): representables satisfy the unit laws but fail associativity, and every non-trivial automorphism-twist of $\mathbb{S} \times \mathcal{C}$ fails the laws.]

One theorem, two poles: the invariant is π_0 of the threading action

Reader and State are now the same theorem, read at two settings of a single dial. Every polynomial monad lifting of a container monad is governed by the *position-threading action* — the way the multiplication carries a position to its endpoint — and specifically by its *action category* $\mathcal{A}$ and the number of connected components $\pi_0(\mathcal{A})$.

base monad	action category $\mathcal{A}$	$\pi_0(\mathcal{A})$	liftings
Reader y^E	*discrete* on E	$\lvert E\rvert$	E-indexed families of small categories
State $(S \times -)^S$	*codiscrete* on S (collapsed)	1	a single small category $\mathcal{C}$

Reader spreads its liftings across $|E|$ independent components — one category per leaf, no transport between them. State collapses to a single component — one category, and the store multiplication that looked so rich contributes *nothing* to the classification. The dial is π_0 of the transport, and the two monads sit at its extremes: maximally disconnected, and maximally connected. So the whole classification would seem to be a *count* — read off π_0 of the threading action and you are done. It is irresistible to conjecture exactly that. Resist it: the next subsection shows the count is an illusion, and the illusion is the most instructive thing in the chapter.

So π_0 organises the two poles. Does it *classify*? Here the story turns — and it is the turn the whole chapter has been walking toward.

6.6.6 The general law: holonomy, and the two poles as its degeneracies

Fair is foul, and foul is fair. The two clean answers — Reader's family of categories, State's single category — looked like the whole truth, and the π_0 dial looked like the law that governed them. Both were degeneracies. There is a wider class of container monads on which the classification is *not* a count, on which the transport carries an invariant π_0 cannot see, and on which the grading we twice declared a mirage is alive and irreducible. Reader and State are holonomy-free not because monad liftings are holonomy-free, but because each of those two monads kills the holonomy in its own particular way. Name the wider class, and both poles fall out as the two ways of being trivial.

The class that reveals it. Between "one shape" (Reader) and "the full transformation monoid" (State) sits the family that interpolates them: the *update monads* of Ahman and Uustalu [8]. Fix a set S of states and a *monoid* P of updates acting on S on the right, $\downarrow\colon S \times P \to S$, with $s{\downarrow}o = s$ and $s{\downarrow}(p \oplus q) = (s{\downarrow}p){\downarrow}q$. The update monad is the polynomial functor

$$\mathrm{Upd}_{(S,P,\downarrow)}(X) \;=\; \sum_{f \in P^S} X^S, \qquad \mu(F)(s) = \big(f(s) \oplus g_s(s{\downarrow}f(s)),\; x_s(s{\downarrow}f(s))\big),$$

its shapes the update-fields $f \in P^S$, its positions the states S. The cardinality of the position set never changes — but the multiplication *threads* a state s to its *endpoint* $s{\downarrow}f(s)$, and that is where the positions genuinely move. Reader is the case $P = 1$ (nothing to apply, $s{\downarrow}o = s$); State is the case $P =$ the overwrite monoid $\{o\} \cup \{w_m : m \in S\}$ with $s{\downarrow}w_m = m$. The threading is no longer a fixed background fact; it is the datum $\downarrow$, and it varies from one update monad to the next.

Definition 6.35 (The position-threading action and its category)**.** The *position-threading action* of $\mathrm{Upd}_{(S,P,\downarrow)}$ is the monoid action $\downarrow\colon P \curvearrowright S$ itself. Its *action category* $\mathbb{A}(\downarrow)$ (the translation category of the action) has

$$\text{objects } S, \qquad \text{an arrow } s \xrightarrow{\;p\;} s{\downarrow}p \text{ for each } p \in P, \qquad \text{composition } (s \xrightarrow{\;p\;} \xrightarrow{\;q\;}) = (s \xrightarrow{\;p\oplus q\;}), \quad \mathrm{id}_s = (s \xrightarrow{\;o\;} s).$$

Its connected components are the orbits, $\pi_0(\mathbb{A}(\downarrow)) = S/{\downarrow}$; its *isotropy monoid* at s is the stabiliser $\mathrm{Stab}(s) = \{p \in P : s{\downarrow}p = s\}$. For Reader, $\mathbb{A}(\downarrow)$ is the *discrete* category on S; for State it is the overwrite category, which we shall see collapses.

Now the reduction runs exactly as it did for the two poles — grade-independence trims the family of aggregators to one, the Reader argument makes each source-fibre a small category $\mathcal{C}_s$ — and the only freedom that remains is the transport: how the comultiplication carries an object $c \in \mathcal{C}_s$ along an update p to a new object in $\mathcal{C}_{s{\downarrow}p}$. Write it $\rho_{s,p}\colon \mathcal{C}_s \to \mathcal{C}_{s{\downarrow}p}$. The deepest object of the associativity law — the same instance that forced grade-independence and endpoint-locality before — now reads, for all $s \in S$ and $p, q \in P$,

$$\rho_{s,\,p\oplus q} \;=\; \rho_{s{\downarrow}p,\,q} \circ \rho_{s,p}, \qquad \rho_{s,o} = \mathrm{id}_{\mathcal{C}_s}.$$

Read the display again: it is *precisely* the statement that ρ is functorial on $\mathbb{A}(\downarrow)$. The transport is not extra structure bolted onto the classification — it *is* a functor out of the action category, and associativity is its functoriality.

Theorem 6.36 (Update-monad liftings are functors on the action category)**.** *The degree-*1 *proof-relevant polynomial monad liftings of* $\mathrm{Upd}_{(S,P,\downarrow)}$ *are in bijection with functors*

$$\mathbb{A}(\downarrow) \longrightarrow \mathbf{Cat}, \qquad s \mapsto \mathcal{C}_s, \quad (s \xrightarrow{\;p\;} s{\downarrow}p) \mapsto \big(\rho_{s,p}\colon \mathcal{C}_s \to \mathcal{C}_{s{\downarrow}p}\big).$$

Equivalently, with $\pi_0(\mathbb{A}(\downarrow))$-indexed families whose component at an orbit $[s]$ is a small category equipped with an action of the isotropy monoid $\mathrm{Stab}(s)$*. Reader ($\mathbb{A}$ discrete) and State ($\mathbb{A}$ collapsing to codiscrete) are the two special cases in which every isotropy action is trivial, so the datum degenerates to a π_0-indexed family of* plain *categories — an arbitrary family for Reader, a single category for State.* *[MacBeth, proved and exhaustively verified at $|S| = 2$, degree-1, the argument uniform in $(S, P, \downarrow)$ (2026-08-11-update-monad-liftings-holonomy-full.md). The update monad and the data $(S, P, \downarrow)$ are Ahman–Uustalu [8]; the $\mathbb{A}(\downarrow)$ framing, the classifier $\mathbf{Fun}(\mathbb{A}(\downarrow), \mathbf{Cat})$, and the isotropy = holonomy diagnosis are the author's. [open] at higher object-degree and beyond the update-monad class; a novelty-check against Uustalu's* container combinatorics *(TTCS 2017) is owed and deferred.]*

Why it holds. Necessity is the displayed law: instantiate $\mu \circ T\mu = \mu \circ \mu T$ on an Upd^3-element with outer update f and constant middles and inners; letting the constants range over P makes $p = f(s)$ and q range over all of P, and the two bracketings agree on the innermost object exactly when $\rho_{s,p\oplus q} = \rho_{s\downarrow p,q} \circ \rho_{s,p}$. That is functoriality of ρ on $\mathbb{A}(\downarrow)$. Sufficiency is the converse construction: given any $F\colon \mathbb{A}(\downarrow) \to \mathbf{Cat}$, the source-routed aggregator $A_f(Q) = \coprod_s \coprod_{c \in \mathrm{ob}\,\mathcal{C}_s} Q_s^{\mathrm{out}(c)}$ with ε the identities and δ the composition-with-ρ satisfies the unit and associativity laws *iff* ρ is functorial. Both directions are the content of the census engine, whose law-checks are exhaustive over the finite shape spaces at $|S| = 2$. □

Functoriality is a weak requirement, and that is the whole point: a functor out of $\mathbb{A}(\downarrow)$ may send the loops at a state — the isotropy — to genuine automorphisms of the fibre category. When it does, the lifting carries holonomy, and no count of components can see it. The smallest monoid already does this.

Example 6.37 ($\mathbb{Z}/2$ acting trivially: four liftings where π_0 predicts one)**.** Take $S = \{0, 1\}$ and $P = \mathbb{Z}/2$ acting *trivially*, $s\downarrow p = s$. Then $\mathbb{A}(\downarrow) = B\mathbb{Z}/2 \sqcup B\mathbb{Z}/2$: two components, each a one-object category whose endomorphism monoid is the *group* $\mathbb{Z}/2$, since $\mathrm{Stab}(0) = \mathrm{Stab}(1) = \mathbb{Z}/2$. Its π_0 is 2 — identical to a two-leaf Reader, whose discrete action category forces the *only* transport on each component to be the identity. Yet here the census is decisive: on a two-object fibre the unit forces $\rho_{s,o} = \mathrm{id}$, and functoriality forces the non-trivial loop $\rho_{s,1}$ to square to the identity, i.e. into the isotropy $\mathbb{Z}/2$ acting on $\mathcal{C}_s$ — leaving the identity or the swap, *independently at each of the two states*:

$$(\rho_{0,1}, \rho_{1,1}) \in \{(\mathrm{id}, \mathrm{id}),\ (\mathrm{id}, \mathrm{swap}),\ (\mathrm{swap}, \mathrm{id}),\ (\mathrm{swap}, \mathrm{swap})\}.$$

All four satisfy the monad laws on every one of the 16384 shapes checked, and an intertwiner computation shows they are *pairwise non-isomorphic* as liftings — e.g. $(\mathrm{id}, \mathrm{id})$ and $(\mathrm{swap}, \mathrm{swap})$ cannot agree, since $\varphi_s \circ \mathrm{id} = \mathrm{swap} \circ \varphi_s$ forces $\mathrm{swap} = \mathrm{id}$. So $\mathbb{Z}/2$-acting-trivially has $\pi_0 = 2$ and **four** non-isomorphic liftings: one $\mathbb{Z}/2$ of holonomy per orbit. The count π_0 said "one lifting per component"; the answer is "one *representation of the isotropy* per component," and the representation is real. [MacBeth, exhaustively verified (2026-08-11-update-monad-liftings-holonomy-full.md §3): four law-satisfying transports, pairwise non-isomorphic, over all 16384 Upd^3 shapes on the two-object fibre.]

With the counterexample in hand, the two poles resolve into a single, sharp dichotomy.

Trivial two ways: why Reader and State are holonomy-free

Holonomy-freeness — the collapse to π_0-indexed *plain* categories — holds exactly when every component of $\mathbb{A}(\downarrow)$ is holonomy-trivial, i.e. every functor out of it is constant up to canonical iso. There are two disjoint ways for a monoid to force this, and Reader and State are one each.

- **Reader: the action category is discrete.** $P = 1$, so the only arrow is the identity, the isotropy is trivial, and there are no cross-state arrows at all. There is no trans-

port *to be* nontrivial: holonomy is absent because there is nothing to transport along. $\mathbf{Fun}(S_{\text{disc}}, \mathbf{Cat})$ is an S-indexed family of categories, $\pi_0 = |S|$.

- **State: reset elements erase the label.** The overwrite monoid has, for each m, a *reset* w_m with $s{\downarrow}w_m = m$ for *all* s. In the associativity instance a reset middle forces the endpoint to m regardless of the source, degenerating the deepest-object law to *endpoint-locality*: ρ depends on the update only through its endpoint. That is exactly a functor out of the *codiscrete* category on S — a trivial coherent system of isomorphisms — which fuses all fibres into one category and trivialises the transport. Here holonomy is not absent; it is *erased*. $\pi_0 = 1$.

So the "invisible" store multiplication of §6.6.5 was not empty — it was a reset performing an erasure, and the erasure is what we mistook for absence. A generic monoid supplies neither escape. (A *third* trivialisation exists but is neither pole: let $\mathbb{Z}/2$ act *freely*, $s{\downarrow}1 = 1 - s$; then $\mathbb{A}(\downarrow)$ is codiscrete with *trivial* isotropy, giving a single lifting — holonomy-free by triviality of the stabiliser, not by erasure. Discrete, reset-collapse, and free are three roads to the same cliff; the generic monoid takes none of them.) Holonomy-freeness is the special case. The rule is holonomy.

An outside view: the same collapse in belief propagation

The collapse mechanism — *trivial holonomy of a transport around cycles forces locally-consistent data to glue into one global object* — is not special to containers. It recurs, provably and independently, in categorical belief propagation: ter Horst, Mahadevan and Zambrano [9] prove (their Thm. 6.14) that locally consistent beliefs on a factor graph glue to a single global descent datum *exactly* when the cycle-transport holonomy is trivial. There is no shared vocabulary — their setting is sheaves on factor graphs, not polynomial monad liftings — and this is emphatically *not* a container result. It is a structural resonance: the "trivial holonomy ⇒ collapse to a global section" skeleton appears to be a genuine cross-domain pattern, and State's collapse is one more instance of it. Its converse — *nontrivial* holonomy obstructs the global glue — is exactly what Example 6.37 exhibits on the container side: four local pieces that refuse to become one.

The holonomy is a group representation — and composition remembers it

Step back and name what the isotropy is doing. A functor $\mathbb{A}(\downarrow) \to \mathbf{Cat}$ restricted to the loops at a state s is an action of the stabiliser $\mathrm{Stab}(s)$ on the fibre category $\mathcal{C}_s$ — a *representation of the isotropy group* on a category. Example 6.37 is the regular $\mathbb{Z}/2$-representation, twice. This is a *second-order* compositional datum: not the behaviour of the lifting but the symmetry it carries, and it survives composition. It is the exact sibling of the class $[\omega] \in H^2$ that governs the directed / Zappa–Szép mode of composition (Chapter 7, Theorem 7.4), where reentrancy is obstructed by a cohomology class rather than a group representation. Two modes, two second-order invariants, one lesson: *composition remembers a group*. For orchestration this is the payoff. A threading agent whose update monoid has nontrivial isotropy does not merely have a family of behaviours; it carries a representation, and orchestrating it composes that representation into the whole. The two "clean" monads — Reader and State — are precisely the agents whose representation happens to be trivial. Everything in between remembers.

The section closes with two forward-glances — one example still to be filled, one line of constructions still to be built.

Higher-order trees, decoded from the free-monad formula

Read a container $C = S \triangleleft P$ as a *signature*: shapes S are operation symbols, positions $P(s)$ are the arity of s. The free monad of §6.3 then *is* the type of finite C-terms: Definition 6.6 gives $S^* = \mu Y.(1 + \sum_s (P(s) \to Y))$, the well-founded trees whose nodes are operations and whose $P(s)$ children are the operands, with variable leaves for the free generators. This is the whole content of "the free monad is syntax": C^* is exactly the abstract-syntax-tree type of the signature C, and the free-monad Lemma (Theorem 6.10) says a map out of it is fixed by its action on the one-node terms.

A *higher-order* signature is one whose operations take operations as arguments — so that an arity $P(s)$ is itself a *function type* $A \to B$ rather than a bare set, and a node's children are indexed by such a type. Feeding such a C into the same formula, C^* becomes the type of higher-order trees: nodes whose subtree-families range over an exponential. Nothing new is proved; the higher-order tree type is *read off* the free-monad formula by taking the arities to be function types. The natural such example is Ghani–Kurz's n-dimensional trees, staged to foreshadow the opetopic tower.

Signature TODO — ask Neil / browse. The *exact* Ghani–Kurz n-dimensional-tree signature $S \triangleleft P$ — which operation symbols, which function-typed arities — is not yet in this book's reading set, so the specific container is left unfilled here rather than guessed. Once it is deep-read, instantiate $C = S \triangleleft P$ above and check that C^* reproduces their tree type verbatim. Do not invent the signature before reading it.

On the horizon: from the store comonad to Workers

Neil's programme continues past the transfer with a specific comonad — the *store* comonad of a state set S — and the arrows it defines. The next section, §6.7, develops these: the *reader/store* modality $\Delta S \otimes (-)$ on **Cont**, its *Kleisli* arrows $\Delta S \otimes p \to q$ ("p-to-q transformations with access to state S"), and the *category of Workers* they assemble into — a category *graded* by $(\mathbf{Set}, \times)$, in which composing two workers multiplies their state spaces to $S \times T$. One entry on Neil's list is left open here: the *oracle/continuation*, a coalgebra $S \to [\![[p, q]]\!](S)$ into the internal-hom container $[p, q]$, read as a stateful container morphism answering p-queries with q-responses; whether these organise into a monad on **Cont** is [to be developed].

6.7 Processes with state: the store comonad and the category of Workers

The transfer produced comonads on **Cont** abstractly, from any base monad. This section works one comonad concretely — the *store* comonad of a state set — and follows it to its natural end: a category whose morphisms are *processes carrying state*, and whose composition *multiplies* the state. It is the point at which the machinery of this chapter touches the everyday object a programmer calls a stateful component, and an economist calls a process with memory. Neil's prompt for it was the reader monad: "the basic definitions of the reader monad ... but different processes have different state, so there is something more to be said." The "something more" is exactly the writeback that turns the read-only reader into the read–write store, and — pushed through composition — forces the state spaces to multiply.

6.7.1 The state object and the store comonad

Definition 6.38 (State object)**.** For a set S, the *state object* is the container $\Delta S := S \triangleleft (s \mapsto S)$: shape set S, and *every* position fibre equal to S. Its extension is $[\![\Delta S]\!]X = \sum_{s:S} X^S = S \times X^S$.

[Definition]

ΔS is the container in which each shape sees all of S as its positions. Read through the object dictionary of Chapter 5, that uniform fibre has a name.

Proposition 6.39 (ΔS is codiscrete; $[\![\Delta S]\!]$ is the store comonad)**.** *Under the correspondence between directed containers and small categories (Theorem 5.4),* ΔS *is the* codiscrete *category on* S *— object set* S*, a unique morphism for each ordered pair* (s, s') *— with directed structure*

$$\mathsf{o}(s) = s, \qquad s \downarrow p = p, \qquad p \oplus p' = p' \quad (\textit{second projection}).$$

The comonad it induces on **Cont** *is the* store *(costate) comonad*

$$[\![\Delta S]\!]X = S \times X^S, \qquad \varepsilon(s, v) = v(s), \qquad \delta(s, v) = \big(s,\ \lambda p.\,(p, v)\big),$$

with $v \colon S \to X$*.* *[MacBeth (identification); store comonad due to Uustalu–Vene [25]; Lean-verified in* `StateComonad.lean`[7]*]*

Proof sketch. The codiscrete category has $\mathcal{C}(s, s') = 1$, so $P(s) = \coprod_{s'} \mathcal{C}(s, s') = \coprod_{s'} 1 = S$, which is ΔS. Reading Theorem 5.4 off this category: the identity at s is the arrow $s \to s$, so $\mathsf{o}(s) = s$; a position $p \in P(s) = S$ names its codomain, so $s \downarrow p = \mathrm{cod}(p) = p$; the unique composite $s \to s' \to s''$ has codomain s'', so $p \oplus p' = p'$. Substituting these into the comonad induced by a directed container (Theorem 5.3), $\varepsilon(s, v) = v(\mathsf{o}(s)) = v(s)$ and $\delta(s, v) = (s, \lambda p.(s \downarrow p, \lambda p'.v(p \oplus p'))) = (s, \lambda p.(p, v))$, the store comonad. □

Two special cases pin the reader/store story. Feeding the tensor unit, $\Delta S \otimes y = \Delta S$, so the store comonad *is* the modality $\Delta S \otimes (-)$ evaluated at y. And the contravariant part of $[\![\Delta S]\!]$ — the exponential $X^S = (S \to X)$ — is precisely the *reader* functor, a monad with $\eta\, x = \lambda s.\, x$ and $\mu\, g = \lambda s.\, g\, s\, s$. Thus

$$[\![\Delta S]\!] \;=\; \underbrace{S \times}_{\text{current state}}\ \underbrace{(-)^S}_{\text{reader}},$$

the reader with a current-state factor bolted on. That extra $S\times$ — the shape, i.e. the state a morphism can *write back* — is Neil's "something more". The rigidity is genuine: perturbing a single fibre of ΔS away from S destroys the constant-fibre profile, and the store counit and comultiplication no longer typecheck (a negative control in the companion computations). The store structure lives exactly in the "all fibres $= S$" condition.

6.7.2 Workers, and why the state multiplies

Definition 6.40 (Worker)**.** A *Worker* from p to q *with state* S is a container morphism $w \colon \Delta S \otimes p \to q$. [Definition]

Writing $q = C \triangleleft D$ and $\Delta S \otimes p = S \times A \triangleleft \big((s, a) \mapsto S \times B(a)\big)$, such a w unpacks into a forward map and a *split* backward map:

- forward $f \colon S \times A \to C$ — from state s and input shape a, the output shape $f(s, a)$;
- backward $f^\sharp \colon D(f(s, a)) \to S \times B(a)$, i.e. a pair $\big(f_1^\sharp(s, a) \colon D(f(s, a)) \to S,\ f_2^\sharp(s, a) \colon D(f(s, a)) \to B(a)\big)$: the first component *writes back* a new state at each output position, the second is the ordinary position back-map.

So a Worker reads a state, transforms a p-interface into a q-interface, and writes a state back at each response — the read–write process the store comonad was built to carry.

The engine of composition is a single strict identity of containers.

[7] The directed-container laws D1–D5 for ΔS and the store comonad's counit/comultiplication laws are machine-checked in `StateComonad.lean`, which is present in the repository's `lean/` tree but not committed as source to this book's tree; the weaker in-text tag is used per the Preface convention.

Lemma 6.41 (The state multiplies)**.** $\Delta S \otimes \Delta T = \Delta(S \times T)$ *(strict equality of containers), and* $\Delta 1 = y$*. Hence* $S \mapsto \Delta S$ *is a strict monoidal map* $(\mathbf{Set}, \times, 1) \to (\mathbf{Cont}, \otimes, y)$ *on objects.* *[MacBeth, Lean-verified in* `Workers.lean/StateComonad.lean`[8]*]*

Proof. $\Delta S \otimes \Delta T = S \times T \triangleleft \big((s,t) \mapsto S \times T\big) = \Delta(S \times T)$: the shape sets and every position fibre coincide. $\Delta 1 = 1 \triangleleft (* \mapsto 1) = y$. □

Remark 6.42 (Why $\otimes$, and not $\times$). Lemma 6.41 is exactly why the tensor is the *Dirichlet* $\otimes$, whose positions multiply. Under the categorical product, $\Delta S \times \Delta T = S \times T \triangleleft \big((s,t) \mapsto S + T\big)$ has fibres of size $|S| + |T|$, not $|S \times T|$ — so $\Delta S \times \Delta T \neq \Delta(S \times T)$, and the state would *not* multiply. Only the tensor whose positions multiply makes the context multiply; the choice of $\otimes$ is forced by the phenomenon it is meant to model.

Given $w\colon \Delta S \otimes p \to q$ and $w'\colon \Delta T \otimes q \to r$, Lemma 6.41 and functoriality of $\otimes$ compose them into a single Worker:

$$\Delta(T \times S) \otimes p \;=\; \Delta T \otimes (\Delta S \otimes p) \xrightarrow{\Delta T \otimes w} \Delta T \otimes q \xrightarrow{w'} r.$$

A state-S worker followed by a state-T worker is a worker whose state is the *product* of the two: composition *accumulates* state. Applying the outer worker first, the coordinates land the state in $T \times S$; via the symmetry $\Delta T \otimes \Delta S \cong \Delta S \otimes \Delta T$ (both $= \Delta(S \times T)$, Lemma 6.41) we record the accumulated grade in the reading order $S \times T$, matching the grade multiplication of Theorem 6.43 below. In coordinates, with $r = E \triangleleft G$ and $e := g(t, f(s,a))$, the composite has forward $((t,s),a) \mapsto g(t, f(s,a))$ and, backward at a response $d' \in G(e)$, the state component $\big(g_1^\sharp(t, f(s,a), d'),\ f_1^\sharp(s, a, g_2^\sharp(t, f(s,a), d'))\big) \in T \times S$ and the position component $f_2^\sharp(s, a, g_2^\sharp(t, f(s,a), d')) \in B(a)$. The identity is $\mathrm{id}_p\colon \Delta 1 \otimes p = p \to p$, with state 1.

Theorem 6.43 (The category of Workers)**.** *There is a category* graded by $(\mathbf{Set}, \times)$, Workers, *with objects the containers, graded hom-sets*

$$\textsc{Workers}_S(p,q) := \mathbf{Cont}(\Delta S \otimes p,\ q),$$

identity in grade 1*, and composition* $\textsc{Workers}_S(p,q) \times \textsc{Workers}_T(q,r) \to \textsc{Workers}_{S \times T}(p,r)$ *as above, associative and unital up to the associator and unitors of* $(\mathbf{Set}, \times)$*. Equivalently,* Workers *is the coKleisli category of the* $(\mathbf{Set}, \times)$*-graded comonad* $S \mapsto \Delta S \otimes (-)$ *on* **Cont***, whose comultiplication is the strict equality* $\Delta(S \times T) \otimes - = \Delta S \otimes \Delta T \otimes -$ *of Lemma 6.41.* *[MacBeth, Lean-verified in* `Workers.lean`[9]*]*

Proof sketch. Composition lands in grade $S \times T$ by Lemma 6.41. Unitality: $\mathrm{id}_q \circ w$ has grade $1 \times S$ and, on unpacking, is w under the unitor $1 \times S \cong S$; symmetrically on the right. Associativity: both bracketings of $w_3 \circ w_2 \circ w_1$ have underlying morphism $w_3 \circ (\Delta U \otimes w_2) \circ (\Delta U \otimes \Delta T \otimes w_1)$ in **Cont**, equal by associativity of composition and functoriality of $\otimes$; the grades $(U \times T) \times S$ and $U \times (T \times S)$ agree by the associator of $(\mathbf{Set}, \times)$. The coKleisli reading holds because the comultiplication is the strict identity of Lemma 6.41. The graded-category laws are the content of `Workers.lean`. □

[8] $\Delta S \otimes \Delta T = \Delta(S \times T)$ holds by `rfl` in `StateComonad.lean`; as with Proposition 6.39, the file is in the `lean/` tree but not committed to this book's tree.

[9] `Workers.lean` defines the composition and checks the three graded-category laws (unit-left, unit-right, associativity), sorry-free; the finite cases of the underlying identities were independently exhausted by direct computation. As above, the file is not committed to this book's tree.

Remark 6.44 (S varies: the Para reading, and the nearest neighbour). Collapsing the grade into a single hom-set, $\textsc{Para}(p, q) := \sum_{S:\mathbf{Set}} \mathbf{Cont}(\Delta S \otimes p, q)$ — a Worker *together with* its choice of state — composed by tensoring the states, is Gavranović's *Para* construction [27] for the action $S \cdot p = \Delta S \otimes p$ of $(\mathbf{Set}, \times)$ on $\mathbf{Cont}$. This identification is honest only over the core groupoid $\mathrm{Core}(\mathbf{Set})$, the level at which Δ is functorial: a bare function $S \to S'$ gives no canonical morphism $\Delta S \to \Delta S'$ (the backward part would need $S' \to S$), so over all of $\mathbf{Set}$ we have a *graded* category, not a strict actegory. The Para identification and its exact match to Gavranović's axioms are *computed / further work*, not proved.

The nearest neighbour in the literature is Capucci–Myers' *contextads* [26]. Their Example 3.24 — every M-graded comonad is a *trivially-fibred contextad*, i.e. a colax action $\mathcal{C} \times M \to \mathcal{C}$ — identifies Workers precisely: with $M = (\mathbf{Set}, \times)$ it is the trivially-fibred, colax-action corner of their framework. Their Theorem A.4 ($\mathrm{Kl}(T) \cong \mathrm{Ctx}(\odot)$, which they leave unproven) sits in the *opposite*, dependency-essential corner: there the grade is a dependent family $X \to S$ over the fixed shapes of one monad, multiplying via that monad's own seq, whereas ours is an external grade ranging over all of $(\mathbf{Set}, \times)$ and multiplying cartesianly. We settle *no* fragment of their A.4; Workers and A.4 are the two ends of the same axis. [Para: MacBeth, computed / further work; citation [27] at browse-summary level (deep-read to-do). Neighbour placement vs [26] (deep-read): MacBeth.]

For the grant: two axes of composition. Workers isolates one of two orthogonal axes along which compositional systems combine. Here the context *multiplies*, $S \times T$; the grade accumulates monotonically and there is *no obstruction* — every pair of Workers composes, always. This is the *state* axis. It is dual to the *directed* axis of Chapter 7, where composing two directed systems is a Zappa–Szép problem that *may fail to have a solution*, the failure measured by a class $[\omega] \in H^2$ (Theorem 7.4). State composition is unobstructed but accumulating; directed composition is non-accumulating but obstructed. A realistic compositional system — an orchestration of stateful agents, a supply chain of processes with memory — lives on both axes at once, and the grant's compositional-correctness story is the joint account: track the state grade for what the system *remembers*, track the H^2 class for whether it *composes at all*. A *third* axis — the effect–coeffect one, obstructed by branching — joins these two once the arrow category of the next chapter is in hand; the three are assembled into a single table in §7.4.

Forward pointer: a Worker as an effect–coeffect arrow

A Worker $\Delta S \otimes p \to q$ reads a coeffect (the incoming state, a *comonadic* context) and, through its writeback, produces an effect (the outgoing state). It is thus one instance of a general *effect–coeffect arrow*, in which a comonad supplies the context and a monad the output, welded by a distributive law relating the two feeds of §6.6.1 — read now as the compositor of a Freyd/arrow category rather than of algebras. That arrow category, and the branching condition under which it exists, are developed as the *third* mode of composition in §7.4 (Theorem 7.6): the arrows form a category iff the effect monad is non-branching. One sub-question is left open here: whether the store comonad's Workers *embed into*, or *generate*, that arrow category, and how the state grade of §6.7.1 interacts with an effect grade, is [in development] — recorded as a pointer, with no result claimed.

6.8 Syntax and behaviour

Step back and the two constructions line up as a single duality. The free monad is *syntax*: the finite terms a signature generates, composed by substitution, universal among monads — the moves a system *can make*. The cofree comonad is *behaviour*: the possibly-infinite execution

trees a signature can unfold, decomposed by taking subtrees, cofree among comonads — the observations a system *can offer*. Both stay inside **Cont**; both are computed by a (co)inductive tree construction; and the position data — leaves for terms, nodes for behaviours — is exactly what the fixed-point recursion produces.

The asymmetry that made the cofree side the easy side (Theorem 3.4: connected-limit preservation) is the same asymmetry that makes the cofree side land back on the equivalence chain. The free monad is a *monad* and stops there; monads for $\triangleleft$ are not, in general, categories. But the cofree *comonad* is a comonoid, and every comonoid for $\triangleleft$ is a directed container — a category. This is why behaviour, and not syntax, is where the geometry of Chapter 5 reappears: the space of behaviours of any container is organised by its subtree category.

One op, two faces. Both machines of this chapter are the fibrewise $(-)^{\mathrm{op}}$ doing its work, and it is worth seeing that they apply it to *different targets*. A container morphism runs backward on positions — $\mathbf{Cont} = \int(\mathrm{cod})^{\mathrm{op}}$, the teachbox after Proposition 6.17 — so every position fibre carries a built-in $(-)^{\mathrm{op}}$. The *transfer* applies that op to the fibre *object*: it turns the base monad M into the fibrewise comonad $(M^{\mathrm{op}})_*$, so a monad on the base becomes a comonad on **Cont** (Proposition 6.17). The *free/cofree* pair applies the same op to the *recursion scheme*: dualising an initial algebra to a final coalgebra, well-founded substitution-syntax μ to possibly-infinite behaviour-syntax ν, sends the free monad to the cofree comonad. One op, two faces — on the fibre object it exchanges monad and comonad; on the recursion scheme it exchanges syntax and behaviour — and the entwining (Theorem 6.21) is what happens when the first face meets a second copy of M: the two container feeds of one monad, welded by M's oplax product-comparison.

For the grant. Free monads and cofree comonads are the syntax/behaviour pair of a compositional system: its available moves and its observable unfolding. A verified account of their composition would let one certify, for instance, that an agent-orchestration combinator preserves the update (*Put*) structure of its parts (Corollary 5.10), by tracking it through the free monad of the combined signature. The free-monad Lemma (Theorem 6.10) is the tool that makes such a combinator *definable* by its action on generators alone — the universal property doing the work a bespoke construction would otherwise have to.

What remains open. Two loose ends are worth naming. First, the cofree side awaits its Lean certificate: the M-type carrier needs a coinductive framework (Theorem 6.13, footnote), which is an infrastructure step, not a mathematical one. Second, the free-monad Lemma is proved in coordinates but only partly machine-checked (Theorem 6.10); completing the multiplication-homomorphism law in Lean is the next formalisation target. Neither gap touches the constructions themselves, which are explicit and, in the finite cases of Examples 6.9 and 6.14, directly computable.

Chapter 7

Composing systems: Zappa–Szép and distributive laws

Chapter 5 settled what a single directed container is and what maps it has. This chapter asks how *two* of them compose into one — the question with the economic consequences (mis-composed supply chains lose liability traceability; a wrong contract composite is an exploit). We package this as a *Zappa–Szép* (two-sided semidirect / matched-pair) product of small categories, the categorical generalisation of the group/monoid Zappa–Szép product.

7.1 The pairwise criterion

For objects a, b of a small category, $\mathrm{Hom}(a, b)$ is canonically a right $\mathrm{End}(a)$-set (precomposition) and a left $\mathrm{End}(b)$-set (postcomposition). A Zappa–Szép decomposition $K = \mathcal{C} \bowtie \mathcal{D}$ of K over a wide subcategory $\mathcal{D}$ asks for a *unique factorisation* of each morphism through a transversal and an endomorphism.

Theorem 7.1 (Pairwise Zappa–Szép criterion)**.** *Let $\mathcal{D}$ be a wide subcategory of K. Then $K = \mathcal{C} \bowtie \mathcal{D}$ for a complementary wide subcategory $\mathcal{C}$ if and only if*

- (L) *each hom-presheaf $\mathrm{Hom}_K(-, b)$ is a* free *right $\mathcal{D}$-set (equivalently: the source-oriented factorisation $f = t \circ e$ is unique for every f), and*
- (G) *the free bases close into a wide subcategory $\mathcal{C}$.*

[MacBeth][1]

Theorem 7.2 (Cocycle axioms $\Longleftrightarrow$ associativity)**.** *The Zappa–Szép cocycle axioms ZS1–ZS4 (relating the two actions of a matched pair) hold if and only if the product on $\mathcal{C} \bowtie \mathcal{D}$ is associative.* *[MacBeth]*[2]

7.2 (G) as a strict factorisation system, and its H^2 obstruction

Proposition 7.3 ((G) is a strict factorisation system)**.** *Condition (G) — that the free bases close into a wide subcategory — is exactly the statement that the decomposition extends to a*

[1] This is the corrected form of the SEED conjecture "every small category is a Zappa–Szép product of a poset skeleton and its vertex monoids," which was *refuted* computationally (smallest counterexample: two objects, one arrow $s\colon a \to b$, $\mathrm{End}(a) = \mathrm{End}(b) = \mathbb{Z}/2$ fixing s, so $2 \nmid 1 = |\mathrm{Hom}(a, b)|$). The freeness half (L) is written up self-contained in `papers/pairwise-zappa-szep.tex` (a built PDF is in the repo; the `.tex` source is not yet committed to this tree). A brute-force enumeration of ≈ 26 small categories matches the criterion exactly.

[2] MacBeth has a Lean formalisation of the monoid case (ZS1–ZS4 $\Longleftrightarrow$ associativity of the Zappa–Szép product) in `ZappaSzep.lean`, built and reported zero-`sorry`; that source is *not yet committed* to this repository's `lean/` tree, so the tag is [MacBeth] rather than Lean-verified here.

strict factorisation system *on* K. *Distributive laws of categories correspond to strict factorisation systems.* *[Cited: Rosebrugh–Wood (distributive laws $\Longleftrightarrow$ SFS)]*

Theorem 7.4 (The (G) obstruction is an H^2 class)**.** *Under a trivial-action hypothesis, (G) is classified by a second cohomology class:* (G) $\Longleftrightarrow$ $[\omega] = 0$, *where* $[\omega]$ *lives in the* Baues–Wirsching *cohomology* $H^2(\mathrm{Sk}_{\mathcal{C}}; \mathcal{D})$ *of the* $\mathcal{C}$*-skeleton with a source-only (presheaf) natural system of coefficients. This is the split case of the Baues–Wirsching linear-extension classification.* *[MacBeth (identification), resting on Cited: Baues–Wirsching 1985]*[3]

7.3 The classical anchor

Proposition 7.5 (Classical Zappa–Szép recovered)**.** *Specialising Theorem 7.1 to a one-object category (a monoid) recovers the classical Zappa–Szép product of monoids, and (L) becomes the familiar condition that the factorisation map is a bijection.* *[MacBeth]*[4]

7.4 Three modes of composition

The book has now built, in three different chapters, three different answers to a single question — *when do two compositional systems compose into one, and what obstructs it?* — and it is worth setting them side by side, because they do not agree, and the disagreement is the content. The Zappa–Szép product of this chapter welds two *directed* systems (two small categories, §7.1); the category of Workers of §6.7 welds two *stateful* processes; and the entwining of §6.6.1 welds an *effect* to a *coeffect*. Each is a genuine mode of composition, each has a precise criterion for when the weld holds, and the three criteria are three different kinds of mathematics. This is the grant's compositional-correctness question, and the honest answer is not one obstruction but a table of them.

Mode	Object welded	Obstruction	Always exists?
Directed (ZS)	$\mathcal{C} \bowtie \mathcal{D}$, two categories	$[\omega] \in H^2(\mathrm{Sk}_{\mathcal{C}}; \mathcal{D})$	*No* — may fail
State (Workers)	$\Delta S \otimes p \to q$, stateful maps	*none*	*Yes* — grade accumulates
Effect–coeffect	arrows $G_M p \to T_M q$	branching (M arity ≥ 2)	*No* — iff M non-branching

A cohomology class, nothing at all, and a Boolean condition on an arity: three obstructions of three different types, for three different ways of gluing. The temptation is to collapse the rows into one master theorem — "compositional systems compose iff a single invariant vanishes" — and it should be resisted, because it is false: H^2, the empty obstruction, and the branching flag do not unify into one class. The synthesis that is true, and that the grant rests on, is weaker and better: *heterogeneous systems weld along an axis, and whether the weld holds is a computable invariant of the pieces — but which invariant depends on the axis.* We take the three rows in turn.

[3] The cohomological *identification* — that the obstruction to (G) is a Baues–Wirsching H^2 class and that "(G)$\Leftrightarrow$ $[\omega] = 0$" is the split case of the BW linear-extension theorem — is MacBeth's; the underlying classification theorem is Baues–Wirsching, *Cohomology of small categories*, JPAA **38** (1985) 187–211 (modern account: arXiv:2508.00727). The rigid twist of the counterexample above is the $\mathbb{Z}/2$ generator; the nonabelian boundary is the Kac/Rosebrugh–Wood law.

[4] MacBeth flags an earlier *over*claim for correction: the "Ahman–Uustalu update monad as a degenerate Zappa–Szép product" reading is *not* a special case of the two-category matched pair — it is a monoid acting on a set, not two composable monoids — and should not be presented as such.

The directed axis — an obstruction that can genuinely fail. Composing two directed containers is the subject of this chapter: it is a Zappa–Szép problem, solvable exactly when the free factorisation bases close into a subcategory (condition (G), Theorem 7.1), and that closure is classified by a cohomology class $[\omega] \in H^2(\mathrm{Sk}_{\mathcal{C}}; \mathcal{D})$ in Baues–Wirsching cohomology (Theorem 7.4). The class can be nonzero — the two-object category with $\mathbb{Z}/2$ vertex groups fixing a single crossing arrow is the minimal witness (footnote to Theorem 7.1) — so this mode of composition *can fail*, and when it fails the failure is a specific, computable cocycle. This is the mode with economic teeth: a mis-composed supply chain, a wrongly matched pair of contracts, is exactly a nonzero $[\omega]$.

The state axis — no obstruction, but the context multiplies. Composing two Workers, by contrast, *always* succeeds. A Worker is a container map $\Delta S \otimes p \to q$ carrying a state space S (§6.7); two of them compose because $\Delta S \otimes \Delta T = \Delta(S \times T)$ strictly (Lemma in §6.7.1), and the composite simply carries the product state $S \times T$. There is no closure condition to check and no class to vanish; the price of unconditional composition is that the grade *accumulates*, monotonically, forever. This is the $(\mathbf{Set}, \times)$-graded category of §6.7: composition is total, the obstruction set is empty, and what a realistic system pays instead is memory.

The effect–coeffect axis — obstructed by branching. The third mode is the one the entwining of §6.6.1 was built for, now read as a composition rule rather than a lifting of algebras. Fix a base monad M and recall its two container feeds: the effect monad $T_M S \triangleleft P = MS \triangleleft P^\star$ of Ahman–Bauer and the coeffect comonad $G_M S \triangleleft P = S \triangleleft M \circ P$ of the transfer. Section 6.6.1 assembled these into the *arrow profunctor* $\mathsf{Arr}_M(p,q) = \mathbf{Cont}(G_M p, T_M q)$ (Definition 6.23): an effect–coeffect arrow $p \rightsquigarrow q$ reads its input through the coeffect G_M (a comonadic context) and delivers its output through the effect T_M. The profunctor exists for every M; the question of *composition* is the question whether Arr_M is the hom-profunctor of a category, and its biKleisli composite threads one arrow's effect past the next arrow's coeffect through the *reverse* entwining $\kappa \colon G_M T_M \Rightarrow T_M G_M$ — the orientation opposite to the law λ of Theorem 6.21. Whether the arrows compose is therefore whether κ is a distributive law, and §6.6.1 already told us the answer: the reverse orientation holds precisely when M does not branch.

Theorem 7.6 (Effect–coeffect arrows, and their positive class)**.** *Let M be a cartesian ($\prod$-cointerpretation)* **Set***-monad, with effect monad T_M and coeffect comonad G_M as above. The arrow profunctor $\mathsf{Arr}_M(p,q) = \mathbf{Cont}(G_M p, T_M q)$ (Definition 6.23), with the biKleisli composite whose compositor is the reverse entwining $\kappa \colon G_M T_M \Rightarrow T_M G_M$, is the hom-profunctor of a category — indeed a Hughes arrow, equivalently a Freyd category, over $(\mathbf{Cont}, \times)$ —* if and only if *M is non-branching (every shape has arity ≤ 1). The sole obstructed axiom is associativity (the multiplication axiom $E2'$ of the distributive law). Moreover, a cartesian M is non-branching if and only if it is a* writer-with-absorbing-exceptions *monad*

$$MX \;\cong\; E + A \times X, \qquad (A, \cdot, e) \text{ a monoid}, \quad (E, \odot) \text{ a left } A\text{-set},$$

with writer unit $x \mapsto (e, x)$ and the log A acting on the exceptions E; and for every *monad in this class κ satisfies all four axioms $E1'$–$E4'$, so the arrow category exists throughout it.* [MacBeth][5]

[5] Proved in `proofs/2026-07-29-effect-coeffect-arrows.md` (the non-branching criterion, Theorem A), `...-arrows-first.md` (the Hughes arrow / Freyd characterisation, Theorem B), and `proofs/2026-07-30-affine-classification.md` (the positive class $E + A \times (-)$ and the fact that $E2'$ holds across it). The effect monad T_M is Ahman–Bauer [10] Thm 6.3; the coeffect comonad G_M and the compositor κ are the transfer (Proposition 6.17) and the reverse of Theorem 6.21. The unit-law fragment of the arrow category is machine-checked (`BiKleisli.lean`, not committed to this tree); the associativity axiom $E2'$ is proved in coordinates and machine-verified for the **Set**-monad examples, not yet Lean-certified in general, so the tag is [MacBeth].

The mechanism behind the last clause is worth one line, because it is why the positive class is so clean. Non-branching means every element of every M-object has at most one leaf, so every product $\prod_b$ that appears in T_M, T_MT_M, G_MT_M, … ranges over *at most one* factor. The lax comparison κ is then, shape by shape, either the identity (one leaf: rewrap the single factor) or the monad unit $\eta^M : 1 \to M1$ (no leaf: the empty product). The two-or-more-leaf comparison — the *union of products versus product of unions* discrepancy that Example 6.22 used to break the reverse law for Pf — never forms, and the associativity axiom $E2'$ degenerates to the associativity of the monoid A, which of course holds. Branching is precisely the appearance of a second leaf, and a second leaf is precisely what lets the multiplication μ^M merge two positions that the cartesian comparison cannot keep correlated.

The deepest point: one entwined structure, two faces, and which direction you commute. The effect–coeffect obstruction is not really about *whether* the two feeds of M interact — they always do — but about *which way*. The single monad M, feeding **Cont** through T_M and G_M, carries two comparison maps between its composites T_MG_M and G_MT_M, and they behave oppositely.

- The **bialgebra face** is $\lambda\colon T_MG_M \Rightarrow G_MT_M$, built from the *oplax* product-comparison str (Theorem 6.21). It is an entwining for *every* M — including the branching ones — and it is what lifts G_M to a comonad on T_M-algebras and T_M to a monad on G_M-coalgebras. This is the unconditional half: the affirmative answer to the Plotkin–Turi question of whether one monad's effect and coeffect feeds admit a bialgebraic semantics. They always do.
- The **arrow face** is $\kappa\colon G_MT_M \Rightarrow T_MG_M$, the *reverse*, *lax* comparison (Theorem 7.6). It is an entwining — and hence turns the arrow profunctor Arr_M into a genuine *category* — *only* when M is non-branching.

Same monad, same two feeds, same **Cont**: the unification of effects and coeffects *exists* unconditionally as a bialgebra, and its *categorical packaging as an arrow calculus* is what branching obstructs. The obstruction is not to the interaction but to its orientation — to commuting the effect past the coeffect in the direction that composition of arrows demands. That the same variance which forced G_M to be a comonad (positions run contravariantly, §1) also forces the *single* legal direction of the interaction is, to this author, the most satisfying fact in the chapter.

Remark 7.7 (A neighbour with a different engine: interaction laws). The reader who knows Katsumata, Rivas and Uustalu's *interaction laws of monads and comonads* [32] should be told how that framework differs from this one, because it answers a nearby question by other means. Their interaction law is a *pairing* $\psi\colon TX \times DY \to X \times Y$ — run an effect against a coeffect, extract a value and a final state — realised as a monoid object in a Chu construction over $([\mathcal{C},\mathcal{C}], \mathrm{Day})$; it is emphatically *not* a distributive law, and indeed distributive laws are *inputs* to their machinery (used to glue pairings on composite monads), never outputs. Our κ is a distributive law and nothing else, so the two accounts are orthogonal in kind — a pairing versus a compositor — rather than rival; what they share is striking only at the level of the obstruction, for their degeneracy theorems (their Thms 1–3) locate the same branching wall from the side of extensive categories that our arity criterion locates from the side of polynomial functors.[6]

For the grant: Path 5. These three modes are the spine of the grant's Path 5 — *a container-native theory of when compositional systems compose*. An orchestration of stateful agents, a supply chain of processes with memory, an ontology merge: each is a system that must be glued

[6] On [32] at deep-read (full text, `reading/2026-07-29-katsumata-rivas-uustalu-1912-13477.md`); the pairing/ compositor distinction and the "distributive laws are inputs" reading are verified against their §3 and §6.

to another, and each gluing runs along one or more of these axes at once. The account this book can offer is exactly the table above: to certify that two systems compose, identify the axis, then compute the corresponding invariant — the Baues–Wirsching class for a directed weld, the accumulated grade for a stateful one, the arity of the effect monad for an effect–coeffect one. None of the three invariants is a heuristic; each is forced by the categorical structure and (for the first two, and the finite cases of the third) machine-checkable. The grant's claim is not that composition is always possible — the directed and effect–coeffect rows say plainly that it is not — but that *whether* it is possible is, in every mode, a decidable property of the parts. That is a stronger and more honest promise than a single master obstruction would have been, and it is what three chapters of container theory have earned the right to state.

7.5 When two of the modes meet: emergent holonomy

The table put *State* and *Directed* in different rows, and it was right to do so: their obstructions are a nothing and a cohomology class, and no honest master theorem folds them together. But a table is a still photograph, and it can hide what happens when you make the systems *move*. Compose two *stateful* agents — two objects from the State row — and watch what appears. What appears is a Zappa–Szép product: an object from the Directed row. The two modes were never as far apart as their rows; they meet on a single object, and on that object something is waiting that neither agent alone contains.

The promise. Composing two holonomy-free threading agents can *synthesise* holonomy that neither possesses; a single integer — the number of states at which the two agents' reachable-state orbits cross — measures exactly how much; and when that integer is 1 everywhere, a degree-2 cohomology class decides whether the two agents' symmetries stay unentangled or fuse. Orchestration remembers more than its parts, and we can say precisely how much more.

One agent, recalled. Chapter 5's climax (§6.6.6) classified the liftings of a *single* update monad $\mathrm{Upd}_{(S,P,\downarrow)}$ by the functors $\mathbb{A}(\downarrow) \to \mathbf{Cat}$ (Theorem 6.36). The one datum that is not a count is the *holonomy*: the restriction of such a functor to the loops at a state s — the isotropy monoid $\mathrm{Stab}(s)$ — is a representation of $\mathrm{Stab}(s)$ on the fibre category $\mathcal{C}_s$. Reader and State are the degenerate agents whose isotropy is trivial; a generic agent carries a real representation, and orchestrating it must compose that representation into the whole. This section is about *how*.

Two agents make a Zappa–Szép product

Take two update monads that share the state set S: $\mathrm{Upd}_{(S,P,\downarrow)}$ and $\mathrm{Upd}_{(S,P',\downarrow')}$, with update monoids P, P' threading S by $\downarrow$ and $\downarrow'$. To compose them into one monad is to give a distributive law between them, and a distributive law is exactly a *matched pair*: mutual actions of P and P' that make the carrier $P \times P'$ a monoid — the Zappa–Szép product $P \bowtie P'$ (§7.1; distributive laws are strict factorisation systems, Rosebrugh–Wood [22]), with

$$(p, p') \cdot (q, q') \;=\; \big(\, p\,(p' \triangleright q)\,,\; (p' \triangleleft q)\,q' \,\big), \qquad \text{unit } (o, o').$$

This is a Zappa–Szép product of two *monoids*, the classical anchor of Proposition 7.5 — two composable agents, not the mis-reading (a single update monad as a degenerate matched pair) that its footnote warns against. The matched pair also threads the shared state: $s \downarrow_{\bowtie} (p, p') := (s \downarrow p) \downarrow' p'$ is a right action of $P \bowtie P'$ on S, and the composite functor is again an update monad — the one built from $P \bowtie P'$ acting on S. So the classification of §6.6.6 applies verbatim to the composite, and it applies through the Zappa–Szép product.

Theorem 7.8 (The classifier composes by the Zappa–Szép product)**.** *The composite of* $\mathrm{Upd}_{(S,P,\downarrow)}$ *and* $\mathrm{Upd}_{(S,P',\downarrow')}$ *is the update monad of* $P \bowtie P'$ *acting on* S *by* $\downarrow_{\bowtie}$*. Its degree-*1 *proof-relevant liftings are therefore classified by*

$$\mathbf{Fun}\big(\mathbb{A}(\downarrow_{\bowtie}), \mathbf{Cat}\big) \;=\; \mathbf{Fun}\big(\mathbb{A}(\downarrow) \bowtie \mathbb{A}(\downarrow'), \mathbf{Cat}\big),$$

where $\mathbb{A}(\downarrow) \bowtie \mathbb{A}(\downarrow')$ *is the Zappa–Szép product of the two action categories: objects* S*, an arrow* $s \xrightarrow{(p,p')} (s \downarrow p) \downarrow' p'$ *for each matched pair, composed by the Zappa–Szép multiplication. [MacBeth (2026-08-12-holonomy-composition-zs-bridge.md, part (a)), on Theorem 6.36 and the update monad of Ahman–Uustalu [8]; the matched-pair composition is the monoid case of §7.1 ([22]). [open] at higher object-degree, inherited.]*

The classifier of the Ch. 5 climax is *monoidal under orchestration*: the classifying object of the composite is built functorially from the two factor action categories by the same Zappa–Szép product that welds two directed systems in §7.1. This is the meeting of the two rows, stated as an equality of classifiers. Now for what lives at the meeting.

The naive guess, and why it is wrong

Each agent has, at a state s, its own isotropy: $\mathrm{Stab}_P(s)$ and $\mathrm{Stab}_{P'}(s)$, and hence its own holonomy. The composite has an isotropy too — $\mathrm{Stab}_{P\bowtie P'}(s)$, the loops at s in the product action category. The natural guess, the one the table invites, is that the composite holonomy is just *the two together*:

$$\text{(guess)} \qquad \mathrm{Stab}_{P\bowtie P'}(s) \;\overset{?}{\cong}\; \mathrm{Stab}_P(s) \bowtie \mathrm{Stab}_{P'}(s).$$

Half of it is true and free: if p fixes s and p' fixes s then so does (p, p'), so the right side always *includes* into the left. The equality is the claim that every loop of the composite factors into a loop of each agent — that the composite remembers nothing the parts did not already carry.

It is false. And its failure is the phenomenon.

Example 7.9 (S_3 on three points: holonomy from nowhere)**.** Let $G = S_3$ act on $S = \{1, 2, 3\}$, and factor it exactly as $G = P \cdot P'$ with $P = A_3 = \langle(123)\rangle$ the rotations and $P' = \langle(12)\rangle$ a single transposition. Every permutation is uniquely a rotation times a chosen transposition ($|A_3| \cdot |\langle(12)\rangle| = 6 = |S_3|$ and $A_3 \cap \langle(12)\rangle = \{e\}$), so this is a genuine Zappa–Szép factorisation, and it acts on S. Look at $s = 1$.

- **Each agent is holonomy-free here.** No rotation fixes 1, so $\mathrm{Stab}_P(1) = \{e\}$; the transposition (12) moves 1, so $\mathrm{Stab}_{P'}(1) = \{e\}$. Both factor holonomies at 1 are trivial. The guess predicts the composite holonomy is trivial too.
- **The composite is not.** In S_3 the stabiliser of 1 is $\mathrm{Stab}_G(1) = \langle(23)\rangle \cong \mathbb{Z}/2$ — and (23) factors as $(132) \cdot (12)$, a rotation times the transposition, *neither of which fixes* 1. Read it as a loop: start at 1, apply the P'-leg (12) to reach 2, then apply the P-leg (132) to return to 1.

  $$1 \xrightarrow{(12)} 2 \xrightarrow{(132)} 1.$$

  A dispatch and a return. Neither step is a stabiliser; their composite is. The composite carries a genuine $\mathbb{Z}/2$ of holonomy on the fibre $\mathcal{C}_1$ — a nontrivial fibre automorphism — that *neither agent possesses*.

This is reentrancy created by orchestration. The equality-guess would have hidden it exactly; its failure is the discovery. [MacBeth (2026-08-12-holonomy-composition-zs-bridge.md, part (b), refuting the equality; the containment holds always). The S_3 witness is Lean-verified, axiom-free (EmergentHolonomy.lean), not committed to this tree. Exhaustively swept: the inclusion held in all 448 point-checks over the exact factorisations of $S_3, S_4, A_4, D_4, \mathbb{Z}/2\times\mathbb{Z}/2$ and was *proper* in 268 — including the abelian $\mathbb{Z}/2\times\mathbb{Z}/2$, so emergence is not a nonabelian artefact.]

The measure of emergence: where the orbits cross

How much holonomy does orchestration synthesise at s? The answer is a single positive integer with a picture attached. Each agent has, from s, a *reachable-state orbit*: $s \cdot P$ under the first, $s \cdot P'$ under the second. In the S_3 example $s \cdot P = \{1, 2, 3\}$ (rotations reach everything) while $s \cdot P' = \{1, 2\}$; the two orbits cross at $\{1, 2\}$, two points. That 2 is exactly the invariant $h(1)$ we are about to define, and here — where both factor isotropies are trivial — it is also the order of the emergent holonomy group.

Theorem 7.10 (Emergent holonomy = meeting points)**.** *For an exact factorisation $G = P \cdot P'$ acting on S — the internal realisation of the composite threading, $s \cdot g = (s \downarrow p) \downarrow' p'$ for $g = pp'$ — write $U = \mathrm{Stab}_G(s)$, $A = \mathrm{Stab}_P(s)$, $B = \mathrm{Stab}_{P'}(s)$. The composite "dispatch-and-return" loops at s, taken up to the two factor symmetries, are in canonical bijection with the crossing points of the two reachable-state orbits; and their number*

$$h(s) \;:=\; |A\backslash U/B| \;=\; \frac{|\mathrm{Stab}_G(s)|}{|\mathrm{Stab}_P(s)| \cdot |\mathrm{Stab}_{P'}(s)|} \;=\; \big|(s \cdot P) \cap (s \cdot P')\big|$$

is a positive integer, equal by all three formulas. Moreover

$$h(s) = 1 \quad\Longleftrightarrow\quad (s \cdot P) \cap (s \cdot P') = \{s\} \quad\Longleftrightarrow\quad \mathrm{Stab}_G(s) = \mathrm{Stab}_P(s) \cdot \mathrm{Stab}_{P'}(s)\,:$$

the two orbits meet only *at s exactly when the composite holonomy is honest — every composite loop splits into one loop of each agent, and the guess above holds after all.* *[MacBeth (2026-08-13-emergent-holonomy-meeting-points.md, Thm. 6, Cor. 7); verified with 0 mismatches across 9 groups incl. A_5, 2594 point-checks.]*

The middle formula deserves a second look, because *a priori it is not even an integer*: it divides the size of a subgroup by the sizes of two others that need not have anything to do with each other. The next box says why it always comes out whole.

Why the naive ratio is a whole number

The one fact behind Theorem 7.10 is a property of exact factorisations alone: for *every* $g \in G$,

$$P \cap gP'g^{-1} = \{e\}.$$

No action, no point s — just that P, P' are complementary subgroups ($P \cap P' = \{e\}$, $|P||P'| = |G|$). It follows that inside $U = \mathrm{Stab}_G(s)$ every double coset AgB has the *same* size, $|A| \cdot |B|$; so their number is exactly $|U|/(|A||B|)$, an integer, and it counts the loops-up-to-factor-symmetry. The geometric identity $= |(s \cdot P) \cap (s \cdot P')|$ is the payoff: the waypoint t of a loop $s \to t \to s$ is a crossing point of the two orbits, and distinct emergent loops pin distinct crossings. One honest double coset (waypoint s) is the aligned part where both legs fix s; the other $h(s) - 1$ are the synthesised reentrant loops. [Lemma 1 of 2026-08-13-emergent-holonomy-meeting-points.md; the disjointness checked 41 064 times, 0 violations.]

So the emergent holonomy is not a mystery to be feared but a number to be computed: $h(s) - 1$ is the count of reentrant loops that orchestration manufactures at s, one for each state where the two agents' reachable orbits cross beyond the obvious. An auditor of a composed system — a supply chain, a pair of contracts, two orchestrated agents — reads it straight off the two state actions. No cohomology is needed to *detect* emergence; only, as we now see, to classify the case where there is none.

The clean case, and the class that certifies it

When is the composite a clean product — when do the two agents' holonomies sit on independent registers of the fibre, ignoring each other? By Theorem 7.10 the precondition is $h(s) = 1$:

the orbits meet only at s, so the composite isotropy $E := \mathrm{Stab}_G(s)$ is genuinely the Zappa–Szép product $A \bowtie B$ of the two factor isotropies. But being a product of the two groups is not yet being a product of their *representations*. That is the last obstruction, and it is one degree up.

Theorem 7.11 (The split obstruction is a degree-2 class)**.** *Suppose s is aligned ($h(s) = 1$) with A abelian and normal in E. Then E is an extension $1 \to A \to E \to B \to 1$, classified by a class $[\omega] \in H^2(B; A)$; and when the conjugation action is trivial (in particular for the $\mathbb{Z}/2$ witness),*

$$[\omega] = 0 \iff E \cong A \times B \iff \textit{every composite holonomy is } \text{unentangled } (\rho(A), \rho(B) \textit{ commute in } \mathrm{Aut}(\mathcal{C}_s)).$$

A nonzero $[\omega]$ is the abstract *non-split extension (e.g. $\mathbb{Z}/4$), a composite holonomy whose two agent-parts cannot be disentangled. [MacBeth (2026-08-12-holonomy-composition-zs-bridge.md, part (c′)), resting on the Eilenberg–Mac Lane / Baues–Wirsching extension classification [23]. Scope: aligned, abelian, normal A — the honest domain; the nonabelian case is deliberately not claimed.* Realisability caveat *(2026-08-14-two-omega-sites-isotropy-restriction.md, Thm. 2.5): within this very hypothesis an aligned point-stabiliser with A normal is a semidirect product $E = A \rtimes B$ and hence* split, *so $[\omega] = 0$ always; the nonzero class classifies an abstract extension but is not realised by an aligned Zappa–Szép point-stabiliser (see the teachbox below).]*

The smallest instance is the whole story in miniature. Take both isotropies to be $\mathbb{Z}/2$, acting trivially, so $H^2(\mathbb{Z}/2; \mathbb{Z}/2) \cong \mathbb{F}_2 = \{0, \varepsilon\}$:

$[\omega]$	extension E classified by $[\omega]$	the two agents' holonomies	composite
0	$\mathbb{Z}/2 \times \mathbb{Z}/2$	two $\mathbb{Z}/2$ representations	*unentangled* (they commute)
ε	$\mathbb{Z}/4$ (cyclic, non-split)	two $\mathbb{Z}/2$ representations	*entangled* order-4 automorphism

The two agents' holonomies are the same in both rows — two copies of $\mathbb{Z}/2$. What differs is whether they combine into two independent commuting symmetries of the fibre or braid into a single order-4 one, and that difference is a single bit, $[\omega]$. This is the clean resolution of a degree gap that dogged the whole story: the factor holonomies are degree-1 data (representations), and *whether they assemble without entanglement* is decided by a degree-2 class — never by an equality of the two.

Same $\mathbb{Z}/2$, two different sites — and why the naive identification fails

Twice now an H^2 class with a $\mathbb{Z}/2$ generator has decided whether a Zappa–Szép composite is clean, and both times the generator is the *same* token bit ε read off the *same* composition data. The first was the handoff obstruction of Theorem 7.4, $[\omega_h] \in H^2(\mathrm{Sk}_{\mathcal{C}}; \mathcal{D})$ — Baues–Wirsching cohomology of the *handoff category*, whose objects are the roles a token passes through, deciding whether two directed systems compose at all. The second is the stabiliser obstruction just above, $[\omega_{\mathrm{st}}] \in H^2(B; A)$ — ordinary group cohomology of a single *point-stabiliser*, whose one object is a group, deciding whether an already-formed composite holonomy splits. Same cyclic group, same generator, same underlying bit. Surely they are one class seen twice?

They are not, and the reason is worth the box. Baues–Wirsching cohomology *generalises* group cohomology: a one-object category *is* a group G, and there H^2_{BW} is exactly $H^2(G; -)$. For a general category $\mathcal{C}$ every object x still carries its automorphism group $\mathrm{Aut}(x)$, and *restriction to isotropy* is a natural map

$$i_x^* \colon H^2_{\mathrm{BW}}(\mathcal{C}; \mathcal{D}) \longrightarrow H^2\big(\mathrm{Aut}(x); \mathcal{D}(x)\big)$$

that reads off the part of a category class living on the *loops* at x. It is the only map of the right shape — degree 2, correct variance — that could turn the handoff class (a category class) into the stabiliser class (a group class). So compute it. The handoff category $\mathrm{Sk}_{\mathcal{C}}$ is *automorphism-rigid*: its objects are the roles Sup $\to$ Wk $\rightrightarrows$ Rt, and not one of them has a non-identity self-map — the token that could have been a loop was pushed into the *coefficient* $\mathcal{D}(\mathrm{Sup}) = \mathbb{Z}/2$, not into an arrow. Every isotropy group is trivial, $H^2(1; -) = 0$, so i_x^* is the *zero map at every object*. It annihilates $[\omega_h]$ — indeed every class on $\mathrm{Sk}_{\mathcal{C}}$ — because the handoff generator is supported on the length-2 chain Sup $\to$ Wk $\to$ Rt through three *distinct* objects: an off-diagonal, nerve-level class, exactly the part isotropy restriction throws away. The stabiliser class, by contrast, is a pure loop class at one object. The two $\mathbb{F}_2$'s coincide numerically, *not* by any restriction map.

Nor does a change of viewpoint rescue the identification, and the collapse is two-sided. The classes live over *different base categories* — the handoff category $\mathrm{Sk}_{\mathcal{C}}$ (objects: roles) and the state-action category $\mathbb{A}(\downarrow_{\bowtie})$ (objects: states) — so relating them would need a common refinement bridging the two sites. No comparison functor $F\colon \mathbb{A}(\downarrow_{\bowtie}) \to \mathrm{Sk}_{\mathcal{C}}$ provides one: a functor sends loops to loops, $\mathrm{Sk}_{\mathcal{C}}$ has none but identities, so every pulled-back class is again isotropy-invisible ($i_s^* F^* [\omega_h] = 0$ for all F). In the cases one can check by hand — one-object group targets, comparison functors in either direction — the bridge is provably obstructed; the fully general many-object cospan is the one honest open edge. And the stabiliser site collapses from its own side: an aligned point-stabiliser with A normal is a semidirect product $E = A \rtimes B$, hence *split*, so $[\omega_{\mathrm{st}}] = 0$ wherever it is even defined. The nonzero $\mathbb{Z}/4$ row of the table above is thus the *abstract* extension classified by ε, not something aligned Zappa–Szép geometry realises — the stabiliser class is forced to 0 exactly where the naive identification wants it nonzero.

The discipline is one this book has had to learn before: two degree-2 obstructions with the same cyclic group and the same generator need *not* be the same class. Check the support. The handoff bit and the stabiliser bit are two images of one Zappa–Szép datum ε under two *incomparable* maps out of category cohomology — an off-diagonal (nerve) coordinate and a pure-isotropy coordinate — and i_x^* is precisely the projection that keeps the second and kills the first. It is a rhyme, not an identity; and knowing *why* it rhymes is better than mistaking it for a repetition. [MacBeth (`2026-08-14-two-omega-sites-isotropy-restriction.md`, Parts 1, 1.5, 2), resting on the isotropy-restriction map of Baues–Wirsching [23]. Open edge: the fully general many-object cospan $\mathrm{Sk}_{\mathcal{C}} \to \mathcal{C} \leftarrow \mathbb{A}(\downarrow_{\bowtie})$.]

What this does to the table. It does not collapse it. The three obstructions of §7.4 — a Baues–Wirsching class, nothing at all, a branching flag — remain three different kinds of mathematics, and there is still no master theorem that makes them one. What this section shows is subtler and, to the author, better: *two of the rows are not disjoint problems but one problem seen from two sides.* Composing two agents from the State row *is* a Zappa–Szép composition from the Directed row, and on that one object both invariants live at once — the isotropy holonomy of the state mode and the $[\omega]$ of the directed mode, meeting at the point-stabiliser. The table's rows are axes, not walls; a single composition can run along two of them, and when it does, the modes talk to each other.

For the grant. Here is the earned sentence, the one the three composition chapters were built to reach: *unprotected orchestration can synthesise holonomy the parts lack, and a degree-2 cohomology class certifies when the composite is a clean product of the parts.* It comes with an operational reading a practitioner can use without any cohomology at all — to find where a composed system carries reentrancy its components do not, count the states where the two components' reachable-state orbits cross. Where they cross only trivially, the system

is holonomy-clean and the finer question of entanglement is a single computable bit. Where they cross more, orchestration has manufactured symmetry, and the audit has found it. Fair is foul: two reentrancy-free agents, composed, can make reentrancy — and the mathematics says exactly where, and exactly how much.

Chapter 8

Functors over the category of containers (Phase 2 — outline)

This chapter is a stub sketching Neil's proposed Phase 2 programme. The earlier chapters studied containers and the maps *between* them, and Chapter 6 built the two canonical endofunctors — the free monad and the cofree comonad. Phase 2 studies *endofunctors of* **Cont** in general: operations **Cont** $\to$ **Cont** (equivalently **Poly** $\to$ **Poly**) and the structures they carry.

What a functor Cont $\to$ Cont is. It assigns to each container a container, functorially in container morphisms. The programme is to identify the natural such operations and the (co)monad structures they support.

Candidate examples (to be developed, not yet results).

- *The derivative / one-hole-context functor* ∂: sends $S \triangleleft P$ to the container of "a shape with one position singled out". For lists, $\partial(\sum_n X^n)$ is the zipper. The combinatorial derivative is classical for species; its container-level naturality and its interaction with $\triangleleft$ are the targets here.
- *The free monad and cofree comonad functors* on $(\mathbf{Poly}, \triangleleft)$: the syntax/behaviour pair. These are no longer stubs — they are developed as results in their own right in Chapter 6, and are recalled here only as the leading examples of the Phase 2 programme.

Appendix A

Reading & citation tracker

The status column records how thoroughly MacBeth has engaged each source: “read in full” (worked through, definitions and key proofs internalised), “skimmed” (read for the relevant statement), or “to-read”.

Paper	arXiv / ref	Status	Used in
Abbott–Altenkirch–Ghani, *Containers: constructing strictly positive types* (2005)	TCS 342	read in full	Ch. 1–3
Abbott–Altenkirch–Ghani, *Categories of containers* (2003)	FoSSaCS, LNCS 2620	Prop. 4.3/Ex. 4.4 verified	Ch. 3
Gambino–Kock, *Polynomial functors and polynomial monads* (2009/13)	arXiv:0906.4931	read in full	Ch. 3
Carboni–Johnstone, *Connected limits, familial representability, Artin glueing* (1995)	MSCS 5	read in full	Ch. 3
Diers, *Catégories localisables* (1977)	Thèse, Paris VI	skimmed	Ch. 3
Ahman–Chapman–Uustalu, *When is a container a comonad?* (2014)	LMCS 10(3)	read in full	Ch. 5
Ahman–Uustalu, *Directed containers as categories* (2016)	arXiv:1604.01187	read in full	Ch. 5
Ahman–Bauer, *Comodule representations of second-order functionals* (2024)	arXiv:2409.17664	read in full	Ch. 7
Le–Siqueira–Kremnitzer, *Free PLTL algebras & hyperdoctrines* (2025)	Topos Inst. blog	read in full	Ch. 7
Spivak, *Functorial aggregation / Polynomial functors* (2021)	arXiv:2111.10968	read in full	Ch. 4–5
Niu–Spivak, *Polynomial Functors: A Mathematical Theory of Interaction* (2023)	arXiv:2312.00990	read in full	Ch. 6
Spivak, *A reference for categorical structures on* Poly (2022)	arXiv:2202.00534	read in full	Ch. 6
Shapiro–Spivak, *Structures on categories of polynomials* (2023)	arXiv:2305.00167	read in full	Ch. 5
Shapiro–Spivak, *A polynomial construction of nerves* (2024)	arXiv:2405.13157	skimmed	Ch. 5
Clarke, *Internal lenses as functors and cofunctors* (2020)	arXiv:2009.06835	read in full	Ch. 5
Clarke, *Delta lenses as coalgebras for a comonad* (2022)	arXiv:2108.00390	read in full	Ch. 5
Rosebrugh–Wood, *Distributive laws and factorization* (JPAA 175, 2002)	JPAA 175	read in full	Ch. 6
Baues–Wirsching, *Cohomology of small categories* (1985)	JPAA 38, 187–211	read in full	Ch. 6
(modern BW account) (2025)	arXiv:2508.00727	skimmed	Ch. 6
Aguiar, *Internal categories and quantum groups* (1997)	PhD, Cornell	skimmed	Ch. 5
Uustalu–Vene, *Comonadic notions of computation* (2008)	ENTCS 203(5)	read for store comonad	Ch. 7

Bibliography

[1] M. Abbott, T. Altenkirch, N. Ghani. *Containers: constructing strictly positive types*. Theoretical Computer Science **342** (2005) 3–27.

[2] M. Abbott, T. Altenkirch, N. Ghani. *Categories of containers*. Proc. FoSSaCS 2003, LNCS **2620**, 23–38.

[3] N. Gambino, J. Kock. *Polynomial functors and polynomial monads*. arXiv:0906.4931; Math. Proc. Cambridge Philos. Soc. **154** (2013) 153–192.

[4] A. Carboni, P. Johnstone. *Connected limits, familial representability and Artin glueing*. Mathematical Structures in Computer Science **5** (1995) 441–459. (Corrigenda: *MSCS* **14** (2004) 185–187.)

[5] Y. Diers. *Catégories localisables*. Thèse de doctorat, Université Paris VI, 1977.

[6] D. Ahman, J. Chapman, T. Uustalu. *When is a container a comonad?* Logical Methods in Computer Science **10**(3), 2014.

[7] D. Ahman, T. Uustalu. *Directed containers as categories*. Proc. MSFP 2016, EPTCS **207**, 89–98. arXiv:1604.01187.

[8] D. Ahman, T. Uustalu. *Update monads: cointerpreting directed containers*. In *19th Int. Conf. on Types for Proofs and Programs (TYPES 2013)*, LIPIcs **26**, Schloss Dagstuhl, 2014, pp. 1–23.

[9] E. ter Horst, S. Mahadevan, J. D. Zambrano. *Categorical belief propagation: sheaf-theoretic inference via descent and holonomy*. arXiv:2601.04456 (2026).

[10] D. Ahman, A. Bauer. *Comodule representations of second-order functionals*. arXiv:2409.17664; J. Log. Algebraic Methods Program. **146** (2025).

[11] Q. Le, J. Siqueira, K. Kremnitzer. *Free PLTL algebras and a coalgebraic extension of hyperdoctrines*. Topos Institute Blog, 26 September 2025. topos.institute.

[12] D. I. Spivak. *Functorial aggregation* (polynomial functors). arXiv:2111.10968, 2021.

[13] N. Niu, D. I. Spivak. *Polynomial Functors: A Mathematical Theory of Interaction*. arXiv:2312.00990, 2023.

[14] D. I. Spivak. *A reference for categorical structures on* Poly. arXiv:2202.00534, 2022.

[15] B. Dorta, M. Jarvis, N. Niu. *Monoidal structures on generalized polynomial categories*. arXiv:2305.05655, 2023.

[16] G. De Pascalis, T. Uustalu, N. Veltrì. *Monoid structures on indexed containers*. arXiv:2509.25879, 2025.

[17] B. Shapiro, D. I. Spivak. *Structures on categories of polynomials*. arXiv:2305.00167, 2023.

[18] B. Shapiro, D. I. Spivak. *A polynomial construction of nerves for higher categories*. arXiv:2405.13157, 2024.

[19] B. Clarke. *Internal lenses as functors and cofunctors*. Proc. ACT 2019, EPTCS **323**, 183–195. arXiv:2009.06835.

[20] B. Clarke. *Delta lenses as coalgebras for a comonad*. arXiv:2108.00390, 2022.

[21] M. Aguiar. *Internal categories and quantum groups*. PhD thesis, Cornell University, 1997.

[22] R. Rosebrugh, R. J. Wood. *Distributive laws and factorization*. Journal of Pure and Applied Algebra **175** (2002) 327–353.

[23] H.-J. Baues, G. Wirsching. *Cohomology of small categories*. Journal of Pure and Applied Algebra **38** (1985) 187–211.

[24] *(Modern account of Baues–Wirsching cohomology)*. arXiv:2508.00727, 2025.

[25] T. Uustalu, V. Vene. *Comonadic notions of computation*. Electronic Notes in Theoretical Computer Science **203**(5) (2008) 263–284.

[26] M. Capucci, D. J. Myers. *Contextads as wreaths; Kleisli, Para, and Span constructions as wreath products*. arXiv:2410.21889, 2024.

[27] M. Capucci, B. Gavranović, J. Hedges, E. F. Rischel. *Towards foundations of categorical cybernetics*. Proc. ACT 2021, EPTCS **372**, 235–248; see also B. Gavranović, *Fundamental components of deep learning*, PhD thesis, 2024.

[28] D. I. Spivak. *Categories by Kan extension*. arXiv:2503.21974, 2025.

[29] J. Beck. *Distributive laws*. In: Seminar on Triples and Categorical Homology Theory, Lecture Notes in Mathematics **80**, Springer (1969) 119–140.

[30] J. Power, H. Watanabe. *Combining a monad and a comonad*. Theoretical Computer Science **280** (2002) 137–162.

[31] T. Brzeziński, S. Majid. *Coalgebra bundles*. Communications in Mathematical Physics **191** (1998) 467–492.

[32] S. Katsumata, E. Rivas, T. Uustalu. *Interaction laws of monads and comonads*. Proc. FoSSaCS 2020, LNCS **12077**, 356–375; arXiv:1912.13477.

[33] T. von Glehn. *Polynomials and models of type theory*. PhD thesis, University of Cambridge, 2015; see also Theory and Applications of Categories **33** (2018).

[34] T. Streicher. *Fibered Categories à la Jean Bénabou*. Lecture notes; arXiv:1801.02927 (orig. 1999).

[35] B. Jacobs. *Categorical Logic and Type Theory*. Studies in Logic and the Foundations of Mathematics **141**, Elsevier, 1999.

[36] R. Street. *The formal theory of monads*. Journal of Pure and Applied Algebra **2** (1972) 149–168.

[37] C. Hermida. *Fibrations, logical predicates and indeterminates*. PhD thesis, University of Edinburgh, 1993.

[38] C. Hermida, B. Jacobs. *Structural induction and coinduction in a fibrational setting.* Information and Computation **145**(2) (1998) 107–152. DOI 10.1006/inco.1998.2725.

[39] Orestis. *Effects* — an Agda formalisation of monad and comonad liftings to containers. Unpublished; personal communication via N. Ghani, 2026. Module `Containers.Effects` (files `CoLift.agda`, `Lift.agda`, `BiLift.agda`).

[40] S. Katsumata. *Relating computational effects by* $\top\top$*-lifting.* Information and Computation **222** (2013) 228–246.

[41] Pradic, Price. *Weihrauch problems as containers.* arXiv:2501.17250, 2025.

[42] A. D. Fairbanks. Answer to *"Examples of non-polynomial comonads on* Set*?"* MathOverflow, question 457580, answer posted 12 January 2025. mathoverflow.net/a/485788.